

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÙI VĂN DŨNG

LUẬN VĂN THẠC SĨ

PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO
BẢO MẬT MẠNG IoT

INTRUSION DETECTION (IDS) BASED ON APACHE SPARK FOR IoT
NETWORK SECURITY

THẠC SĨ NGÀNH CÔNG NGHỆ THÔNG TIN

TP. HỒ CHÍ MINH, 2026

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



BÙI VĂN DŨNG – 220201014

LUẬN VĂN THẠC SĨ

**PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO
BẢO MẬT MẠNG IoT**

**INTRUSION DETECTION (IDS) BASED ON APACHE SPARK FOR IoT
NETWORK SECURITY**

THẠC SĨ NGÀNH CÔNG NGHỆ THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN

TS. NGUYỄN VĂN DŨ

TS. ĐỖ TRÍ NHỰT

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG BẢO VỆ LUẬN VĂN THẠC SĨ

Hội đồng chấm luận văn thạc sĩ, thành lập theo Quyết định số ngày
..... của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

1. Chủ tịch:

2. Thư ký:

3. Ủy viên:

LỜI CẢM ƠN

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành đến quý thầy cô của Trường Đại học Công Nghệ Thông Tin, Đại học Quốc gia Thành Phố Hồ Chí Minh đã tận tình chỉ dạy, giúp tôi tiếp thu được nhiều kiến thức quý báu trong suốt thời gian học tập tại trường, cũng như tạo điều kiện thuận lợi cho tôi thực hiện luận văn này. Kính chúc thầy cô luôn dồi dào sức khỏe và gặt hái được nhiều thành công trong tương lai.

Đặc biệt, tôi xin chân thành cảm ơn thầy TS. Nguyễn Văn Dũ và TS. Đỗ Trí Nhật đã tận tình giúp đỡ, hướng dẫn, cung cấp cho tôi những kiến thức, kỹ năng cần thiết để thực hiện nghiên cứu và hoàn thành đề tài này.

Mặc dù đã cố gắng trong quá trình nghiên cứu, nhưng do kiến thức còn hạn chế nên luận văn vẫn còn nhiều thiếu sót. Vì vậy, tôi rất mong nhận được sự đóng góp ý kiến của Quý Thầy Cô để đề tài của tôi được hoàn thiện hơn.

Tôi xin chân thành cảm ơn!

Bùi Văn Dũng

TÓM TẮT

Sự phát triển nhanh chóng của Internet of Things (IoT) đã mang lại nhiều tiện ích trong các lĩnh vực như nhà thông minh, công nghiệp, y tế và giao thông. Tuy nhiên, cùng với sự phát triển này là nguy cơ ngày càng gia tăng về các mối đe dọa an ninh mạng, đặc biệt là khi các thiết bị IoT thường có ít tài nguyên tính toán và bảo mật kém. Trong bối cảnh đó, Intrusion Detection Systems (IDS) được xem là một giải pháp quan trọng nhằm phát hiện và ngăn chặn các hành vi tấn công bất thường vào hệ thống mạng IoT.

Luận văn này tập trung nghiên cứu, thiết kế và triển khai một hệ thống IDS dựa trên nền tảng xử lý dữ liệu lớn Apache Spark. Spark có khả năng xử lý dữ liệu phân tán, tốc độ cao và hỗ trợ hiệu quả các thuật toán học máy qua thư viện MLlib. Hệ thống được xây dựng nhằm phát hiện các hành vi xâm nhập trong dữ liệu mạng IoT thông qua việc ứng dụng các mô hình học máy truyền thống, các mô hình học tổ hợp và mô hình học sâu MLP (cùng một autoencoder đóng vai trò cổng phát hiện bất thường ở tầng biên).

Trong quá trình thực hiện, luận văn đã tiến hành phân tích đặc điểm dữ liệu mạng IoT, lựa chọn và xử lý dữ liệu đầu vào từ các tập dữ liệu công khai như CICIDS2017, sau đó tiến hành huấn luyện mô hình, đánh giá hiệu quả hệ thống theo các chỉ số chính như độ chính xác (Accuracy), precision, recall và F1-score, đồng thời ghi lại thời gian huấn luyện và kích thước mô hình. Quá trình đánh giá áp dụng quy trình loại trừ rõ ràng dữ liệu (loại đặc trưng rõ ràng, loại bỏ trùng lặp ở mức đặc trưng) nhằm bảo đảm kết quả phản ánh đúng năng lực mô hình. Bên cạnh đó, hệ thống được kiểm thử hiệu năng trong môi trường xử lý phân tán (proof-of-concept trên cụm thiết bị biên) nhằm khảo sát khả năng mở rộng và tính khả thi triển khai thực tiễn.

Dưới cấu hình đánh giá loại trừ rõ ràng dữ liệu, lựa chọn đặc trưng SHAP Top-30 giữ được hiệu năng gần tương đương cấu hình đầy đủ trong khi giảm khoảng 60% số đặc trưng: Random Forest đạt F1 0,9955 và XGBoost (XGB) đạt 0,9970 trên CICIDS2017; các khác biệt giữa phương pháp được kiểm chứng bằng kiểm định hoán vị ghép cặp và khoảng tin cậy hiệu chỉnh Nadeau–Bengio. Học tổ hợp (Hybrid Bagging 0,9966) không vượt được mô hình đơn tốt nhất trên cấu hình này, trong khi chi phí huấn luyện và dung lượng mô hình cao hơn nhiều lần. Trên cụm hai thiết bị biên Jetson Orin Nano Super, kiến trúc tách pipeline (cổng phát hiện bất thường + bộ phân loại PySpark) nâng thông lượng phán quyết từ 2,6 lên 62,5 luồng/giây (gấp ≈ 24 lần) với độ trễ p95 xấp xỉ 13 ms, trong đó cổng lọc 95,8% lưu lượng bình thường trước khi gọi suy luận Spark. Khía cạnh xử lý phân tán được

kiểm chứng ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên, thay vì khẳng định quy mô dữ liệu lớn thực thụ. Luận văn không chỉ góp phần vào việc xây dựng quy trình phát hiện xâm nhập theo thời gian thực mà còn mở ra các hướng nghiên cứu tiềm năng trong việc kết hợp học máy với các nền tảng xử lý dữ liệu lớn phục vụ bảo mật hệ thống IoT.

MỤC LỤC

Thông tin hội đồng bảo vệ luận văn	i
Lời cảm ơn	ii
Tóm tắt	iii
Mục lục	v
Danh mục các bảng	viii
Danh mục các hình vẽ và đồ thị	ix
Danh mục từ viết tắt	x
Chương 1. Mở đầu	1
1.1 Lý do chọn đề tài	1
1.2 Các nghiên cứu liên quan	1
1.3 Mục tiêu, đối tượng và phạm vi nghiên cứu	4
1.3.1 Mục tiêu nghiên cứu	4
1.3.2 Đối tượng nghiên cứu	5
1.3.3 Phạm vi nghiên cứu	5
1.3.4 Câu hỏi nghiên cứu	6
1.4 Phương pháp nghiên cứu	7
1.5 Các đóng góp chính của đề tài	7
1.6 Công trình khoa học đã công bố	8
1.7 Cấu trúc của luận văn	9
Chương 2. Cơ sở lý thuyết	11
2.1 Tổng quan về IoT	11
2.2 Hệ thống phát hiện xâm nhập (IDS)	11
2.3 Apache Spark và xử lý dữ liệu lớn	12
2.3.1 Dữ liệu lớn	12
2.3.2 Các thành phần của Apache Spark	14
2.3.3 Kiến trúc của Apache Spark	15

MỤC LỤC

2.4	Học máy trong phát hiện xâm nhập	16
2.4.1	Một số thuật toán học máy	17
2.4.2	Một số thuật toán học tổ hợp	21
Chương 3.	Phương pháp thực hiện	24
3.1	Thu thập dữ liệu và tiền xử lý	24
3.2	Lựa chọn đặc trưng và giảm chiều dữ liệu	29
3.3	Huấn luyện mô hình	31
3.3.1	Huấn luyện các mô hình cơ sở	31
3.3.2	Các phương pháp Ensemble Learning	32
a)	Áp dụng Voting	32
b)	Áp dụng Bagging	34
3.4	Đánh giá mô hình	36
3.5	Tối ưu hóa tham số và huấn luyện mô hình	38
3.6	Triển khai biên	40
Chương 4.	Thực nghiệm, đánh giá và thảo luận	41
4.1	Môi trường thực nghiệm	41
4.2	Kết quả thực nghiệm	42
4.2.1	Kịch bản 1: Đánh giá hiệu năng cơ sở (Baseline)	45
4.2.2	Kịch bản 2: So sánh chiến lược giảm chiều dữ liệu	45
4.2.3	Kịch bản 3: Tối ưu hóa tham số	46
4.2.4	Phân tích khả năng giải thích với SHapley Additive exPlanations (SHAP)	47
4.2.5	Phân tích rò rỉ dữ liệu (data leakage)	48
4.2.6	Phân tích hiệu năng hệ thống	49
4.2.7	Kiểm định ý nghĩa thống kê	50
4.2.8	Kiểm tra độ ổn định trong-phân-phối	51
4.2.8.1	Tổng quát hóa liên bộ dữ liệu	51
4.2.9	Đánh giá đa lớp theo loại tấn công	53
4.3	Thực nghiệm và đánh giá hiệu năng mô hình trên Jetson Orin Nano Super	58
4.3.1	Các công cụ sử dụng trong hệ thống	58
4.3.2	Mô hình hệ thống đề xuất	60
4.3.3	Lựa chọn mô hình cho thiết bị biên	62

MỤC LỤC

4.3.4	Kết quả huấn luyện các mô hình ứng viên trên cụm Spark	63
4.3.5	Kết quả đo hiệu năng (benchmark) trên Jetson Orin Nano Super . . .	67
4.3.6	So sánh engine suy luận: chi phí của việc đồng nhất stack	68
4.3.7	Kết quả ba chế độ triển khai phân tán	69
4.3.8	Phân tích và lựa chọn mô hình	71
4.4	Thảo luận và So sánh tổng hợp	72
Chương 5. Kết luận và hướng phát triển		74
5.1	Những đóng góp chính của luận văn	74
5.2	Hạn chế của đề tài	75
5.3	Hướng phát triển trong tương lai	75
Phụ lục A. Công trình khoa học đã công bố		77
Tài liệu tham khảo		85

DANH MỤC CÁC BẢNG

3.1	Mô tả bộ dữ liệu CICIDS2017	25
3.2	Mô tả bộ dữ liệu CSE-CIC-IDS2018 (dùng cho thí nghiệm liên bộ dữ liệu) .	26
3.3	Tám bộ phân loại cơ sở được sử dụng trong pipeline	31
3.4	Các phương pháp học tổ hợp (ensemble) sử dụng trong luận văn	32
3.5	Các tiêu chí đánh giá mô hình	36
3.6	Ma trận nhầm lẫn của mô hình phát hiện xâm nhập	37
3.7	Thông số phần cứng thiết bị biên	40
4.1	Tham số tối ưu của các mô hình học máy	43
4.2	Hiệu năng phân lớp kịch bản Baseline	45
4.3	So sánh hiệu năng tốt nhất của các phương pháp giảm chiều	46
4.4	So sánh lựa chọn đặc trưng: RF Importance vs SHAP (Top-30)	47
4.5	Kết quả hiệu năng phân lớp kịch bản Tối ưu hóa tham số	47
4.6	Phân tích loại bỏ (ablation) rò rỉ đặc trưng cổng đích	49
4.7	So sánh thời gian xử lý và kích thước mô hình kịch bản Baseline	49
4.8	Độ ổn định qua nhiều lần lấy mẫu và kiểm định ghép cặp	51
4.9	Kiểm tra độ ổn định trong-phân-phối trên mẫu con tập kiểm tra	51
4.10	Tổng quát hóa liên bộ dữ liệu CICIDS2017 và CSE-CIC-IDS2018	53
4.11	Hiệu năng theo từng lớp tấn công	54
4.12	Điểm vận hành của cổng bất thường theo ngưỡng quantile	62
4.13	Các mô hình ứng viên cho triển khai biên (SHAP Top-30)	63
4.14	So sánh hiệu năng triển khai trên Jetson Orin Nano Super	68
4.15	Chi phí suy luận một nút: Spark, sklearn và ONNX	68
4.16	So sánh ba chế độ triển khai phân tán trên cụm 2 Jetson Orin Nano Super .	69

DANH MỤC CÁC HÌNH VẼ VÀ ĐỒ THỊ

2.1	Mô hình 5V của dữ liệu lớn	13
2.2	Các thành phần của Apache Spark	14
2.3	Sơ đồ kiến trúc hệ thống Apache Spark	15
3.1	Quy trình tổng thể huấn luyện, đánh giá và triển khai mô hình đề xuất	24
3.2	Sơ đồ phân chia dữ liệu CICIDS2017	27
4.1	Cụm huấn luyện phân tán Apache Spark Standalone	44
4.2	Top 30 đặc trưng quan trọng nhất theo SHAP (mô hình XGBoost)	55
4.3	Biểu đồ SHAP Summary (beeswarm)	56
4.4	Ma trận nhầm lẫn đa lớp theo loại tấn công (chuẩn hóa theo hàng)	57
4.5	Kiến trúc hệ thống IDS phân tán trên cụm 2 Jetson Orin Nano Super	60
4.6	Giao diện terminal của pipeline IDS trên hai node Jetson	64
4.7	Dashboard Grafana giám sát hiệu năng IDS theo thời gian thực	65
4.8	Email cảnh báo tấn công gửi qua Mailtrap	65
4.9	Dashboard Grafana chi tiết các cuộc tấn công được phát hiện	66
4.10	Dashboard Grafana tại thời điểm tải đỉnh của chế độ tách pipeline	66
4.11	Thông báo cảnh báo tấn công gửi qua Slack	67
4.12	Thông lượng và độ trễ p95 của bốn cấu hình triển khai	69
4.13	So sánh hiệu năng ba mô hình trên Jetson Orin Nano Super	71
4.14	Biểu đồ radar đánh đổi giữa các mô hình trên thiết bị biên	72

DANH MỤC TỪ VIẾT TẮT

AC	accuracy
BD	Big Data
CNN	Convolutional Neural Network
CV	Cross-Validation
DT	Decision Trees
EL	Ensemble Learning
FS	F1-score
GBT	Gradient Boosted Trees
GS	Grid Search
IDS	Intrusion Detection Systems
IoT	Internet of Things
kNN	k-Nearest Neighbors
LGBM	LightGBM
LR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
MLP	Multilayer Perceptron
NB	Naive Bayes
PCA	Principal Component Analysis
PR	precision
RE	recall
RF	Random Forest
RFI	Random Forest Importance
RNN	Recurrent Neural Network
SCP	Secure Copy Protocol
SHAP	SHapley Additive exPlanations
SVM	Support Vector Machine
XGB	XGBoost

Chương 1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong kỷ nguyên của Cách mạng Công nghiệp 4.0, IoT đang phát triển nhanh và được ứng dụng rộng rãi trong nhiều lĩnh vực như nhà thông minh, y tế, giao thông, công nghiệp thông minh,... Sự bùng nổ về số lượng thiết bị IoT dẫn đến sự gia tăng đáng kể về lưu lượng và loại hình dữ liệu truyền trong mạng. Tuy nhiên, các thiết bị IoT thường có khả năng xử lý hạn chế và thiếu các cơ chế bảo mật mạnh, khiến chúng trở thành mục tiêu dễ bị tấn công bởi tin tặc.

Để đối phó với các mối đe dọa an ninh mạng, các hệ thống phát hiện xâm nhập (IDS) đang được nghiên cứu và phát triển rộng rãi. Đặc biệt, trong những năm gần đây, việc kết hợp học máy (Machine Learning (ML)) với các nền tảng xử lý dữ liệu lớn giúp hệ thống nâng cao độ chính xác phát hiện và thích nghi tốt hơn với môi trường mạng thay đổi liên tục. Cần lưu ý rằng bộ phân loại chính trong luận văn là mô hình học có giám sát trên CICIDS2017 nên chỉ phát hiện được các họ tấn công đã biết; khả năng phát hiện hành vi bất thường chưa từng thấy (nếu có) đến từ cổng phát hiện bất thường (anomaly gate) học không giám sát, chứ không phải từ bộ phân loại. Tuy nhiên, phần lớn các nghiên cứu hiện nay tập trung vào việc xây dựng mô hình học máy trên tập dữ liệu tĩnh, dung lượng nhỏ và huấn luyện ngoại tuyến (offline).

Trong khi đó, mạng IoT hiện đại sinh ra lượng dữ liệu rất lớn, liên tục theo thời gian thực đòi hỏi các hệ thống IDS có khả năng xử lý dữ liệu lớn (Big Data (BD)) hiệu quả, có thể hoạt động trong môi trường phân tán và thời gian thực.

Apache Spark là một nền tảng xử lý dữ liệu lớn, với khả năng xử lý song song, phân tán và dễ dàng tích hợp với thư viện học máy Spark MLlib. Đây là một giải pháp tiềm năng để xây dựng hệ thống IDS hiện đại đáp ứng yêu cầu về quy mô và thời gian thực trong môi trường IoT.

1.2 Các nghiên cứu liên quan

Trong những năm gần đây, các hệ thống phát hiện xâm nhập (Intrusion Detection System – IDS) dựa trên học máy và học sâu đã thu hút nhiều sự quan tâm của cộng đồng nghiên cứu nhằm nâng cao độ chính xác và khả năng thích ứng trước các hình thức tấn công mạng ngày càng tinh vi [1, 2, 3]. Nhiều công trình tập trung khai thác các kiến trúc học sâu như

autoencoder, mạng tích chập (Convolutional Neural Network (CNN)) và mạng nơ-ron hồi quy (Recurrent Neural Network (RNN), Long Short-Term Memory (LSTM)) để tự động trích xuất đặc trưng và phát hiện bất thường trong lưu lượng mạng. Tuy nhiên, phần lớn các nghiên cứu hiện nay vẫn được triển khai trong môi trường đơn máy, xử lý dữ liệu tĩnh với quy mô tương đối nhỏ, và chưa tích hợp các nền tảng xử lý dữ liệu lớn như Apache Spark hay Hadoop nhằm đáp ứng yêu cầu về khả năng mở rộng và xử lý thời gian thực trong các hệ thống mạng hiện đại.

Một số nghiên cứu đã đề xuất các mô hình IDS học sâu dựa trên các kiến trúc như autoencoder, CNN và LSTM nhằm nâng cao khả năng học đặc trưng và phát hiện xâm nhập mạng. Các mô hình này được đánh giá trên các tập dữ liệu chuẩn như NSL-KDD, CIC-DDoS2019 và CICIDS2017, cho thấy hiệu quả cao trong việc phát hiện bất thường. Tuy nhiên, quá trình huấn luyện và đánh giá vẫn chủ yếu được thực hiện trong môi trường cục bộ và theo cơ chế xử lý theo lô (batch) [4, 5]. Tương tự, các mô hình lai dựa trên LSTM (chẳng hạn CNN-LSTM, hay LSTM được tối ưu hóa siêu tham số) cho bài toán phát hiện bất thường trong mạng cũng đạt được kết quả khả quan, nhưng vẫn triển khai trên nền tảng đơn nút (single-node) và chưa khai thác được khả năng mở rộng của các hệ thống Big Data [6, 7]. Gần đây, một số nghiên cứu đã triển khai IDS học máy trực tiếp trên phần cứng biên NVIDIA Jetson [8, 9], hay trên môi trường biên mô phỏng ràng buộc tài nguyên [10]. Xu hướng này tiếp tục được mở rộng trong các công trình mới nhất: mô hình sinh (generative) được chạy trực tiếp trên Jetson Orin Nano để vừa phân loại lưu lượng vừa sinh dữ liệu huấn luyện tổng hợp [11], các mô hình rút gọn được thiết kế riêng cho lưu lượng điều khiển công nghiệp [12], và năng lượng tiêu thụ trên mỗi lần suy luận bắt đầu được xem là một chỉ số đánh giá hạng nhất bên cạnh độ chính xác [13]. Tuy vậy, điểm chung của các nghiên cứu này là vẫn dừng ở quy mô *đơn nút*: bộ phát hiện được đo trên một bo mạch duy nhất, nên câu hỏi phân bổ tải suy luận trên nhiều nút biên và lợi ích thực tế của việc bổ sung nút thứ hai vẫn còn bỏ ngỏ. Ở hướng ngược lại, việc ghép Kafka với Spark để phát hiện xâm nhập phân tán đã được nghiên cứu [14], nhưng chủ yếu trên hạ tầng máy chủ và hiếm khi công bố vết tài nguyên (CPU, bộ nhớ, năng lượng) theo từng nút trên phần cứng ràng buộc.

Bên cạnh các phương pháp học sâu, các kỹ thuật học máy truyền thống như Support Vector Machine (Support Vector Machine (SVM)), k-Nearest Neighbors (k-Nearest Neighbors (kNN)) và Decision Tree vẫn được sử dụng rộng rãi trong các hệ thống IDS. Một số nghiên cứu đã tiến hành so sánh hiệu năng của các mô hình này trên các bộ dữ liệu phổ biến như NSL-KDD và UNSW-NB15, qua đó chỉ ra rằng các phương pháp học máy truyền

thống có tiềm năng ứng dụng thực tiễn cao. Tuy nhiên, các mô hình này chủ yếu hoạt động ở chế độ offline, xử lý dữ liệu theo từng đợt và chưa đáp ứng được yêu cầu về xử lý dữ liệu liên tục trong môi trường mạng động [15]. Ngoài ra, một số công trình so sánh nhiều mô hình học máy và học sâu cho IDS, ghi nhận các mô hình dựa trên cây như Random Forest cho kết quả tốt trên các bộ dữ liệu lớn, nhưng toàn bộ thực nghiệm vẫn được thực hiện trên một máy đơn [16].

Ngoài các kiến trúc nêu trên, nhiều mô hình học sâu khác như CNN, RNN và các mô hình lai CNN-LSTM cũng đã được nghiên cứu và đánh giá cho bài toán phát hiện xâm nhập mạng. Các nghiên cứu này cho thấy khả năng cải thiện đáng kể độ chính xác so với các phương pháp truyền thống khi thử nghiệm trên các bộ dữ liệu như CICIDS2018. Tuy nhiên, phần lớn các mô hình vẫn thiếu khả năng triển khai trong môi trường thời gian thực và chưa tận dụng được sức mạnh của các nền tảng xử lý dữ liệu lớn [17].

Trong bối cảnh Internet of Vehicles (IoV), một số nghiên cứu đã đề xuất các hệ thống IDS dựa trên học máy và học sâu nhằm bảo vệ các mạng phương tiện thông minh. Các phương pháp này bao gồm việc sử dụng các mô hình dựa trên cây (tree-based), các hệ thống phát hiện xâm nhập đa tầng, cũng như các mô hình ensemble để cải thiện độ chính xác phát hiện tấn công. Mặc dù đạt được kết quả khả quan trên các bộ dữ liệu thử nghiệm, các hệ thống này vẫn chủ yếu được huấn luyện và đánh giá trong môi trường ngoại tuyến, xử lý dữ liệu theo batch và chưa tích hợp các nền tảng xử lý dữ liệu lớn hoặc cơ chế xử lý dữ liệu theo luồng [18, 19, 20].

Bên cạnh đó, một số hướng tiếp cận gần đây đã nghiên cứu việc áp dụng học bán giám sát, học liên kết (federated learning) và học tăng dần (incremental/active learning) cho bài toán IDS nhằm nâng cao khả năng thích ứng của mô hình và bảo vệ quyền riêng tư dữ liệu. Các phương pháp này cho phép phân tán quá trình huấn luyện trên nhiều nút hoặc thiết bị IoT khác nhau; một khảo sát hệ thống gần đây đã tổng hợp toàn cảnh hướng nghiên cứu IDS dựa trên học liên kết [21]. Tuy nhiên, cần lưu ý rằng việc phân tán ở đây diễn ra trong pha *huấn luyện*, còn suy luận vẫn thực hiện cục bộ trên từng nút; ngoài ra dữ liệu vẫn chủ yếu được xử lý theo từng đợt, thiếu cơ chế xử lý luồng tập trung và chưa khai thác đầy đủ khả năng mở rộng cũng như song song hóa của các nền tảng Big Data [22, 23, 24, 25, 26].

Từ tổng quan các công trình nghiên cứu nêu trên, có thể nhận thấy rằng phần lớn các hệ thống IDS hiện tại vẫn hoạt động trong môi trường ngoại tuyến, chưa hỗ trợ xử lý dữ liệu liên tục theo thời gian thực và chưa tận dụng được khả năng mở rộng của các nền tảng xử lý dữ liệu lớn. Do đó, luận văn này hướng đến việc tích hợp các mô hình học máy dựa trên

nền tảng Apache Spark nhằm xây dựng một hệ thống phát hiện xâm nhập có khả năng xử lý dữ liệu lớn, hỗ trợ thời gian thực và áp dụng hiệu quả trong môi trường mạng IoT hiện đại.

1.3 Mục tiêu, đối tượng và phạm vi nghiên cứu

1.3.1 Mục tiêu nghiên cứu

Mục tiêu của đề tài “PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO BẢO MẬT MẠNG IoT” là xây dựng một hệ thống phát hiện xâm nhập có khả năng phân tích và phát hiện các hành vi bất thường hoặc tấn công mạng trong môi trường IoT bằng cách kết hợp các mô hình học máy với nền tảng xử lý dữ liệu lớn Apache Spark nhằm đạt được:

- Phát hiện hiệu quả các dạng tấn công mạng phổ biến (như DoS, probing, brute-force) trên các tập dữ liệu hiện đại như CICIDS2017.
- Thiết kế và triển khai pipeline xử lý dữ liệu phân tán và song song sử dụng Apache Spark, bao gồm các bước: tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình và phát hiện thời gian thực.
- Khảo sát hiệu năng hệ thống, hướng tới khả năng mở rộng, độ trễ thấp và tính linh hoạt; kiểm chứng khả năng xử lý phân tán ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên, làm tiền đề cho việc triển khai trên các hệ thống lớn hoặc mạng IoT nhiều thiết bị.
- So sánh và đánh giá các mô hình học máy phổ biến như Decision Tree, Support Vector Machine, Logistic Regression, Naive Bayes (NB), Random Forest, Gradient Boosted Trees (GBT), XGB, Multilayer Perceptron (MLP) và các thuật toán học tổ hợp (ensemble) trong môi trường xử lý phân tán, từ đó chọn ra mô hình phù hợp nhất.
- Áp dụng quy trình đánh giá chặt chẽ, loại trừ rò rỉ dữ liệu (loại đặc trưng rò rỉ nhãn, loại bỏ trùng lặp ở mức đặc trưng, kiểm định ý nghĩa thống kê) nhằm đảm bảo kết quả phản ánh đúng năng lực mô hình.
- Đề xuất giải pháp thực tiễn giúp phát hiện tấn công nhanh chóng và chính xác trong các hệ thống mạng hiện đại có yêu cầu xử lý dữ liệu lớn và liên tục.

1.3.2 Đối tượng nghiên cứu

Đề tài này tập trung nghiên cứu và triển khai phương pháp phát hiện xâm nhập trong lĩnh vực bảo mật mạng IoT bằng cách tận dụng sức mạnh của nền tảng xử lý dữ liệu lớn Apache Spark. Với sự phát triển nhanh chóng của các thiết bị IoT, vấn đề đảm bảo an toàn và bảo mật cho hệ thống ngày càng trở nên cấp thiết, đặc biệt là trong việc phát hiện các hành vi bất thường hoặc tấn công mạng tiềm ẩn.

Apache Spark, với khả năng xử lý dữ liệu phân tán và tốc độ tính toán cao, cho phép xử lý khối lượng lớn dữ liệu sinh ra từ các thiết bị IoT trong thời gian thực. Trong khuôn khổ nghiên cứu này, luận văn sử dụng các tính năng của Spark như xử lý luồng dữ liệu (streaming), học máy phân tán (MLlib) và phân tích dữ liệu quy mô lớn để xây dựng mô hình phát hiện xâm nhập hiệu quả. Qua đó, đề tài không chỉ đề xuất một giải pháp kỹ thuật có tính ứng dụng cao mà còn góp phần nâng cao hiệu quả bảo mật cho các hệ thống IoT trong thực tiễn.

1.3.3 Phạm vi nghiên cứu

Đề tài tập trung vào việc nghiên cứu và áp dụng các kỹ thuật phân tích dữ liệu lớn nhằm phát hiện xâm nhập trong hệ thống mạng IoT với mục đích là khai thác những điểm mạnh của nền tảng Apache Spark để xử lý và phân tích dữ liệu hiệu quả. Cụ thể, phạm vi nghiên cứu bao gồm:

- **Dữ liệu đầu vào:** Dữ liệu được sử dụng là tập dữ liệu công khai CICIDS2017.
- **Công nghệ sử dụng:** Sử dụng Apache Spark làm nền tảng xử lý dữ liệu lớn, đặc biệt tập trung vào các thành phần như Spark Core, Spark SQL, Spark Streaming và Spark MLlib để hỗ trợ tiền xử lý dữ liệu, huấn luyện mô hình học máy và phát hiện xâm nhập theo thời gian thực.
- **Phương pháp phát hiện xâm nhập:** Áp dụng các thuật toán học máy được triển khai và tối ưu hóa trong môi trường Spark để đảm bảo khả năng mở rộng và hiệu năng cao.
- **Môi trường triển khai:** Thử nghiệm hệ thống trên một môi trường giả lập mạng IoT và bộ dữ liệu mô phỏng công khai (CICIDS2017) nhằm đánh giá khả năng phát hiện xâm nhập.

Lưu ý về phạm vi dữ liệu. Cần nhấn mạnh rằng CICIDS2017 là bộ dữ liệu lưu lượng mạng doanh nghiệp mô phỏng, không chứa lưu lượng đặc thù IoT (ví dụ các giao thức MQTT, CoAP) hay thiết bị IoT thực. Tính chất “IoT” của đề tài do đó nằm ở *kịch bản triển khai*: toàn bộ pipeline suy luận được thiết kế, tối ưu và đo đạc trên thiết bị biên tài nguyên hạn chế (NVIDIA Jetson Orin Nano Super), lớp thiết bị đóng vai trò gateway/biên trong kiến trúc IoT ba tầng — thay vì ở bản chất lưu lượng của dữ liệu huấn luyện. CICIDS2017 được chọn vì là bộ chuẩn công khai, được cộng đồng sử dụng rộng rãi, có nhãn chi tiết và bao phủ các họ tấn công (DoS/DDoS, brute-force, web attack, botnet) cũng xuất hiện phổ biến trong các mạng IoT; các nguồn lỗi của bộ dữ liệu này đã được nghiên cứu kỹ, cho phép áp dụng quy trình loại trừ rõ ràng một cách có căn cứ. Việc kiểm chứng trên các bộ dữ liệu IoT chuyên biệt (Bot-IoT, TON_IoT, CIC-IoT-2023) được xác định là hướng phát triển tiếp theo (Chương 5).

Mô hình mối đe dọa (threat model). Luận văn giả định kẻ tấn công nằm bên trong hoặc ở biên mạng IoT và tìm cách né tránh phát hiện; phạm vi bảo vệ giới hạn ở việc phát hiện các tấn công đã biết theo kiểu CICIDS2017, dưới dạng phân loại nhị phân Benign/Attack. Các kịch bản đối kháng chủ động (adversarial/evasion) nhằm cố ý đánh lừa mô hình nằm ngoài phạm vi đánh giá của luận văn.

1.3.4 Câu hỏi nghiên cứu

Luận văn được định hướng bởi ba câu hỏi nghiên cứu:

- **RQ1:** Việc giảm chiều/lựa chọn đặc trưng (RF Importance, SHapley Additive exPlanations (SHAP), Principal Component Analysis (PCA)) có giữ được hiệu năng phát hiện gần với mức dùng toàn bộ đặc trưng hay không, khi số đặc trưng giảm hơn 60%?
- **RQ2:** Các mô hình học tổ hợp (Ensemble Learning (EL)) có mang lại lợi thế đáng kể so với mô hình đơn tốt nhất hay không, khi khác biệt được kiểm chứng bằng kiểm định thống kê thay vì so sánh một lần đo?
- **RQ3:** Một pipeline IDS dựa trên Spark có khả thi về thông lượng, độ trễ và năng lượng khi triển khai phân tán trên cụm thiết bị biên tài nguyên hạn chế (Jetson Orin Nano Super) hay không?

1.4 Phương pháp nghiên cứu

- **Thu thập, phân tích, xử lý dữ liệu:** Sử dụng tập dữ liệu CICIDS2017, từ đó tiến hành phân tích, xử lý và trực quan hóa dữ liệu.
- **Nghiên cứu về phương pháp tối ưu hóa siêu tham số, các phương pháp giảm chiều dữ liệu:** Nghiên cứu các phương pháp giảm chiều dữ liệu như PCA, SHAP và Random Forest Importance nhằm giảm nhiễu và tăng độ chính xác, đồng thời tối ưu hóa siêu tham số thông qua tìm kiếm lưới (Grid Search) và kiểm định chéo (cross-validation) giúp nâng cao hiệu năng mô hình. Các bước này được triển khai trên nền tảng Apache Spark nhằm đảm bảo khả năng xử lý dữ liệu lớn hiệu quả và mở rộng.
- **Nghiên cứu và đề xuất mô hình học máy, học sâu (MLP; và một autoencoder làm công phát hiện bất thường), học tổ hợp phát hiện xâm nhập sử dụng Spark Machine Learning:** Sử dụng nhiều thuật toán khác nhau như Decision Tree, Support Vector Machine, Logistic Regression, NB, Random Forest, GBT, XGB, MLP và các thuật toán học tổ hợp (ensemble) để xây dựng mô hình nhằm so sánh, đánh giá mô hình có hiệu năng tốt nhất.
- **Thực nghiệm và đánh giá hiệu năng mô hình trên cụm Jetson Orin Nano Super:** Xây dựng một hệ thống phát hiện xâm nhập mạng IDS *phân tán* triển khai trên cụm hai thiết bị biên Jetson Orin Nano Super (công phát hiện bất thường trên một nút, bộ phân loại PySpark trên nút còn lại) nhằm đánh giá khả năng hoạt động trong môi trường tài nguyên hạn chế. Hệ thống được thiết kế theo kiến trúc xử lý dữ liệu thời gian thực, tận dụng Apache Kafka cho cơ chế truyền tải thông điệp, Spark Streaming cho xử lý luồng dữ liệu, PostgreSQL để quản lý dữ liệu, InfluxDB để lưu trữ chỉ số hiệu năng theo thời gian, Grafana để trực quan hóa và Mailtrap phục vụ kiểm thử cơ chế cảnh báo qua email.

1.5 Các đóng góp chính của đề tài

- **Đề xuất giải pháp ứng dụng Apache Spark trong phát hiện xâm nhập cho hệ thống mạng IoT:** Đề tài đã xây dựng một mô hình phát hiện xâm nhập dựa trên nền tảng xử lý dữ liệu lớn Apache Spark, tận dụng khả năng xử lý phân tán và thời gian thực của Spark, hướng tới đáp ứng yêu cầu về hiệu năng và tính mở rộng trong môi trường IoT có khối lượng dữ liệu lớn và biến đổi liên tục; khả năng xử lý phân tán

được kiểm chứng ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên.

- **Xây dựng quy trình xử lý và phát hiện xâm nhập toàn diện trên dữ liệu IoT:** Từ bước tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình đến đánh giá kết quả, đề tài đã thiết kế và hiện thực một quy trình phát hiện xâm nhập hoàn chỉnh có thể áp dụng cho các hệ thống thực tế.
- **Ứng dụng và đánh giá các thuật toán học máy trong môi trường phân tán:** Đề tài triển khai và so sánh hiệu quả của một số thuật toán học máy và mô hình học sâu MLP trên nền tảng Apache Spark MLlib (ngoài ra một autoencoder được dùng làm công phát hiện bất thường ở tầng biên), qua đó làm rõ ưu điểm và hạn chế của từng mô hình trong việc xử lý dữ liệu IoT.
- **Đánh giá hiệu năng hệ thống trong môi trường xử lý phân tán:** Không chỉ đánh giá độ chính xác của mô hình, đề tài còn phân tích hiệu năng xử lý (thời gian thực thi, khả năng mở rộng) của hệ thống nhằm khảo sát tính khả thi của giải pháp; việc kiểm chứng được thực hiện ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên với bộ dữ liệu CICIDS2017, làm cơ sở cho việc mở rộng quy mô về sau.
- **Quy trình đánh giá loại trừ rò rỉ dữ liệu và có kiểm định thống kê:** Đề tài nhận diện và loại bỏ các nguồn rò rỉ dữ liệu thường gặp trên CICIDS2017 (đặc trưng `destination_port` rò rỉ nhãn, trùng lặp luồng ở mức đặc trưng), đồng thời áp dụng kiểm định ý nghĩa thống kê, giúp kết quả phản ánh đúng năng lực mô hình thay vì các con số bị thổi phồng.

1.6 Công trình khoa học đã công bố

Một phần kết quả của luận văn đã được công bố thành hai bài báo khoa học (viết chung với Nguyễn Vũ Thái, Đỗ Trí Nhựt và TS. Nguyễn Văn Dũ), cả hai đều đã được chấp nhận đăng:

1. Thai Nguyen Vu, Dung Van Bui, Nhut Tri Do, Van Du Nguyen, “A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline,” National Conference on Fundamental and Applied Information Technology Research (FAIR), 2026.

2. Thai Nguyen Vu, Dung Van Bui, Nhut Tri Do, Van Du Nguyen, “A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark,” Symposium on Information and Communication Technology (SOICT), 2026.

1.7 Cấu trúc của luận văn

Luận văn với đề tài “PHÁT HIỆN XÂM NHẬP (IDS) DỰA TRÊN APACHE SPARK CHO BẢO MẬT MẠNG IoT” được trình bày bao gồm 5 chương. Nội dung tóm tắt từng chương được trình bày như sau:

- **Chương 1: Mở đầu:** Trình bày tổng quan về bối cảnh và lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu, câu hỏi nghiên cứu và phương pháp tiếp cận. Ngoài ra, chương này cũng nêu rõ ý nghĩa thực tiễn và khoa học của đề tài, cũng như bố cục tổng thể của luận văn.
- **Chương 2: Cơ sở lý thuyết:** Trình bày các khái niệm cơ bản và nền tảng lý thuyết phục vụ cho đề tài, bao gồm tổng quan về mạng IoT, các lỗ hổng và mối đe dọa bảo mật, phương pháp phát hiện xâm nhập (IDS), và kiến thức liên quan đến Apache Spark. Ngoài ra, chương này cũng phân tích các thuật toán học máy thường được sử dụng để phát hiện xâm nhập.
- **Chương 3: Phương pháp thực hiện:** Trình bày chi tiết các bước xây dựng hệ thống phát hiện xâm nhập, bao gồm thu thập và tiền xử lý dữ liệu, thiết kế kiến trúc hệ thống sử dụng Apache Spark, lựa chọn và triển khai các thuật toán học máy, cùng quy trình huấn luyện và đánh giá mô hình.
- **Chương 4: Thực nghiệm, đánh giá và thảo luận:** Trình bày kết quả thực nghiệm trên bộ dữ liệu thực tế hoặc mô phỏng, đánh giá hiệu năng của các mô hình đã triển khai theo các tiêu chí như độ chính xác, Recall, F1-score, kích thước của mô hình và thời gian xử lý. Đồng thời, chương này cũng phân tích, so sánh và thảo luận về các kết quả đạt được nhằm làm rõ hiệu quả của phương pháp đề xuất.
- **Chương 5: Kết luận và hướng phát triển:** Tổng kết những kết quả chính mà đề tài đạt được, nêu bật những đóng góp chính, đồng thời chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai, như việc tích hợp các mô hình học

CHƯƠNG 1. MỞ ĐẦU

sâu hoặc triển khai hệ thống trên nền tảng đám mây để nâng cao tính khả dụng và khả năng mở rộng.

Chương 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày các nền tảng lý thuyết liên quan đến đề tài, bao gồm tổng quan về Internet vạn vật (IoT), các mối đe dọa an ninh trong hệ thống IoT, và vai trò của hệ thống phát hiện xâm nhập (IDS). Bên cạnh đó, trong chương này, luận văn cũng giới thiệu kiến trúc và khả năng của Apache Spark trong xử lý dữ liệu lớn, các kỹ thuật học máy ứng dụng trong phát hiện xâm nhập và cách Spark MLlib hỗ trợ xây dựng mô hình học máy phân tán. (Các nghiên cứu liên quan và khoảng trống nghiên cứu đã được khảo sát ở Chương 1.)

2.1 Tổng quan về IoT

Internet of Things (IoT) là mạng lưới các thiết bị vật lý được tích hợp cảm biến, phần mềm và khả năng kết nối mạng, cho phép thu thập và trao đổi dữ liệu qua Internet [27]. IoT bao gồm các thiết bị như cảm biến môi trường, thiết bị gia dụng thông minh và phương tiện giao thông tạo ra khối lượng dữ liệu lớn với đặc điểm đa dạng, tốc độ cao và yêu cầu xử lý thời gian thực. Theo báo cáo Cisco Annual Internet Report (2018–2023), tổng số thiết bị nối mạng toàn cầu được dự báo đạt khoảng 29,3 tỷ vào năm 2023, trong đó khoảng một nửa là kết nối máy-với-máy (M2M/IoT), nhấn mạnh tầm quan trọng của bảo mật trong lĩnh vực này [28]. Kiến trúc IoT thường được chia thành ba tầng:

- **Tầng cảm biến (Perception Layer):** Thu thập dữ liệu từ cảm biến và thiết bị.
- **Tầng mạng (Network Layer):** Truyền tải dữ liệu qua các giao thức như MQTT, CoAP hoặc HTTP.
- **Tầng ứng dụng (Application Layer):** Phân tích dữ liệu để cung cấp dịch vụ [29].

Tuy nhiên, các thiết bị IoT thường có tài nguyên hạn chế (bộ nhớ, năng lượng, khả năng tính toán), dẫn đến thách thức trong việc triển khai các giải pháp bảo mật truyền thống. Các mối đe dọa như tấn công từ chối dịch vụ (DDoS), tấn công giả mạo và khai thác lỗ hổng giao thức đòi hỏi các giải pháp bảo mật tiên tiến như IDS [30].

2.2 Hệ thống phát hiện xâm nhập (IDS)

Hệ thống phát hiện xâm nhập (IDS) giám sát và phân tích hoạt động mạng hoặc hệ thống để phát hiện các hành vi bất thường hoặc đáng ngờ, nhằm bảo vệ hệ thống khỏi các cuộc

tấn công mạng [31]. IDS đóng vai trò quan trọng trong việc đảm bảo tính bảo mật, toàn vẹn và sẵn sàng của hệ thống IoT. IDS được phân loại như sau:

Dựa trên vị trí triển khai:

- **Network-based IDS (NIDS):** Giám sát lưu lượng mạng để phát hiện các mẫu tấn công.
- **Host-based IDS (HIDS):** Tập trung vào hoạt động của một thiết bị cụ thể.

Dựa trên phương pháp phát hiện:

- **Signature-based IDS:** Sử dụng các mẫu tấn công đã biết, hiệu quả với các mối đe dọa quen thuộc nhưng hạn chế với các tấn công mới.
- **Anomaly-based IDS:** Phát hiện hành vi bất thường dựa trên mô hình hoạt động bình thường, phù hợp với các cuộc tấn công chưa biết [32, 33]; các bộ phát hiện dựa trên học sâu như autoencoder [34] hay mạng niềm tin sâu (Deep Belief Network) [35] là một hướng phổ biến của nhóm này.
- **Hybrid IDS:** Kết hợp cả hai phương pháp để tăng độ chính xác [31].

Trong IoT, IDS phải xử lý khối lượng dữ liệu lớn và đáp ứng yêu cầu thời gian thực, đồng thời phù hợp với tài nguyên hạn chế của thiết bị. Các giải pháp IDS dựa trên dữ liệu lớn và học máy, như sử dụng Apache Spark, là hướng đi tiềm năng để giải quyết các thách thức này [30].

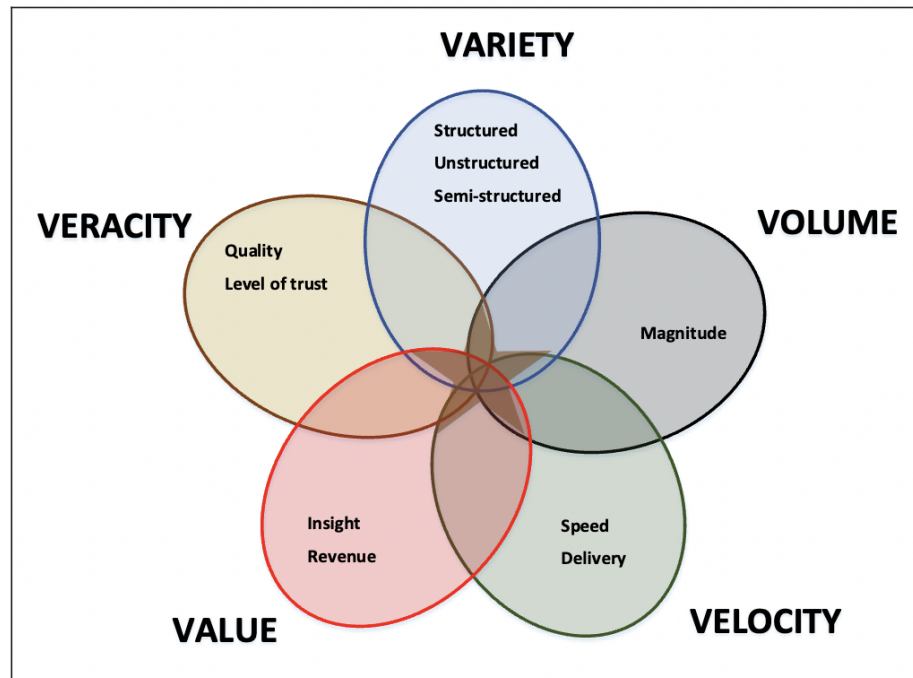
2.3 Apache Spark và xử lý dữ liệu lớn

2.3.1 Dữ liệu lớn

Dữ liệu lớn (*Big Data*) là một thuật ngữ được sử dụng để mô tả tập hợp dữ liệu có quy mô lớn, phức tạp và đa dạng, bao gồm dữ liệu có cấu trúc, dữ liệu không có cấu trúc và dữ liệu bán cấu trúc đến mức không thể xử lý hiệu quả bằng các công cụ và phương pháp truyền thống.

Ngày nay, kích thước của các tập dữ liệu có thể được coi là dữ liệu lớn dao động từ **terabyte**, **petabyte** đến **exabyte**. Khái niệm về dữ liệu lớn sẽ phụ thuộc vào ngành công nghiệp, cách dữ liệu được sử dụng, lượng dữ liệu lịch sử liên quan và nhiều đặc điểm khác.

Dữ liệu lớn thường được nhận diện thông qua năm đặc điểm cốt lõi: **Khối lượng** (*Volume*), **Tốc độ** (*Velocity*), **Tính đa dạng** (*Variety*), **Tính xác thực** (*Veracity*) và **Giá trị** (*Value*) (Hình 2.1). Đây là những yếu tố quan trọng giúp phân biệt dữ liệu lớn với các loại dữ liệu thông thường, phản ánh quy mô, tốc độ phát sinh, sự phong phú về định dạng, độ tin cậy và tiềm năng khai thác thông tin từ dữ liệu.



Hình 2.1. Mô hình 5V của dữ liệu lớn (Nguồn: [36])

- **Khối lượng (Volume):** Đề cập đến quy mô dữ liệu cực lớn.
- **Tốc độ (Velocity):** Tốc độ dữ liệu được tạo ra, truyền tải và xử lý.
- **Tính đa dạng (Variety):** Nguồn và định dạng dữ liệu đa dạng như văn bản, hình ảnh, video, v.v.
- **Tính xác thực (Veracity):** Mức độ đáng tin cậy và chất lượng của dữ liệu.
- **Giá trị (Value):** Tiềm năng khai thác thông tin hữu ích từ dữ liệu.

Dữ liệu sinh ra từ các thiết bị IoT được xem là một trong những nguồn tạo ra dữ liệu lớn (BD) quan trọng và phát triển nhanh chóng hiện nay. Với hàng tỷ thiết bị cảm biến, máy móc, thiết bị đeo tay và phương tiện kết nối Internet, lượng dữ liệu được tạo ra mỗi ngày rất lớn, đa dạng về định dạng và tốc độ phát sinh liên tục theo thời gian thực. Dữ liệu IoT mang đầy đủ các đặc điểm của dữ liệu lớn như: khối lượng (volume) lớn, tốc độ (velocity)

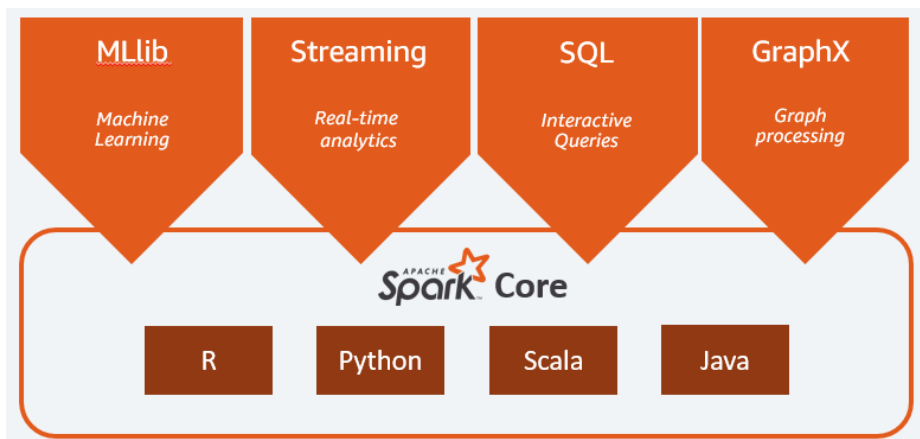
cao, tính đa dạng (variety) về loại dữ liệu, mức độ xác thực (veracity) không đồng đều và tiềm năng mang lại giá trị (value) lớn nếu được khai thác hiệu quả. Chính vì vậy, dữ liệu IoT không chỉ là một phần của dữ liệu lớn mà còn đóng vai trò quan trọng trong việc thúc đẩy nhu cầu phát triển các nền tảng xử lý dữ liệu lớn hiện đại.

2.3.2 Các thành phần của Apache Spark

Trong những năm gần đây, các nền tảng xử lý dữ liệu lớn như Apache Spark đã thu hút sự quan tâm rộng rãi từ cộng đồng nghiên cứu và công nghiệp. Apache Spark là một hệ thống xử lý dữ liệu phân tán quy mô lớn, có khả năng thực thi nhanh các tác vụ trên các cụm dữ liệu lớn. So với Hadoop MapReduce, Spark có thể đạt tốc độ xử lý nhanh hơn khoảng 100 lần trong một số trường hợp nhờ vào khả năng xử lý trong bộ nhớ (*in-memory processing*) [37, 38].

Bên cạnh đó, Spark cung cấp giao diện lập trình linh hoạt thông qua các API hỗ trợ nhiều ngôn ngữ phổ biến như Java, Scala, Python và R, giúp việc phát triển và triển khai ứng dụng trở nên thuận tiện hơn.

Apache Spark bao gồm nhiều thành phần chính tạo thành một hệ sinh thái hoàn chỉnh, hỗ trợ đa dạng các tác vụ phân tích dữ liệu như xử lý dữ liệu theo lô (batch), truy vấn SQL, phân tích luồng dữ liệu thời gian thực, học máy và xử lý đồ thị (Hình 2.2).



Hình 2.2. Các thành phần của Apache Spark (Nguồn: [38])

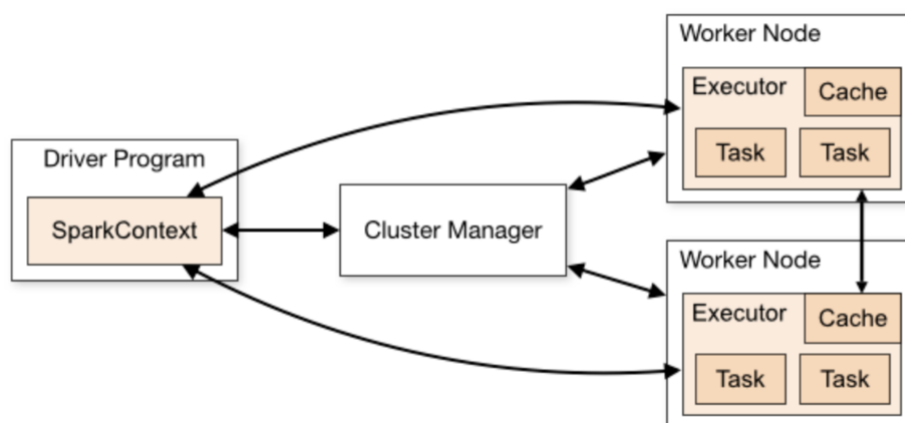
Apache Spark bao gồm nhiều thành phần cốt lõi, mỗi thành phần đảm nhận một chức năng cụ thể trong quá trình xử lý dữ liệu lớn:

- **Spark Core:** Thành phần trung tâm của Spark, chịu trách nhiệm quản lý bộ nhớ, lập lịch, xử lý lỗi, phân phối tác vụ và giao tiếp với các hệ thống lưu trữ. Nó cung cấp các API cấp cao cho Java, Scala, Python và R, giúp đơn giản hóa việc xử lý phân tán.

- **MLlib**: Thư viện học máy tích hợp sẵn trong Spark [39], hỗ trợ huấn luyện và triển khai các mô hình học máy trên quy mô lớn. MLlib cung cấp các thuật toán như phân loại, hồi quy, phân cụm, khai thác mẫu và lọc cộng tác.
- **Spark Streaming**: Công cụ xử lý dữ liệu thời gian thực bằng cách chia luồng dữ liệu thành các lô nhỏ để xử lý tương tự như dữ liệu theo lô. Công cụ này cho phép sử dụng lại mã nguồn và hỗ trợ nhiều nguồn dữ liệu như Kafka, Flume, Twitter, HDFS.
- **Spark SQL**: Thành phần truy vấn dữ liệu phân tán, hỗ trợ ngôn ngữ SQL tiêu chuẩn và HiveQL với hiệu suất cao nhờ tối ưu hóa Catalyst và định dạng cột. Nó hỗ trợ nhiều nguồn dữ liệu như JSON, Hive, Parquet, JDBC, MongoDB, Cassandra.
- **GraphX**: Thư viện xử lý đồ thị phân tán, hỗ trợ biểu diễn, chuyển đổi và phân tích dữ liệu đồ thị. GraphX đi kèm với các thuật toán đồ thị phổ biến như PageRank, Connected Components và nhiều thuật toán phân tích khác.

2.3.3 Kiến trúc của Apache Spark

Kiến trúc của Apache Spark được xây dựng theo mô hình master-worker trên nền tảng xử lý phân tán. Thành phần trung tâm là **Spark Driver**, chịu trách nhiệm điều phối quá trình thực thi ứng dụng, bao gồm việc lên lịch, phân phối tác vụ và giám sát tiến trình. Driver xây dựng một đồ thị phi chu trình có hướng (*DAG* – Directed Acyclic Graph) để xác định luồng thực thi công việc; tổng quan kiến trúc master-worker được minh họa ở Hình 2.3.



Hình 2.3. Sơ đồ kiến trúc hệ thống Apache Spark (Nguồn: [37])

Các tác vụ được phân phối tới các **Worker Nodes**, nơi chứa các **Executor** — tiến trình thực thi thực tế. Mỗi Executor chịu trách nhiệm thực thi các tác vụ (task) và lưu trữ dữ liệu

trong bộ nhớ hoặc ổ đĩa nếu cần. Spark sử dụng **Cluster Manager** (như YARN, Mesos hoặc Standalone) để quản lý tài nguyên trên cụm. Dữ liệu được lưu trữ trên các hệ thống như HDFS, S3, hoặc các cơ sở dữ liệu quan hệ.

2.4 Học máy trong phát hiện xâm nhập

Học máy (ML) đã trở thành một công cụ thiết yếu trong việc phát hiện các mẫu tấn công và hành vi bất thường trong lưu lượng mạng IoT, nhờ khả năng tự động học và thích ứng từ dữ liệu lịch sử [40]. Các kỹ thuật học máy phổ biến bao gồm:

- **Học có giám sát (Supervised Learning):** Sử dụng dữ liệu đã được gán nhãn để huấn luyện mô hình phân loại. Một số thuật toán điển hình gồm Random Forest (RF), SVM và Mạng nơ-ron nhân tạo (Neural Networks).
- **Học không giám sát (Unsupervised Learning):** Áp dụng cho các tình huống không có nhãn, tập trung vào việc phát hiện các mẫu bất thường. K-Means là một thuật toán phổ biến trong nhóm này.
- **Học tăng cường (Reinforcement Learning):** Mô hình học dựa trên phản hồi từ môi trường nhằm tối ưu hóa hành vi hoặc quyết định của hệ thống theo thời gian; hướng này không thuộc phạm vi thực nghiệm của luận văn và chỉ được nêu để hoàn chỉnh bức tranh phân loại.

Trong bối cảnh mạng IoT, học máy được ứng dụng rộng rãi để phân tích lưu lượng dữ liệu và nhận diện các loại tấn công như DDoS, botnet hay giả mạo gói tin. Chẳng hạn, Koroniotis và cộng sự (2019) xây dựng bộ dữ liệu Bot-IoT và báo cáo các mô hình học máy, học sâu đạt độ chính xác gần tuyệt đối khi phát hiện các tấn công DoS/DDoS trong môi trường IoT [41]. Tuy nhiên, các kết quả gần như tuyệt đối (xấp xỉ 100%) trên các tập dữ liệu công khai cần được xem xét thận trọng, vì chúng thường là dấu hiệu của rò rỉ dữ liệu (data leakage) hơn là năng lực thực sự của mô hình. *Rò rỉ dữ liệu* là tình huống trong đó thông tin lẽ ra không sẵn có ở thời điểm triển khai thực tế lại lọt vào quá trình huấn luyện hoặc đánh giá, khiến điểm số bị thổi phồng và không phản ánh đúng khả năng tổng quát hóa của mô hình. Luận văn này nhận diện và xử lý vấn đề đó một cách tường minh ở Chương 4.

Tuy nhiên, việc ứng dụng học máy trong hệ thống IoT vẫn còn gặp nhiều thách thức vì yêu cầu xử lý theo thời gian thực và hạn chế về tài nguyên tính toán. Việc tích hợp học máy

với các nền tảng xử lý dữ liệu lớn như Apache Spark mang lại lợi thế đáng kể, giúp xử lý khối lượng dữ liệu lớn với tốc độ cao và hỗ trợ triển khai các thuật toán học máy hiệu quả trong môi trường phân tán.

2.4.1 Một số thuật toán học máy

- **Cây quyết định (Decision Trees (DT))** là một trong những thuật toán học máy có giám sát phổ biến, được áp dụng rộng rãi trong các bài toán phân loại. Cấu trúc cơ bản của một cây quyết định bao gồm ba thành phần chính: **nút gốc**, **nút quyết định** và **nút lá**. Cây quyết định có ưu điểm là dễ hiểu, dễ diễn giải, có thể trực quan hóa và không yêu cầu giả định về phân phối dữ liệu. Tuy nhiên, DT cũng dễ bị quá khớp (*overfitting*) nếu không chọn được các tham số hợp lý hoặc xử lý trên dữ liệu nhiễu [42].
- **Rừng ngẫu nhiên (RF)** là một thuật toán học máy được sử dụng phổ biến trong cả hai bài toán phân loại và hồi quy. Đây là một kỹ thuật học tổ hợp (*ensemble learning*) dựa trên việc xây dựng và tổng hợp nhiều cây quyết định độc lập nhằm nâng cao độ chính xác và độ ổn định của mô hình. Thuật toán hoạt động bằng cách tạo ra nhiều cây quyết định trên các tập con khác nhau được chọn ngẫu nhiên từ tập dữ liệu huấn luyện. Mỗi cây được xây dựng dựa trên một mẫu bootstrap (lấy mẫu có hoàn lại), đồng thời tại mỗi nút chia, một tập con ngẫu nhiên của các đặc trưng được chọn để tìm điểm chia tốt nhất, thay vì sử dụng toàn bộ đặc trưng như trong cây quyết định truyền thống [43].

Kết quả cuối cùng được xác định bằng cách:

- ◇ Trong bài toán phân loại: lấy kết quả theo nguyên tắc đa số phiếu (*majority voting*) từ các cây.
- ◇ Trong bài toán hồi quy: lấy giá trị trung bình từ các dự đoán của từng cây.

Việc kết hợp nhiều cây giúp RF giảm thiểu hiện tượng quá khớp (*overfitting*) vốn là một nhược điểm thường gặp của các cây quyết định đơn lẻ. Khi số lượng cây trong rừng tăng lên, hiệu năng dự đoán của mô hình có xu hướng ổn định và cải thiện hơn.

- **Hồi quy logistic (Logistic Regression (LR))** là một phương pháp thống kê được giới thiệu bởi David Cox vào cuối thập niên 1950. Đây là một kỹ thuật học máy có giám sát thường được sử dụng để giải quyết các bài toán phân loại nhị phân, trong đó biến

đầu ra (phản hồi) là một biến định tính, nhận giá trị 0 hoặc 1. Mô hình hồi quy logistic cho phép ước tính xác suất xảy ra của một sự kiện cụ thể dựa trên một hoặc nhiều biến đầu vào (độc lập). Một trong những điểm mạnh của phương pháp này là khả năng xác định mức độ ảnh hưởng của từng yếu tố đến xác suất xuất hiện của kết quả, giúp các nhà phân tích hiểu rõ hơn mối quan hệ giữa biến đầu vào và đầu ra [44]. Về mặt toán học, hàm hồi quy logistic được biểu diễn dưới dạng tuyến tính như sau:

$$z_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \cdots + \beta_n x_{i,n} \quad (2.1)$$

Trong đó:

- ◇ $x_{i,j}$ là giá trị của biến độc lập thứ j tại quan sát thứ i
- ◇ β_0 là hệ số chặn (bias), β_j là hệ số hồi quy ứng với biến x_j
- ◇ z_i là đầu ra tuyến tính trước khi đưa qua hàm kích hoạt (sigmoid)

Sau đó, xác suất xảy ra của sự kiện được tính thông qua hàm logistic (sigmoid):

$$P(Y_i = 1 \mid \mathbf{x}_i) = \frac{1}{1 + e^{-z_i}} \quad (2.2)$$

Trong quá trình huấn luyện, mô hình tìm các hệ số β_j tối ưu sao cho hàm mất mát (cross-entropy loss) được tối thiểu hóa.

- **GBT** là một phương pháp *boosting* (kết hợp tuần tự các mô hình học yếu; lưu ý phân biệt với “học tăng cường” – reinforcement learning) sử dụng các cây quyết định như mô hình học yếu (weak learners). Ý tưởng chính của boosting là xây dựng một mô hình mạnh bằng cách kết hợp nhiều mô hình học yếu được huấn luyện theo trình tự tuần tự. Ở mỗi bước, thuật toán cố gắng giảm thiểu lỗi của mô hình trước bằng cách học từ phần dư (residual) hoặc gradient của hàm mất mát [45]. Quá trình huấn luyện GBT gồm các bước:

- ◇ Bắt đầu với một mô hình dự đoán ban đầu (thường là giá trị trung bình với bài toán hồi quy).
- ◇ Tính toán sai số còn lại giữa nhãn thực và dự đoán.
- ◇ Huấn luyện một cây quyết định để dự đoán phần dư.

- ◇ Cập nhật mô hình tổng bằng cách cộng mô hình hiện tại với mô hình mới nhân với một hệ số học η (learning rate).

Giả sử $F_0(x)$ là mô hình khởi tạo ban đầu, mô hình tổng tại bước m được cập nhật theo công thức:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x) \quad (2.3)$$

Trong đó:

- ◇ $h_m(x)$ là cây quyết định thứ m học theo gradient của hàm mất mát.
- ◇ η là hệ số học (learning rate), thường $\eta \in (0, 1)$.
- **LightGBM (LGBM)** là một framework triển khai thuật toán GBT, được phát triển nhằm tối ưu hóa tốc độ huấn luyện và khả năng mở rộng trên các tập dữ liệu lớn [46], với cùng sơ đồ cập nhật boosting theo gradient đã trình bày ở mục GBT phía trên. Khác với GBT truyền thống, LGBM sử dụng cơ chế phát triển cây theo hướng leaf-wise thay vì level-wise, đồng thời áp dụng thuật toán histogram-based splitting và kỹ thuật Gradient-based One-Side Sampling (GOSS) nhằm giảm chi phí tính toán và tăng tốc độ huấn luyện. Tuy nhiên, LGBM *không* được đưa vào bộ thuật toán so sánh thực nghiệm vì lý do tương thích phần cứng trình bày ở Chương 3; XGBoost và GBT đại diện cho họ gradient boosting.
- **NB** là một phương pháp phân loại xác suất dựa trên định lý Bayes với giả định độc lập có điều kiện giữa các đặc trưng [47]. Mặc dù giả định này thường không hoàn toàn đúng trong thực tế, mô hình vẫn cho hiệu quả tốt trong nhiều bài toán phân loại dữ liệu nhiều chiều.

Theo định lý Bayes:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)} \quad (2.4)$$

Với giả định độc lập giữa các đặc trưng:

$$P(x|y) = \prod_{i=1}^n P(x_i|y) \quad (2.5)$$

Mô hình lựa chọn lớp có xác suất hậu nghiệm lớn nhất:

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i|y) \quad (2.6)$$

Ưu điểm của NB là đơn giản, tốc độ huấn luyện nhanh và phù hợp với dữ liệu kích thước lớn; tuy nhiên, hạn chế chính nằm ở giả định độc lập giữa các đặc trưng.

- **Support Vector Machine (SVM)** là một thuật toán học có giám sát được đề xuất bởi Cortes và Vapnik [48], nhằm tìm siêu phẳng tối ưu phân tách dữ liệu với khoảng cách biên (margin) lớn nhất.

Trong trường hợp dữ liệu tuyến tính, bài toán tối ưu được biểu diễn như sau:

$$\min_{w,b} \frac{1}{2} ||w||^2 \quad (2.7)$$

với điều kiện:

$$y_i(w \cdot x_i + b) \geq 1 \quad (2.8)$$

Trong trường hợp dữ liệu không tuyến tính, SVM sử dụng kỹ thuật *kernel trick* để ánh xạ dữ liệu sang không gian nhiều chiều hơn, cho phép phân tách tuyến tính trong không gian mới. Nhờ cơ chế tối ưu hóa chặt chẽ, SVM thường đạt hiệu năng cao trong các bài toán phân loại với dữ liệu nhiều chiều.

- **Multilayer Perceptron - MLP** là một mô hình học sâu cơ bản gồm nhiều lớp ẩn, có khả năng mô hình hóa các mối quan hệ phi tuyến giữa các đặc trưng [49].

Mỗi nơ-ron trong mạng thực hiện phép biến đổi tuyến tính:

$$z = w^T x + b \quad (2.9)$$

sau đó áp dụng hàm kích hoạt phi tuyến:

$$a = \sigma(z) \quad (2.10)$$

Trong đó $\sigma(\cdot)$ có thể là các hàm kích hoạt như sigmoid, tanh hoặc ReLU. Quá trình huấn luyện sử dụng thuật toán lan truyền ngược (*backpropagation*) nhằm cập nhật

trọng số thông qua tối thiểu hóa hàm mất mát bằng gradient descent. MLP có khả năng biểu diễn mạnh nhưng yêu cầu dữ liệu huấn luyện đủ lớn và quá trình điều chỉnh siêu tham số cẩn thận để tránh hiện tượng quá khớp.

- **XGB** là một phiên bản cải tiến của thuật toán GBT truyền thống, được đề xuất bởi Chen và Guestrin vào năm 2016. XGB được thiết kế để tối ưu hóa tốc độ huấn luyện, hiệu năng mô hình, và khả năng xử lý dữ liệu quy mô lớn trong các hệ thống tính toán phân tán [50]. XGB sử dụng các chiến lược sau để cải thiện hiệu năng:

- ◇ Khai thác gradient và Hessian để tối ưu hóa hiệu quả (second-order optimization).
- ◇ Cắt tỉa cây (pruning) theo độ phức tạp thay vì độ sâu cố định.
- ◇ Tính song song trong việc tìm điểm chia tối ưu (split finding).
- ◇ Hỗ trợ xử lý thiếu giá trị (missing values) tự động.

Mô hình XGB đã được ứng dụng thành công trong nhiều bài toán học máy thực tế như dự báo tài chính, chẩn đoán y tế, phát hiện gian lận và phân tích hành vi khách hàng, nhờ độ chính xác cao và hiệu quả tính toán tốt.

2.4.2 Một số thuật toán học tổ hợp

Học tổ hợp (Ensemble Learning, EL) là một kỹ thuật trong học máy nhằm kết hợp nhiều mô hình riêng lẻ để nâng cao độ chính xác và độ ổn định trong dự đoán. Thay vì chỉ dựa vào một mô hình đơn lẻ, phương pháp EL tận dụng sức mạnh tổng hợp của nhiều mô hình (ví dụ DT, RF, LR) để xây dựng một hệ thống dự đoán chính xác hơn. Nguyên lý cốt lõi của EL là: mỗi mô hình riêng lẻ có thể chỉ đạt độ chính xác thấp hoặc dự đoán sai ở một số trường hợp, nhưng sự kết hợp hợp lý của chúng có thể giảm thiểu sai số, tăng độ chính xác nhờ tận dụng điểm mạnh và khắc phục điểm yếu của từng mô hình thành phần [51].

Kỹ thuật EL đã chứng minh được tính hiệu quả cao trong nhiều lĩnh vực yêu cầu độ chính xác cao như: phát hiện gian lận, phân tích dữ liệu lớn, nhận diện hình ảnh và xử lý ngôn ngữ tự nhiên. Trong lĩnh vực bảo mật mạng IoT, EL cũng được ứng dụng để cải thiện khả năng phát hiện bất thường hoặc xâm nhập dựa trên nhiều mô hình học máy được huấn luyện song song hoặc tuần tự.

Một số phương pháp phổ biến của EL bao gồm:

- **Voting:** Với bài toán phân loại, mỗi mô hình con bỏ phiếu cho một lớp và lớp có số phiếu cao nhất sẽ là đầu ra cuối cùng (đa số phiếu hoặc trung bình xác suất). Các bước của voting được mô tả chi tiết ở Thuật toán 1.
- **Boosting:** Huấn luyện các mô hình tuần tự, trong đó mỗi mô hình mới tập trung vào các mẫu mà mô hình trước đã dự đoán sai (ví dụ: GBT, AdaBoost, XGB). Các bước của boosting được mô tả chi tiết ở Thuật toán 2.
- **Bagging (Bootstrap Aggregating):** Huấn luyện nhiều mô hình trên các tập con ngẫu nhiên (lấy mẫu có hoàn lại) của tập dữ liệu gốc, sau đó tổng hợp kết quả dự đoán (ví dụ: Random Forest). Các bước của bagging được mô tả chi tiết ở Thuật toán 3.

EL đặc biệt hữu ích trong các hệ thống phát hiện xâm nhập (IDS) vì cần phân tích lượng lớn dữ liệu phức tạp, và việc kết hợp nhiều mô hình có thể giúp tăng khả năng phát hiện các gói tin bất thường mà một mô hình đơn lẻ có thể bỏ sót.

Thuật toán 1. Voting: Kết hợp dự đoán từ nhiều mô hình phân loại

Đầu vào:

- Tập huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- Tập mô hình phân loại cơ sở $M_1, M_2, \dots, M_K$
- Kiểu voting: HARD hoặc SOFT

Đầu ra: Nhân dự đoán cuối cùng y_{vote} cho điểm dữ liệu x

```

1: Khởi tạo: danh sách  $S = \{\}$  ▷ Lưu dự đoán hoặc xác suất
2: if voting = HARD then
3:   for  $j = 1$  to  $K$  do
4:      $y_j \leftarrow M_j(x)$  ▷ Dự đoán nhân của mô hình thứ  $j$ 
5:     Thêm  $y_j$  vào  $S$ 
6:   end for
7:    $y_{\text{vote}} \leftarrow$  nhân xuất hiện nhiều nhất trong  $S$ 
8: else if voting = SOFT then
9:   Khởi tạo xác suất tổng  $P_{\text{sum}}[c] = 0, \forall c \in \{1, \dots, C\}$ 
10:  for  $j = 1$  to  $K$  do
11:    Lấy xác suất  $P_j(y = c | x)$  từ mô hình  $M_j$ 
12:    for mỗi lớp  $c = 1$  to  $C$  do
13:       $P_{\text{sum}}[c] \leftarrow P_{\text{sum}}[c] + P_j(y = c | x)$ 
14:    end for
15:  end for
16:   $y_{\text{vote}} \leftarrow \arg \max_c P_{\text{sum}}[c]$ 
17: end if
18: return  $y_{\text{vote}}$ 

```

Thuật toán 2. Boosting: Kết hợp tuần tự các mô hình học yếu

Đầu vào:

- Tập dữ liệu huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- Số lượng vòng lặp (mô hình yếu): T

Đầu ra: Mô hình boosting $F_T(x)$

- 1: **Khởi tạo:** $F_0(x) \leftarrow 0$ $\triangleright$ Mô hình khởi đầu có thể là 0 hoặc dự đoán trung bình
 - 2: **for** $t = 1$ to T **do**
 - 3: Tính giả-phần-dư (pseudo-residuals) theo gradient âm của hàm mất mát: $r_i^{(t)} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{t-1}}$ $\triangleright$ Với mất mát bình phương (hồi quy), rút gọn thành $y_i - F_{t-1}(x_i)$; với phân loại dùng log-loss
 - 4: Huấn luyện mô hình yếu $h_t(x)$ trên các giả-phần-dư $r_i^{(t)}$
 - 5: Cập nhật mô hình: $F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$
 - 6: **end for**
 - 7: **return** $F_T(x)$
-

Thuật toán 3. Bagging: Bootstrap Aggregating

Đầu vào:

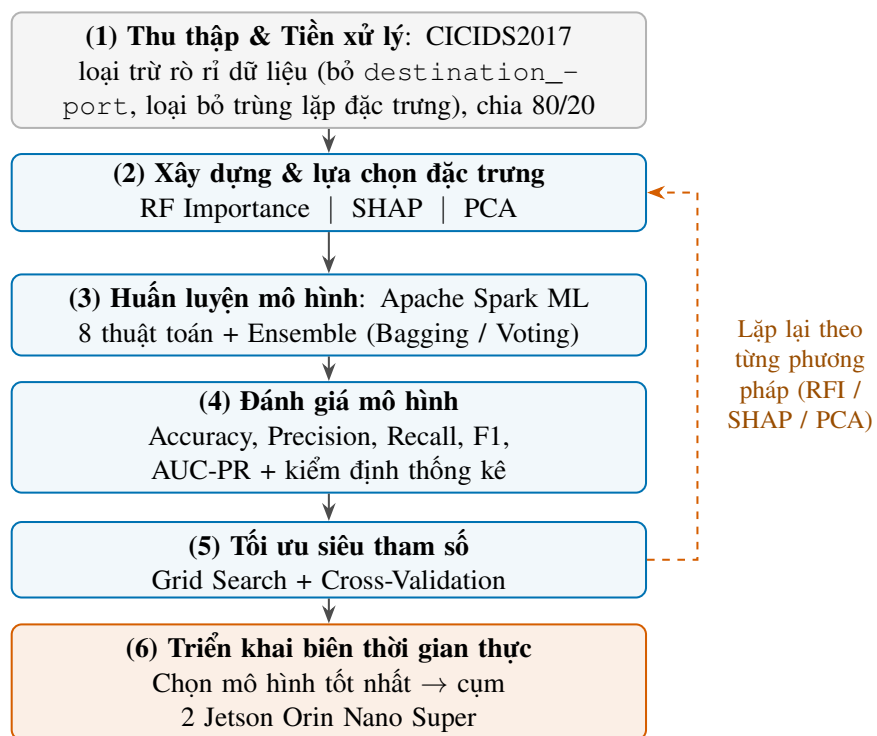
- Tập huấn luyện $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, với $x_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$ (tổng quát: C lớp)
- Số lượng mô hình cơ sở: K

Đầu ra: Dự đoán cuối cùng y_{bag} cho một điểm dữ liệu x

- 1: **Khởi tạo:** Danh sách mô hình $M_1, M_2, \dots, M_K$
 - 2: **for** $k = 1$ to K **do**
 - 3: Tạo mẫu bootstrap $\mathcal{D}^{(k)}$ từ $\mathcal{D}$ (lấy mẫu ngẫu nhiên có hoàn lại)
 - 4: Huấn luyện mô hình $M_k \leftarrow \text{Train}(\mathcal{D}^{(k)})$
 - 5: **end for**
 - 6: **Khởi tạo:** Danh sách dự đoán $Y = \{\}$
 - 7: **for** $k = 1$ to K **do**
 - 8: Thực hiện dự đoán $y_k \leftarrow M_k(x)$
 - 9: Thêm y_k vào Y
 - 10: **end for**
 - 11: Tính $y_{\text{bag}} \leftarrow \text{Voting}(Y)$ $\triangleright$ Hard voting hoặc trung bình xác suất nếu dùng soft
 - 12: **return** y_{bag}
-

Chương 3. PHƯƠNG PHÁP THỰC HIỆN

Trong chương này, luận văn sẽ trình bày phương pháp đề xuất để xây dựng mô hình phát hiện xâm nhập dựa trên Apache Spark cho bảo mật mạng IoT. Quá trình thực hiện bao gồm 6 bước chính, được thể hiện qua Hình 3.1: (1) Thu thập dữ liệu và tiền xử lý; (2) Xây dựng đặc trưng và lựa chọn đặc trưng; (3) Huấn luyện mô hình; (4) Đánh giá mô hình; (5) Tối ưu siêu tham số; và (6) Triển khai biên thời gian thực. Các bước (1)–(5) được lặp lại tương ứng với từng phương pháp lựa chọn đặc trưng (Random Forest Importance (RFI), SHAP, PCA), sau đó lựa chọn mô hình tốt nhất để triển khai trên thiết bị biên.



Hình 3.1. Quy trình tổng thể huấn luyện, đánh giá và triển khai mô hình đề xuất

3.1 Thu thập dữ liệu và tiền xử lý

Trong nghiên cứu này, tập dữ liệu được sử dụng là CICIDS2017 được phát hành bởi Viện An ninh mạng Canada (CIC). Tập dữ liệu này được xây dựng nhằm mô phỏng lưu lượng mạng thực tế kết hợp với nhiều loại tấn công mạng hiện đại, phục vụ cho việc huấn luyện và đánh giá các mô hình phát hiện xâm nhập. Tập dữ liệu CICIDS2017 tập trung vào các kịch bản tấn công đơn lẻ trong môi trường kiểm soát. Cần lưu ý rằng các bộ dữ liệu phát hiện xâm nhập mạng công khai — bao gồm CICIDS2017 — có những giới hạn đã được khảo sát một cách hệ thống (lỗi gán nhãn, tạo tác khi sinh lưu lượng, và khả năng tổng quát

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

hóa hạn chế) [52]; đây là lý do luận văn áp dụng quy trình loại trừ rõ ràng ở bước tiền xử lý và bổ sung đánh giá liên bộ dữ liệu ở Chương 4. Thông tin chi tiết về tập dữ liệu được trình bày trong Bảng 3.1.

Bảng 3.1. Mô tả bộ dữ liệu CICIDS2017

Thuộc tính	Giá trị
Tổ chức phát hành	Canadian Institute for Cybersecurity (CIC)
Thời gian phát hành	2017
Giao thức mạng	HTTP, HTTPS, FTP, SSH, DNS, ...
Số lượng bản ghi (CSV)	Hơn 2.8 triệu bản ghi
Thời gian thu thập	03/07/2017 – 07/07/2017
Loại tấn công mô phỏng	DoS, DDoS, PortScan, Brute-force, Infiltration, Botnet, ...
Số loại nhãn	15 (1 nhãn bình thường + 14 loại tấn công)
Định dạng dữ liệu	CSV
Mục đích	Phát hiện xâm nhập mạng trong môi trường thực tế giả lập
Nguồn dữ liệu	[53]

Bên cạnh CICIDS2017 (bộ dữ liệu chính cho toàn bộ các thí nghiệm), luận văn sử dụng thêm **CSE-CIC-IDS2018** cho thí nghiệm *tổng quát hóa liên bộ dữ liệu* (Mục 4.2.8.1): huấn luyện trên một bộ, kiểm tra trên bộ kia và ngược lại. Đây là bộ kế nhiệm của CICIDS2017, do CIC phối hợp với Cơ quan An ninh Truyền thông Canada (CSE) thu thập trên hạ tầng AWS quy mô lớn hơn nhiều; đặc trưng luồng được trích bằng CICFlowMeter phiên bản mới hơn nên tên cột được ánh xạ về danh mục tên cột chuẩn của CICIDS2017 qua bảng bí danh tường minh trong mã nguồn. Thông tin chi tiết ở Bảng 3.2.

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

Bảng 3.2. Mô tả bộ dữ liệu CSE-CIC-IDS2018 (dùng cho thí nghiệm liên bộ dữ liệu)

Thuộc tính	Giá trị
Tổ chức phát hành	CIC phối hợp CSE (Communications Security Establishment)
Thời gian phát hành	2018
Hạ tầng thu thập	Testbed trên AWS (~450 máy nạn nhân, 50 máy tấn công)
Số lượng bản ghi (CSV)	~16,2 triệu luồng (10 tệp CSV theo ngày)
Thời gian thu thập	14/02/2018 – 02/03/2018
Loại tấn công mô phỏng	DDoS (HOIC, LOIC-HTTP/UDP), DoS (Hulk, GoldenEye, Slowloris, SlowHTTPTest), Bot, Brute-force (SSH, FTP, Web, XSS), SQL Injection, Infiltration
Số loại nhãn	15 (1 nhãn bình thường + 14 loại tấn công)
Phân bố nhị phân (sau làm sạch)	~13,35 triệu Benign / ~2,35 triệu Attack
Phần dùng trong luận văn	Mẫu phân tầng theo nhãn 20% ($seed=42$), còn ~2,39 triệu luồng sau khử trùng lặp mức đặc trưng
Định dạng dữ liệu	CSV (CICFlowMeter v3; tên cột ánh xạ về chuẩn CICIDS2017)
Nguồn dữ liệu	[54]

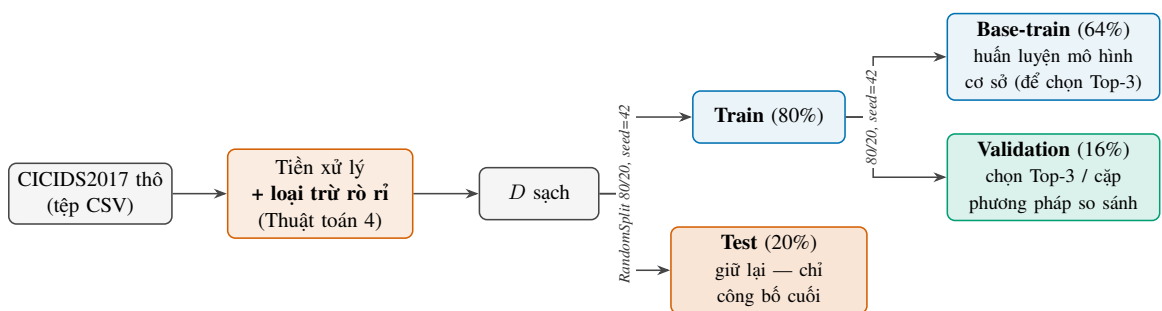
Tiền xử lý dữ liệu Việc tiền xử lý dữ liệu nhằm loại bỏ nhiễu, giá trị ngoại lệ và giá trị bị thiếu được thực hiện trước khi đưa dữ liệu vào huấn luyện mô hình. Các kỹ thuật được áp dụng gồm chuẩn hóa tên cột, căn chỉnh lược đồ giữa các tệp, thay thế giá trị vô cùng, loại bỏ bản ghi trùng lặp/thiếu, và gán nhãn nhị phân (BENIGN = 0, Attack = 1).

Bên cạnh đó, để bảo đảm kết quả đánh giá phản ánh đúng năng lực mô hình, luận văn áp dụng thêm hai bước *loại trừ rò rỉ dữ liệu* (loại cạm bẫy “data snooping” được xem là phổ biến nhất của học máy trong an ninh [55]) ngay trong giai đoạn tiền xử lý, trước khi phân chia tập: (i) **loại đặc trưng rò rỉ nhãn** `destination_port` (và `source_port` nếu có) — vì trong CICIDS2017, mỗi loại tấn công được sinh nhằm vào những cổng dịch vụ cố định nên cổng đích gần như mã hóa trực tiếp nhãn, khiến mô hình “ghi nhớ” ánh xạ cổng–nhãn thay vì học hành vi luồng (cột `protocol` cũng được loại với lý do tương tự: đây là biến phân loại có ít mức giá trị, tương quan mạnh với cổng dịch vụ nên mang cùng nguy cơ “lối tắt” định danh dịch vụ thay vì hành vi); và (ii) **loại bỏ trùng lặp ở mức đặc trưng theo cơ chế tất định** — các luồng được gom nhóm theo toàn bộ vector đặc trưng; với nhóm có nhãn nhất quán, giữ lại đúng một đại diện; với nhóm trùng đặc trưng nhưng *mâu thuẫn nhãn* (cùng vector đặc trưng, nhãn vừa Benign vừa Attack, một lỗi gán nhãn đã được ghi nhận của CICIDS2017 [56]), *toàn bộ nhóm bị loại* vì không thể xác định nhãn đúng, thay vì giữ lại một hàng tùy ý theo phân mảnh dữ liệu (vốn không tái lập được giữa các lần chạy); số nhóm và số hàng xung đột nhãn được ghi log tường minh (270 nhóm,

44.260 hàng). Cơ chế này tránh việc cùng một mẫu xuất hiện ở cả tập huấn luyện lẫn kiểm tra khi phân chia ngẫu nhiên. Đối với tập dữ liệu CICIDS2017, quá trình tiền xử lý dữ liệu được thể hiện chi tiết ở Thuật toán 4. Cần thừa nhận rằng bước loại bỏ trùng lặp ở mức đặc trưng làm *thay đổi phân bố lớp*: do các luồng benign trùng lặp nhiều hơn nên bị loại nhiều hơn, khiến tỷ lệ benign/attack sau khi xử lý khác với tỷ lệ ban đầu; vì vậy phân bố lớp trước và sau khi loại trùng lặp được báo cáo tường minh để bảo đảm minh bạch.

Phân chia tập dữ liệu

Tập dữ liệu CICIDS2017 được phân chia theo tỷ lệ 80/20 cho huấn luyện và kiểm tra bằng phép chia ngẫu nhiên (random split) với seed cố định (=42) để đảm bảo tái lập. Các chỉ số hiệu năng được **công bố trên tập kiểm tra** (tập giữ lại cuối cùng, không tham gia huấn luyện, xếp hạng đặc trưng hay tinh chỉnh siêu tham số); kết quả trên tập huấn luyện chỉ được dùng để chẩn đoán hiện tượng quá khớp (overfitting) hoặc thiếu khớp (underfitting). Đáng lưu ý, `StandardScaler`, PCA và việc xếp hạng đặc trưng (RF Importance, SHAP) đều được *fit chỉ trên tập huấn luyện* D_{train} rồi *transform* lên tập kiểm tra, nhằm tránh rò rỉ thông tin từ tập kiểm tra vào quá trình huấn luyện. Riêng việc **chọn Top-3 mô hình thành phần** cho các ensemble (cũng như việc chọn mô hình đại diện mỗi phương pháp và cặp phương pháp đưa vào kiểm định thống kê, Chương 4) được thực hiện trên một *tập kiểm định* (*validation*) tách từ tập huấn luyện (theo tỷ lệ 80/20, `seed=42`, các mô hình cơ sở huấn luyện trên phần còn lại); nhờ vậy tập kiểm tra hoàn toàn không tham gia vào khâu lựa chọn mô hình, khép kín nguy cơ rò rỉ chọn lọc. Toàn bộ quy trình phân chia được minh họa ở Hình 3.2.



Hình 3.2. Sơ đồ phân chia dữ liệu CICIDS2017. Sau tiền xử lý và loại trừ rò rỉ (Thuật toán 4), dữ liệu được chia 80/20 thành train/test (`seed=42`); tập test được giữ lại, chỉ dùng để công bố kết quả cuối. `StandardScaler`, PCA và việc xếp hạng đặc trưng (RF/SHAP) được *fit trên toàn bộ tập train* rồi giữ cố định. Riêng để chọn Top-3 mô hình thành phần và cặp phương pháp đưa vào kiểm định, tập train được tách tiếp 80/20 thành base-train (huấn luyện mô hình cơ sở) và validation (chấm điểm chọn lọc); ngưỡng cổng bất thường được chọn trên 10% lưu lượng benign của tập train (`seed=42`). Tập test không tham gia bất kỳ khâu lựa chọn nào.

Thuật toán 4. Thuật toán Tiền xử lý Dữ liệu CICIDS2017

Đầu vào: Tập N tệp CSV từ bộ dữ liệu CICIDS2017: $\{F_1, F_2, \dots, F_N\}$

Tỷ lệ phân chia huấn luyện/kiểm tra $\alpha = 0.8$

Đầu ra: Tập huấn luyện D_{train} và tập kiểm tra D_{test} ở định dạng Parquet

```

1: Bước 1: Gộp dữ liệu từ nhiều tệp CSV
2:  $D \leftarrow \emptyset$ 
3: for  $i = 1$  to  $N$  do
4:   Đọc tệp  $F_i$  thành DataFrame  $df_i$  với lược đồ suy luận tự động
5:    $\text{CLEANCOLUMNAMES}(df_i)$   $\triangleright$  Chuẩn hóa tên cột: xóa khoảng trắng, chuyển chữ thường
6:    $\text{HANDLEINFINITYVALUES}(df_i)$   $\triangleright$  Thay thế  $\pm\infty$  bằng giá trị NaN
7:   if  $D = \emptyset$  then
8:      $S \leftarrow \text{Schema}(df_i)$   $\triangleright$  Lưu lược đồ chuẩn từ tệp đầu tiên
9:      $D \leftarrow df_i$ 
10:  else
11:     $\text{ALIGNSCHEMA}(df_i, S)$   $\triangleright$  Căn chỉnh lược đồ về chuẩn  $S$ 
12:     $D \leftarrow D \cup df_i$   $\triangleright$  Gộp theo tên cột (unionByName)
13:  end if
14: end for
15: Bước 2: Làm sạch dữ liệu
16:  $D \leftarrow \text{RemoveDuplicates}(D)$   $\triangleright$  Loại bỏ các bản ghi trùng lặp hoàn toàn (giống hệt trên mọi cột)
17:  $D \leftarrow \text{RemoveNulls}(D)$   $\triangleright$  Loại bỏ các bản ghi chứa giá trị null/NaN
18: Bước 3: Tạo nhãn nhị phân
19:  $\forall (x_i, \text{label}_i) \in D : y_i \leftarrow \begin{cases} 0 & \text{nếu label}_i = \text{"BENIGN"} \\ 1 & \text{ngược lại (Attack)} \end{cases}$ 
     $\rightarrow$  Hai bước loại trừ rò rỉ dữ liệu (data leakage): Bước 4 và Bước 4b
20: Bước 4 [loại rò rỉ – I]: Xác định các đặc trưng số (đã loại trừ rò rỉ)
21:  $C_{\text{exclude}} \leftarrow \{\text{label, label\_binary, source\_ip, destination\_ip, flow\_id, timestamp, protocol, destination\_port, source\_port}\}$   $\triangleright$  Loại bỏ đặc trưng gây rò rỉ nhãn
22:  $\mathcal{F} \leftarrow \{c \in \text{columns}(D) \mid c \notin C_{\text{exclude}} \wedge \text{dtype}(c) \in \{\text{double, float, int, bigint}\}\}$ 
23: Bước 4b [loại rò rỉ – II]: Loại bỏ trùng lặp ở mức đặc trưng (tắt định)
24:  $G \leftarrow \text{GroupBy}(D, \mathcal{F})$   $\triangleright$  Gom nhóm theo toàn bộ vector đặc trưng
25: for mỗi nhóm  $g \in G$  do
26:   if  $|\{y_i : (x_i, y_i) \in g\}| > 1$  then  $\triangleright$  Nhóm mâu thuẫn nhãn (Benign lẫn Attack)
27:     Loại toàn bộ nhóm  $g$  khỏi  $D$ ; ghi log số nhóm/hàng xung đột
28:   else
29:     Giữ lại đúng một đại diện của  $g$  (thứ tự tắt định theo nhãn)
30:   end if
31: end for
32: Bước 5: Phân chia và lưu trữ
33:  $D_{\text{train}}, D_{\text{test}} \leftarrow \text{RandomSplit}(D, [\alpha, 1 - \alpha], \text{seed} = 42)$ 
34: Lưu  $D_{\text{train}}, D_{\text{test}}$  dưới định dạng Parquet
35: return  $D_{\text{train}}, D_{\text{test}}$ 

```

3.2 Lựa chọn đặc trưng và giảm chiều dữ liệu

Trong quá trình tiền xử lý dữ liệu cho bài toán phát hiện xâm nhập mạng, lựa chọn đặc trưng bao gồm **chọn đặc trưng** và **trích xuất đặc trưng** là một bước quan trọng nhằm giảm chiều dữ liệu, loại bỏ những đặc trưng không cần thiết, và cải thiện hiệu năng của mô hình học máy [57]. Dữ liệu số được gom vào vector đặc trưng bằng `VectorAssembler` (khoảng 78 đặc trưng số trước khi loại cổng; số đặc trưng chính xác sau khi loại `destination_port/source_port` là 77) và được chuẩn hóa bằng `StandardScaler` (chuẩn hóa Z-score). Ba phương pháp được sử dụng trong luận văn là *RF*, *SHAP* và *PCA*.

- a) **Principal Component Analysis (PCA)**: Là phương pháp giảm chiều dữ liệu không giám sát, nhằm biến đổi tập đặc trưng ban đầu thành một tập các thành phần chính (principal components) mới không tương quan tuyến tính với nhau và giữ lại phần lớn phương sai của dữ liệu. PCA thực hiện bằng cách tính ma trận hiệp phương sai và tìm các vector riêng (eigenvectors) tương ứng với các giá trị riêng (eigenvalues). Các thành phần chính được xác định bằng cách giải bài toán:

$$Cv = \lambda v \quad (3.1)$$

Trong đó:

- C là ma trận hiệp phương sai của dữ liệu.
- v là vector riêng (principal component).
- λ là giá trị riêng, biểu thị lượng phương sai được giải thích.

Các thành phần chính có giá trị riêng lớn nhất sẽ được lựa chọn để tạo thành không gian đặc trưng mới với số chiều thấp hơn nhưng vẫn giữ được phần lớn thông tin của dữ liệu. Đây là phương pháp được sử dụng phổ biến trong phân tích dữ liệu đa biến và học máy để giảm chiều, loại bỏ nhiễu và trực quan hóa dữ liệu [58].

- b) **SHapley Additive exPlanations (SHAP)**: Là phương pháp giải thích mô hình dựa trên lý thuyết trò chơi, dùng để đo lường mức độ đóng góp của từng đặc trưng vào dự đoán của mô hình. SHAP tính giá trị Shapley cho mỗi đặc trưng bằng cách xét trung bình đóng góp biên của đặc trưng đó trên tất cả các tập con có thể:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \quad (3.2)$$

Trong đó:

- F là tập tất cả các đặc trưng.
- S là tập con của các đặc trưng không chứa i .
- $f(S)$ là giá trị dự đoán của mô hình khi chỉ sử dụng tập đặc trưng S .
- ϕ_i là mức độ đóng góp của đặc trưng i vào kết quả dự đoán.

Các đặc trưng có giá trị SHAP lớn (theo độ lớn tuyệt đối) được xem là có ảnh hưởng quan trọng hơn đến quyết định của mô hình. Phương pháp này được ứng dụng rộng rãi trong giải thích mô hình phức tạp như XGBoost, Random Forest, và mạng nơ-ron [59].

- c) **Random Forest Feature Importance:** Thuật toán RF có khả năng đánh giá tầm quan trọng của từng đặc trưng dựa trên mức độ đóng góp vào việc giảm độ bất thuần (impurity) tại các nút phân chia của cây quyết định. Độ quan trọng $I(f_j)$ của đặc trưng f_j được tính như sau:

$$I(f_j) = \frac{1}{T} \sum_{t=1}^T \sum_{n \in N_t(f_j)} \Delta i(n) \quad (3.3)$$

Trong đó:

- T là số lượng cây trong rừng.
- $N_t(f_j)$ là tập các nút trong cây t sử dụng đặc trưng f_j để phân chia.
- $\Delta i(n)$ là lượng giảm độ bất thuần (ví dụ: Gini hoặc Entropy) tại nút n .

Các đặc trưng có độ quan trọng cao hơn sẽ được ưu tiên lựa chọn để đưa vào mô hình huấn luyện. Sau khi huấn luyện mô hình RF, các đặc trưng có giá trị quan trọng thấp hơn một ngưỡng định trước sẽ bị loại bỏ, giữ lại những đặc trưng có ảnh hưởng lớn đến việc phân loại.

3.3 Huấn luyện mô hình

3.3.1 Huấn luyện các mô hình cơ sở

Tám thuật toán phân loại có giám sát được huấn luyện độc lập trên tập đặc trưng đã qua bước xây dựng và lựa chọn đặc trưng. Bảng 3.3 liệt kê các thuật toán cùng thông tin chi tiết về biến thể và phương pháp học.

Bảng 3.3. Tám bộ phân loại cơ sở được sử dụng trong pipeline

Thuật toán	Biến thể / Ghi chú	Phương pháp học
Decision Tree	CART	Phân chia đệ quy theo Gini / Entropy
Logistic Regression	Chuẩn hóa L2	Tối ưu hàm mất mát logistic
Linear SVM	SVC tuyến tính	Tối đa hóa biên phân loại
Naive Bayes	Phân phối Gaussian	Ước lượng xác suất hậu nghiệm
Random Forest	Bagging	Tổ hợp cây quyết định ngẫu nhiên
GBT	Gradient Boosting	Tối thiểu hóa hàm mất mát theo gradient
XGBoost	Gradient Boosting tối ưu	Regularised Gradient Boosting
MLP	Mạng nơ-ron nhân tạo	Lan truyền ngược (Backpropagation)

Đáng lưu ý, LightGBM (một thư viện gradient boosting phổ biến) *không* được đưa vào danh sách trên vì thư viện gốc (native, thông qua SynapseML) chỉ hỗ trợ kiến trúc x86_64, không chạy được trên thiết bị biên Jetson Orin Nano Super (ARM64), vốn là mục tiêu triển khai của nghiên cứu; XGBoost và GBT đã đại diện đầy đủ cho họ gradient boosting.

Xử lý mất cân bằng lớp. Do tỷ lệ Benign/Attack lệch nhau đáng kể, sáu mô hình MLlib (Decision Tree, Logistic Regression, Linear SVC, Random Forest, GBT, Naive Bayes) được huấn luyện với trọng số lớp (`weightCol`) đặt theo tỷ lệ nghịch của tần suất lớp benign/attack; XGBoost dùng tham số `scale_pos_weight` với vai trò tương đương. Riêng MLP là ngoại lệ không áp trọng số lớp do API `MultilayerPerceptronClassifier` của Spark MLlib không hỗ trợ `weightCol`.

Sau khi huấn luyện, tám mô hình được xếp hạng theo chiều giảm dần của F1-Score trên tập kiểm định (validation, tách từ tập huấn luyện, xem mục Phân chia tập dữ liệu) để xác định top-3 mô hình tốt nhất phục vụ bước kết hợp ensemble tiếp theo; tập kiểm tra hoàn toàn không được sử dụng ở bước chọn mô hình này nhằm tránh rò rỉ chọn lọc (selection leakage).

3.3.2 Các phương pháp Ensemble Learning

Bảng 3.4. Các phương pháp học tổ hợp (ensemble) sử dụng trong luận văn

Phương pháp	Mô tả chi tiết
Hybrid Bagging	Top-3 mô hình kết hợp theo sơ đồ (3-2-2) sub-samples với <i>Weighted Soft Voting</i> — dự đoán xác suất có trọng số
Soft Voting	Trung bình xác suất Top-3 mô hình theo F1-Score Nhãn <i>Attack</i> nếu xác suất trung bình $\geq 0,5$

a) Áp dụng Voting

Sau khi hoàn tất giai đoạn huấn luyện đơn mô hình, luận văn tiến hành xây dựng mô hình ensemble dựa trên kỹ thuật Voting để nâng cao độ chính xác và tính ổn định của hệ thống phân loại; hai phương pháp tổ hợp được sử dụng được tóm tắt trong Bảng 3.4. Cụ thể, ba mô hình phân loại nhị phân có chỉ số F1-Score cao nhất ở giai đoạn trước sẽ được lựa chọn để tham gia vào quá trình ensemble.

Mỗi mô hình trong tập được chọn sẽ đưa ra dự đoán độc lập cho từng mẫu trong tập kiểm tra. Các dự đoán này sau đó được tổng hợp thông qua cơ chế bầu chọn. Trong trường hợp Soft Voting, xác suất dự đoán lớp *Attack* của các mô hình thành phần được lấy trung bình; mẫu được gán nhãn *Attack* nếu xác suất trung bình đạt từ 0,5 trở lên, ngược lại được gán nhãn *Benign*.

Việc áp dụng kỹ thuật Voting cho phép khai thác sự đa dạng giữa các mô hình học cơ sở, từ đó giảm sai số do thiên lệch của từng mô hình riêng lẻ và cải thiện khả năng tổng quát hóa trên dữ liệu chưa từng thấy. Nhờ cơ chế kết hợp này, hệ thống có thể nâng cao hiệu năng tổng thể trong việc phát hiện và phân loại chính xác các hành vi xâm nhập mạng. Quy trình chi tiết được trình bày trong Thuật toán 5.

Thuật toán 5. Thuật toán Soft Voting Ensemble

Đầu vào: Tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}_{i=1}^n$ với $x_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$
 Tập kiểm định D_{val} (tách từ D , dùng để xếp hạng và chọn mô hình thành phần)
 Tập dữ liệu kiểm tra $D_{\text{test}} = \{(x_j)\}_{j=1}^m$
 Tập K thuật toán học cơ sở khác loại $\{\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_K\}$

Đầu ra: Nhân dự đoán cuối cùng $y_{\text{vote}} \in \{0, 1\}$ cho mỗi mẫu

- 1: **Bước 1: Huấn luyện và đánh giá K mô hình cơ sở**
- 2: **for** $k = 1$ to K **do**
- 3: $M_k \leftarrow \text{Train}(\mathcal{L}_k, D \setminus D_{\text{val}})$ ▷ Huấn luyện trên phần train lỗi, tách khỏi D_{val}
- 4: $F_k \leftarrow \text{F1-Score}(M_k, D_{\text{val}})$ ▷ Tính điểm F1 trên tập kiểm định (validation), không chạm tập kiểm tra
- 5: **end for**
- 6: **Bước 2: Chọn Top-3 mô hình có F1 cao nhất**
- 7: Sắp xếp $\{M_1, \dots, M_K\}$ theo F_k giảm dần
- 8: Chọn 3 mô hình tốt nhất: $\mathcal{S} = \{M^{(1)}, M^{(2)}, M^{(3)}\}$ với $F^{(1)} \geq F^{(2)} \geq F^{(3)}$
- 9: **Bước 3: Thu thập xác suất dự đoán từ 3 mô hình được chọn**
- 10: **for** mỗi mẫu $x_j \in D_{\text{test}}$ **do**
- 11: **for** mỗi $M^{(s)} \in \mathcal{S}$, $s = 1, 2, 3$ **do**
- 12: $p_s^{(j)} \leftarrow P^{(s)}(y = 1 \mid x_j)$ ▷ Xác suất lớp Attack từ mô hình $M^{(s)}$
- 13: **end for**
- 14: **end for**
- 15: **Bước 4: Kết hợp bằng bầu chọn mềm (Soft Voting)**
- 16: **for** mỗi mẫu $x_j \in D_{\text{test}}$ **do**
- 17: $\bar{p}_j \leftarrow \frac{1}{3} \sum_{s=1}^3 p_s^{(j)}$ ▷ Trung bình xác suất lớp Attack
- 18: **if** $\bar{p}_j \geq 0.5$ **then**
- 19: $y_{\text{vote}}^{(j)} \leftarrow 1$ ▷ Dự đoán: Attack
- 20: **else**
- 21: $y_{\text{vote}}^{(j)} \leftarrow 0$ ▷ Dự đoán: Benign
- 22: **end if**
- 23: **end for**
- 24: **return** $\{y_{\text{vote}}^{(j)}\}_{j=1}^m$

b) Áp dụng Bagging

Để khai thác hiệu quả sức mạnh của kỹ thuật học tổ hợp Bagging, luận văn đề xuất xây dựng một mô hình ensemble lai (Hybrid Bagging Ensemble) dựa trên các mô hình cơ sở có hiệu năng cao nhất (đã huấn luyện và xếp hạng ở bước trước). Cụ thể, thay vì chỉ sử dụng một mô hình đơn lẻ, quy trình bắt đầu bằng việc huấn luyện và đánh giá K thuật toán học cơ sở trên tập dữ liệu huấn luyện D . Hiệu năng của từng mô hình được đo lường thông qua chỉ số F1-Score, từ đó lựa chọn ba mô hình có kết quả tốt nhất để đưa vào giai đoạn kết hợp.

Sau khi xác định Top-3 mô hình, phương pháp Bagging được áp dụng theo phân bố số bản sao (3, 2, 2). Đối với mỗi mô hình trong ba mô hình được chọn, nhiều tập dữ liệu bootstrap được tạo ra bằng cách lấy mẫu ngẫu nhiên có hoàn lại từ tập huấn luyện ban đầu. Trên mỗi tập bootstrap này, một bản sao của mô hình tương ứng được huấn luyện độc lập, tạo thành các *bộ học cơ sở* (*base learners*) trong ensemble. Mỗi base learner được gán trọng số tỷ lệ với F1-Score của mô hình gốc nhằm phản ánh mức độ tin cậy tương đối của nó trong quá trình tổng hợp. Việc phân bố bất đối xứng (3, 2, 2), tức cấp nhiều bản sao bootstrap hơn cho mô hình hạng cao, là một lựa chọn thiết kế thực nghiệm nhằm thiên ensemble về phía các bộ học mạnh hơn trong khi vẫn giữ tính đa dạng từ ba thuật toán khác loại; đây là một heuristic, không phải kết quả của tối ưu hóa lý thuyết, và các tỷ lệ khác (chẳng hạn 3-3-3) có thể được khảo sát trong nghiên cứu tiếp theo.

Trong giai đoạn dự đoán, toàn bộ các mô hình trong ensemble đưa ra phân phối xác suất dự đoán cho mẫu đầu vào mới. Kết quả cuối cùng được xác định thông qua cơ chế Soft Voting có trọng số, trong đó xác suất của từng lớp được tính bằng tổng có trọng số của các xác suất thành phần. Nhãn dự đoán cuối cùng được chọn là lớp có xác suất tổng hợp lớn nhất.

Cách tiếp cận này cho phép giảm phương sai của các mô hình cơ sở nhờ cơ chế lấy mẫu bootstrap, đồng thời duy trì khả năng học biểu diễn của từng thuật toán riêng lẻ. Việc kết hợp chọn lọc các mô hình có hiệu năng cao nhất trước khi áp dụng Bagging giúp tối ưu hóa cả độ chính xác và tính ổn định của hệ thống. Chi tiết quy trình được trình bày trong Thuật toán 6.

Thuật toán 6. Thuật toán Hybrid Bagging Ensemble Learning

Đầu vào: Tập dữ liệu huấn luyện $D = \{(x_i, y_i)\}_{i=1}^n$ với $x_i \in \mathbb{R}^d$, $y_i \in \{0, 1\}$ (BENIGN = 0, Attack = 1)

Tập kiểm định D_{val} (tách từ D , dùng để xếp hạng và chọn Top-3)

Tập K thuật toán học cơ sở $\{\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_K\}$

Phân bố số bản sao $\mathbf{r} = (r_1, r_2, r_3) = (3, 2, 2)$

Đầu ra: Nhãn dự đoán cuối cùng $y_{\text{bag}} \in \{0, 1\}$

1: **Bước 1: Huấn luyện và đánh giá K mô hình cơ sở**

2: **for** $k = 1$ to K **do**

3: $M_k \leftarrow \text{Train}(\mathcal{L}_k, D \setminus D_{\text{val}})$ ▷ Huấn luyện trên phần train lỗi, tách khỏi D_{val}

4: $F_k \leftarrow \text{F1-Score}(M_k, D_{\text{val}})$ ▷ Tính điểm F1 trên tập kiểm định (validation), không chạm tập kiểm tra

5: **end for**

6: **Bước 2: Chọn Top-3 mô hình có F1 cao nhất**

7: Sắp xếp $\{M_1, \dots, M_K\}$ theo F_k giảm dần

8: Chọn 3 mô hình tốt nhất: $M^{(1)}, M^{(2)}, M^{(3)}$ với $F^{(1)} \geq F^{(2)} \geq F^{(3)}$

9: **Bước 3: Tạo ensemble theo phân bố 3-2-2**

10: $\mathcal{E} \leftarrow \emptyset$ ▷ Tập ensemble gồm 7 mô hình

11: **for** $j = 1$ to 3 **do**

12: **for** $i = 1$ to r_j **do**

13: Tạo tập bootstrap $D_{j,i}$ bằng cách lấy ngẫu nhiên n mẫu từ $D \setminus D_{\text{val}}$ **có hoàn lại**

14: $h_{j,i} \leftarrow \text{Train}(\mathcal{L}^{(j)}, D_{j,i})$ ▷ Huấn luyện bản sao thứ i của mô hình hạng j

15: Gán trọng số $w_{j,i} \leftarrow F^{(j)}$ ▷ Trọng số dựa trên F1-Score

16: Thêm $(h_{j,i}, w_{j,i})$ vào $\mathcal{E}$

17: **end for**

18: **end for**

19: Chuẩn hóa trọng số: $w_{j,i} \leftarrow \frac{w_{j,i}}{\sum_{(h,w) \in \mathcal{E}} w}$

20: **Bước 4: Dự đoán cho mẫu mới x (Soft Voting có trọng số)**

21: $T \leftarrow |\mathcal{E}| = 7$ ▷ Tổng số mô hình trong ensemble

22: **for** mỗi $(h_t, w_t) \in \mathcal{E}$, $t = 1, \dots, T$ **do**

23: $\mathbf{p}_t \leftarrow h_t(x)$ ▷ Lấy vector xác suất dự đoán

24: **end for**

25: Tính xác suất có trọng số: $\bar{p}(c | x) = \sum_{t=1}^T w_t \cdot P_t(c | x), \quad \forall c \in \{0, 1\}$

26: $y_{\text{bag}} \leftarrow \arg \max_{c \in \{0,1\}} \bar{p}(c | x)$

27: **return** y_{bag}

3.4 Đánh giá mô hình

Mỗi mô hình (cơ sở và ensemble) được đánh giá toàn diện trên bốn chiều:

Bảng 3.5. Các tiêu chí đánh giá mô hình

Tiêu chí	Mô tả
Độ chính xác (Accuracy)	Tỉ lệ dự đoán đúng trên toàn bộ mẫu kiểm tra
Precision	Tỉ lệ cảnh báo đúng trong tổng số cảnh báo được phát ra
Recall	Tỉ lệ phát hiện đúng trong tổng số mẫu tấn công thực tế
F1-Score	Trung bình điều hòa của Precision và Recall
Area Under the Curve – Receiver Operating Characteristic	Diện tích dưới đường cong ROC — đo hiệu năng phân biệt lớp
Area Under the Curve – PR	Diện tích dưới đường cong Precision-Recall
Chi phí tính toán	Thời gian huấn luyện, thời gian dự đoán và kích thước mô hình
Khả năng giải thích (Explainable Artificial Intelligence)	Biểu đồ SHAP: summary plot, bar chart và waterfall plot

Hiệu năng của mô hình được đánh giá thông qua các chỉ số phân loại phổ biến như Accuracy, Precision, Recall và F1-Score, kết hợp với phân tích đường cong AUC (ROC và PR), chi phí tính toán (thời gian huấn luyện và suy luận), cũng như khả năng giải thích của mô hình (ví dụ: biểu đồ tổng quan SHAP); các tiêu chí đánh giá được tổng hợp trong Bảng 3.5. Để kiểm chứng mô hình phát hiện xâm nhập, tập dữ liệu kiểm tra được sử dụng nhằm đánh giá khả năng tổng quát hóa của mô hình đã huấn luyện, trong đó kết quả dự đoán được đối chiếu với nhãn thực tế và đo lường bằng các thước đo đánh giá. Các chỉ số như accuracy (AC), precision (PR), recall (RE), F1-score (FS) được tính toán theo các công thức (3.4), (3.5), (3.6) và (3.7). Những kết quả này là cơ sở quan trọng để so sánh giữa các phương pháp lựa chọn đặc trưng và lựa chọn mô hình phù hợp cho triển khai thực tế.

$$\text{Accuracy (AC)} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.4)$$

$$\text{Precision (PR)} = \frac{TP}{TP + FP} \quad (3.5)$$

$$\text{Recall (RE)} = \frac{TP}{TP + FN} \quad (3.6)$$

$$\text{F1-Score (FS)} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.7)$$

Các đại lượng TP, FP, FN, TN dùng trong các công thức trên được minh họa qua ma trận nhầm lẫn ở Bảng 3.6.

Bảng 3.6. Ma trận nhầm lẫn của mô hình phát hiện xâm nhập

	Dự đoán: Positive	Dự đoán: Negative
Thực tế: Positive	TP (True Positive)	FN (False Negative)
Thực tế: Negative	FP (False Positive)	TN (True Negative)

Trong đó, TP (True Positive) là số lượng lưu lượng mạng hoặc phiên kết nối thực sự là tấn công và được hệ thống IDS phát hiện đúng là xâm nhập. TN (True Negative) là những lưu lượng bình thường (không phải tấn công) và được mô hình phân loại chính xác là hợp lệ. FP (False Positive) là các lưu lượng bình thường nhưng bị hệ thống nhận diện nhầm là tấn công. Ngược lại, FN (False Negative) là những cuộc tấn công thực sự xảy ra nhưng không được hệ thống phát hiện và bị phân loại nhầm là lưu lượng bình thường.

3.5 Tối ưu hóa tham số và huấn luyện mô hình

Trong quá trình xây dựng mô hình học máy, việc lựa chọn các siêu tham số (hyperparameters) phù hợp là một yếu tố quan trọng ảnh hưởng đến hiệu năng của mô hình. Để tối ưu hóa các siêu tham số này, phương pháp **Tìm kiếm theo lưới** (Grid Search (GS)) kết hợp với **Cross-Validation (CV)** đã được sử dụng trong môi trường phân tán của Apache Spark. Các mô hình phân loại được huấn luyện và so sánh gồm: Decision Tree, Logistic Regression, Linear SVC, Naive Bayes, Random Forest, GBT, XGBoost và MLP; sau đó xếp hạng theo F1-Score để chọn Top-3 mô hình cho các phương pháp ensemble.

- **GS**: Là phương pháp thử nghiệm tất cả các tổ hợp có thể của các siêu tham số được định nghĩa trước. Ví dụ, với một mô hình Random Forest, các tham số có thể bao gồm số lượng cây (numTrees), độ sâu tối đa (maxDepth), và chiến lược chọn tập con đặc trưng tại mỗi nút (featureSubsetStrategy). Spark MLlib hỗ trợ đối tượng ParamGridBuilder để xây dựng lưới các tham số.
- **CV**: Là phương pháp đánh giá mô hình bằng cách chia dữ liệu huấn luyện thành k phần, sau đó huấn luyện và đánh giá mô hình k lần, mỗi lần sử dụng một phần khác nhau làm dữ liệu kiểm tra và các phần còn lại làm dữ liệu huấn luyện. Trong Spark, đối tượng CrossValidator được sử dụng để thực hiện quy trình này.

Quy trình tối ưu hóa được thực hiện như sau:

1. Xây dựng pipeline gồm các bước tiền xử lý dữ liệu (ví dụ: chuẩn hóa, biến đổi đặc trưng) và mô hình học máy.
2. Sử dụng ParamGridBuilder để định nghĩa tập lưới các siêu tham số cần tìm kiếm.
3. Khởi tạo CrossValidator với pipeline, lưới các tham số, và bộ đánh giá hiệu năng (ví dụ: BinaryClassificationEvaluator).
4. Huấn luyện mô hình bằng cách gọi phương thức fit() của CrossValidator trên tập dữ liệu huấn luyện.
5. Chọn mô hình tốt nhất dựa trên điểm đánh giá trung bình trên các folds.

Ví dụ đoạn mã sử dụng Spark MLlib:

```
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
from pyspark.ml import Pipeline

rf = RandomForestClassifier(labelCol="label", featuresCol="
    features")

pipeline = Pipeline(stages=[rf])

paramGrid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [10, 20, 50]) \
    .addGrid(rf.maxDepth, [5, 10, 15]) \
    .build()

evaluator = BinaryClassificationEvaluator(labelCol="label",
    metricName="areaUnderPR")

cv = CrossValidator(estimator=pipeline,
                    estimatorParamMaps=paramGrid,
                    evaluator=evaluator,
                    numFolds=5)

cvModel = cv.fit(trainingData)
```

Quy trình này giúp đảm bảo lựa chọn được tổ hợp siêu tham số tối ưu, đồng thời giúp tối ưu thời gian huấn luyện. Spark giúp tăng tốc quá trình này nhờ khả năng xử lý phân tán và song song hóa trên cụm máy.

3.6 Triển khai biên

Mô hình tốt nhất sau đánh giá được đóng gói thành `PipelineModel` và triển khai trên một cụm *hai* thiết bị biên NVIDIA Jetson Orin Nano Super (CPU ARM 64-bit + GPU NVIDIA), tách vai trò: Jetson #1 chạy cổng phát hiện bất thường (autoencoder) để lọc lưu lượng bình thường, Jetson #2 chạy bộ phân loại PySpark trên các luồng nghi vấn. Mô hình sau khi huấn luyện trên cụm được chuyển sang thiết bị biên bằng Secure Copy Protocol (SCP). Quy trình suy luận thời gian thực theo chuỗi: Kafka Consumer nhận dữ liệu mạng (topic `ids-network-flow`), cổng bất thường chuyển tiếp luồng nghi vấn sang `ids-suspicious-flow`, PySpark Streaming (micro-batch) xử lý và trích xuất đặc trưng, kết quả dự đoán ghi vào PostgreSQL phục vụ trực quan hóa trên Grafana và kích hoạt cảnh báo khi phát hiện bất thường. Hệ thống hỗ trợ ba chế độ: tách pipeline (pipeline split), mở rộng theo chiều ngang (horizontal scaling) và cụm Spark (Spark cluster). Thông số phần cứng chi tiết của thiết bị biên được trình bày trong Bảng 3.7.

Bảng 3.7. Thông số phần cứng thiết bị biên

Thuộc tính	Giá trị
Nền tảng	NVIDIA Jetson Orin Nano Super Developer Kit (×2)
CPU	6 nhân Arm Cortex-A78AE, 1,7 GHz
GPU	NVIDIA Ampere, 1024 nhân CUDA + 32 Tensor Core
RAM	8 GB LPDDR5 (102 GB/s)
Lưu trữ	SSD NVMe 256 GB
Hiệu năng AI	tới ~67 TOPS (khung 25 W)
Hệ điều hành	Ubuntu (NVIDIA JetPack / L4T)
Vai trò	Worker huấn luyện (Spark) & thiết bị suy luận biên

Chương 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

Trong chương này, luận văn trình bày kết quả đạt được khi áp dụng kỹ thuật lựa chọn đặc trưng để huấn luyện các mô hình học máy. Bên cạnh đó, quy trình tinh chỉnh siêu tham số bằng tìm kiếm lưới kết hợp kiểm định chéo (cross-validation) cũng được trình bày, cùng với đánh giá kết quả thực nghiệm khi áp dụng các thuật toán học máy như SVM, LR, NB, RF, DT, GBT, XGB và MLP cả trong trường hợp có và không sử dụng kỹ thuật giảm chiều dữ liệu. Ngoài ra, luận văn cũng xác định và trình bày các giá trị tham số tối ưu cho từng thuật toán thông qua phương pháp tìm kiếm theo lưới kết hợp với kiểm định chéo (cross-validation). Dựa trên các kết quả thu được, luận văn sẽ làm rõ vấn đề liệu việc giảm chiều dữ liệu có thực sự cần thiết hay không. Cuối cùng, luận văn đánh giá hiệu quả của các mô hình EL, sau đó so sánh với các thuật toán học máy truyền thống, từ đó trả lời câu hỏi: *Liệu có nên sử dụng EL so với các mô hình học máy truyền thống khi làm việc với dữ liệu IoT hay không?*

4.1 Môi trường thực nghiệm

Việc *huấn luyện* mô hình được thực hiện trên một **cụm Apache Spark Standalone phân tán** gồm một máy chủ và **hai thiết bị biên NVIDIA Jetson Orin Nano Super Developer Kit**. Điểm đáng chú ý là cùng một cặp Jetson vừa đóng vai trò *worker* của cụm Spark trong pha huấn luyện, vừa là thiết bị *triển khai* trong pha suy luận thời gian thực (Hình 4.5), giúp kiểm chứng tính khả thi của toàn bộ pipeline trên đúng phần cứng mục tiêu.

- **Máy chủ (Spark master):** máy tính xách tay MacBook Air, vi xử lý Apple M3, RAM 16 GB, macOS Sonoma 14.3; đảm nhận điều phối cụm (master), tiền xử lý dữ liệu thô (từ CSV sang Parquet) và chạy hạ tầng Docker (Kafka, PostgreSQL, InfluxDB, Grafana).
- **Hai nút worker: NVIDIA Jetson Orin Nano Super Developer Kit;** cấu hình chi tiết (CPU/GPU/RAM/lưu trữ) nêu tại Bảng 3.7, Chương 3; hệ điều hành Ubuntu (NVIDIA JetPack/L4T). Jetson #1 đồng thời chạy tiến trình *driver* của Spark (driver là tiến trình điều khiển ứng dụng, khác với vai trò *master* điều phối cụm của MacBook).
- **Phần mềm:** Python 3, Apache Spark / PySpark 3.4.1, MLlib; cấu hình cụm tiêu biểu: `executor.memory=3 GB`, `executor.cores=4`, `driver.memory=3 GB`,

`shuffle.partitions=32`; mỗi Jetson được cấp 8 GB swap trên ổ NVMe để tránh tràn bộ nhớ khi cao điểm (xem `cluster/spark_cluster.env`, `cluster/setup_swap_jetson.sh`).

Các thuật toán học máy, phương pháp giảm chiều và các bước đánh giá đều được triển khai bằng PySpark và thực thi phân tán trên hai nút Jetson; tập dữ liệu Parquet được đồng bộ sang cả hai worker trước khi huấn luyện. Kiến trúc cụm huấn luyện phân tán (gồm vai trò *master* điều phối trên MacBook và hai *worker/executor* trên Jetson) được minh họa ở Hình 4.1. Cần phân biệt rõ với Hình 4.5 (pha suy luận): cùng một cặp Jetson nhưng ở pha huấn luyện đóng vai trò worker của cụm Spark, còn ở pha suy luận đóng vai trò cổng lọc (gate) và bộ phân loại.

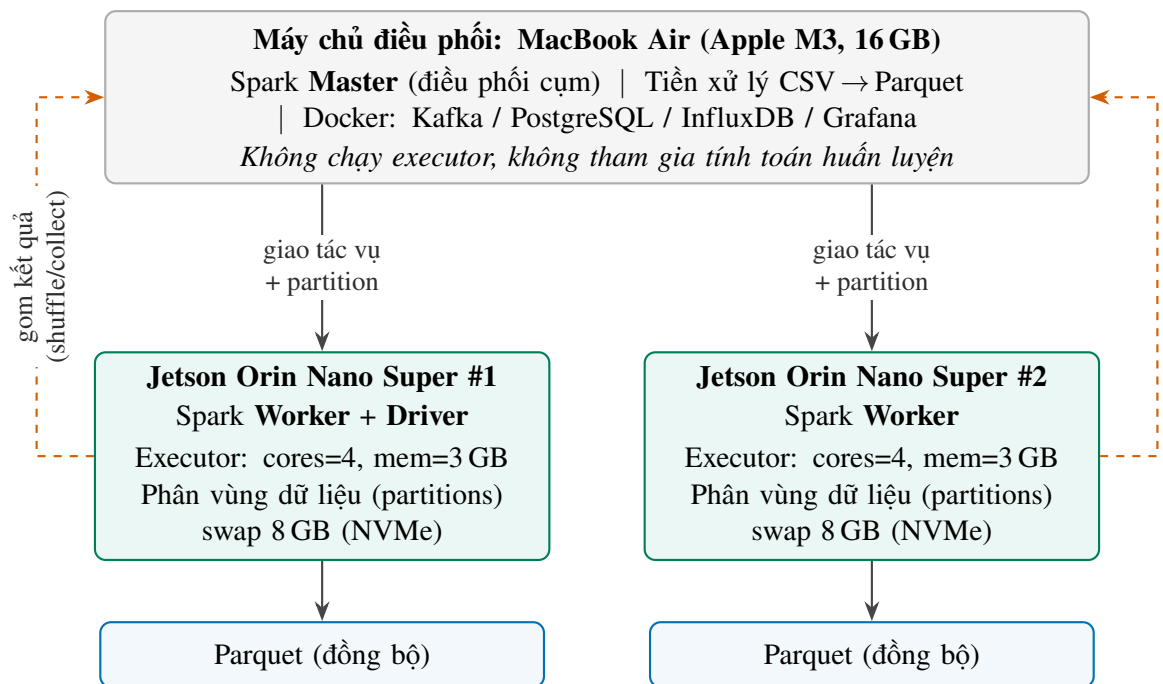
Cần nhấn mạnh rằng máy chủ MacBook *chỉ* đảm nhận vai trò *master* điều phối cụm cùng tiền xử lý và hạ tầng dịch vụ, mà *không* tham gia tính toán huấn luyện (không chạy *executor*). Lựa chọn này là có chủ đích: toàn bộ số liệu hiệu năng được báo cáo nhằm phản ánh trung thực khả năng *huấn luyện phân tán ngay trên phần cứng biên* Jetson, đúng với mục tiêu cốt lõi của luận văn. Nếu đưa MacBook (vi xử lý Apple M3, mạnh hơn đáng kể) vào làm nút tính toán, phần lớn khối lượng công việc sẽ dồn về máy chủ, gây mất cân bằng tải (hiện tượng *straggler*) và làm sai lệch các phép đo thời gian/thông lượng, khiến kết quả không còn phản ánh đúng năng lực của cụm biên. Vì vậy, cụm đo đạc được giữ *đồng nhất* gồm hai nút Jetson; máy chủ chỉ được dùng để phát triển và kiểm thử nhanh trong giai đoạn xây dựng pipeline.

4.2 Kết quả thực nghiệm

Bảng 4.1 tóm tắt thiết lập tham số của các mô hình học máy được sử dụng trong nghiên cứu này. Trong đó, XGB và MLP được tinh chỉnh bằng Grid Search kết hợp kiểm định chéo (Kịch bản 3); các mô hình còn lại dùng bộ tham số cố định từ trước, lựa chọn dựa trên các nghiên cứu gần đây và thực nghiệm sơ bộ, nhằm bảo đảm khác biệt quan sát được giữa các phương pháp giảm chiều phản ánh đúng phương pháp thay vì mức độ tinh chỉnh riêng từng mô hình.

Bảng 4.1. Tham số tối ưu của các mô hình học máy

Mô hình	Tham số	Giá trị	Ý nghĩa/Mô tả
Decision Tree	maxDepth	15	Bắt các mẫu tần công phức tạp.
	impurity	entropy	Tiêu chí chia nhánh tối ưu.
	minInstancesPerNode	5	Số mẫu tối thiểu tại mỗi nút lá.
Logistic Reg.	maxIter	200	Đảm bảo hội tụ trên tập dữ liệu lớn.
	regParam	0,001	Kiểm soát mức điều chuẩn (regularization).
	elasticNetParam threshold	0,0 0,5	Sử dụng Ridge (L2) ổn định hơn. Ngưỡng quyết định nhị phân.
SVM (LinearSVC)	maxIter	200	Số lần lặp tối ưu hóa biên.
	regParam	0,001	Tham số điều chuẩn biên quyết định.
NB	modelType	gaussian	Phù hợp cho đặc trưng liên tục.
	smoothing	1,0	Làm mượt Laplace.
Random Forest	numTrees	200	Đảm bảo độ ổn định kết quả.
	maxDepth	15	Chiều sâu tối đa của cây rừng.
	featureSubsetStrategy	sqrt	Chọn đặc trưng ngẫu nhiên mỗi nút.
GBT	maxIter	150	Số vòng boosting tối đa.
	maxDepth	6	Chiều sâu của các cây yếu.
	stepSize	0,05	Tốc độ học (learning rate) của boosting.
	subsamplingRate	0,8	Tỉ lệ lấy mẫu dữ liệu mỗi vòng boosting.
XGB	n_estimators	300	Số lượng cây boost trong mô hình.
	max_depth	8	Chiều sâu tối đa mỗi cây.
	learning_rate	0,05	Kiểm soát mức độ học của cây mới.
	subsample	0,8	Tỉ lệ lấy mẫu dữ liệu mỗi cây.
	colsample_bytree	0,8	Tỉ lệ lấy mẫu đặc trưng mỗi cây.
MLP	layers	[d , 64, 32, 2]	Mạng nơ-ron 2 lớp ẩn (d = số đặc trưng đầu vào).
	maxIter	150	Số vòng lặp huấn luyện tối đa.
	stepSize	0,01	Tốc độ học của mạng nơ-ron.



Hình 4.1. Cụm huấn luyện phân tán Apache Spark Standalone: MacBook giữ vai trò *master* điều phối (không chạy executor), hai Jetson Orin Nano Super là *worker/executor* thực thi tính toán; dữ liệu Parquet được phân vùng và đồng bộ sang cả hai nút, kết quả được gom về qua shuffle/collect.

4.2.1 Kịch bản 1: Đánh giá hiệu năng cơ sở (Baseline)

Trong kịch bản này, toàn bộ tập đặc trưng số (khoảng 78 đặc trưng trước khi loại cổng; số chính xác sau khi loại `destination_port/source_port` là 77) được đưa vào mô hình mà chưa qua bất kỳ bước lựa chọn đặc trưng hay giảm chiều nào. Mục tiêu là thiết lập một mức hiệu năng tham chiếu cho các thử nghiệm tiếp theo.

Kết quả hiệu năng của các thuật toán ML đơn lẻ và phương pháp Ensemble được trình bày trong Bảng 4.2.

Bảng 4.2. Kết quả hiệu năng phân lớp kịch bản Baseline (toàn bộ đặc trưng số, ≈ 78 trước khi loại cổng)

Thuật toán	Accuracy	Precision	Recall	F1-Score	AUC-ROC	AUC-PR
Decision Tree	0,9986	0,9925	0,9983	0,9954	0,9994	0,9988
Logistic Regression	0,9288	0,6937	0,9422	0,7991	0,9859	0,9509
SVM (LinearSVC)	0,9199	0,6650	0,9395	0,7788	0,9843	0,9485
NB	0,3163	0,1793	0,9933	0,3038	0,8829	0,6364
Random Forest	0,9985	0,9939	0,9960	0,9949	0,9999	0,9997
GBT	0,9980	0,9900	0,9965	0,9932	0,9998	0,9991
XGB	0,9991	0,9950	0,9993	0,9971	1,0000	0,9999
MLP	0,9900	0,9844	0,9487	0,9662	0,9984	0,9936
Hybrid Bagging	0,9990	0,9950	0,9984	0,9967	1,0000	0,9999
Soft Voting	0,9991	0,9952	0,9985	0,9969	1,0000	0,9999

Với cấu hình đã loại trừ rò rỉ dữ liệu, nhóm mô hình dựa trên cây (Random Forest, XGB, GBT) được kỳ vọng dẫn đầu, với F1 thấp hơn mức gần-tuyệt-đối của các cấu hình có rò rỉ và phản ánh đúng độ khó thực của bài toán; kết luận “mô hình nào tốt nhất” chỉ được đưa ra khi chênh lệch vượt khoảng tin cậy (xem mục kiểm định thống kê). Do dữ liệu mất cân bằng mạnh, AUC-PR và recall trên lớp tấn công được ưu tiên hơn Accuracy khi diễn giải, vì Accuracy có thể gây lạc quan giả tạo (một mô hình luôn đoán lớp benign chiếm đa số vẫn đạt Accuracy cao).

4.2.2 Kịch bản 2: So sánh chiến lược giảm chiều dữ liệu

Chúng tôi thực hiện so sánh 3 chiến lược chính nhằm giảm số lượng đặc trưng nhưng vẫn đảm bảo hiệu năng:

1. **RF Feature Importance:** Trích xuất 30 đặc trưng có điểm số quan trọng cao nhất từ Random Forest.
2. **SHAP Importance:** Sử dụng giá trị Shapley từ mô hình XGB để chọn ra 30 đặc trưng có ảnh hưởng lớn nhất.

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

3. **PCA (k=40)**: Giảm chiều dữ liệu từ không gian đặc trưng gốc (≈ 78 chiều trước khi loại cổng) xuống 40 thành phần chính.

Lưu ý rằng PCA dùng $k = 40$ — nhiều hơn 30 đặc trưng của RF/SHAP, vì đây là phép *trích xuất* chứ không phải *chọn lọc*: mỗi thành phần chính chỉ là một tổ hợp tuyến tính mang một phần phương sai, nên cần nhiều thành phần hơn để giữ lượng thông tin (phương sai tích lũy) tương đương với tập 30 đặc trưng gốc. Trong dải quét chung $\{20, 30, 40\}$, $k = 40$ là mức giữ phương sai cao nhất, nên là điểm vận hành đại diện công bằng nhất cho PCA; ép $k = 30$ sẽ làm PCA mất thêm phương sai và bất lợi khi so sánh.

Bảng 4.3 tóm tắt sự khác biệt giữa các phương pháp. Mô hình đại diện (cột *Mô hình tốt nhất*) của mỗi phương pháp được chọn theo F1 trên tập kiểm định (validation) tách từ tập huấn luyện; tập kiểm tra không tham gia khâu chọn lọc này. Mục tiêu là kiểm chứng giả thuyết: lựa chọn đặc trưng (RF/SHAP Top-30) giữ được hiệu năng gần với baseline trong khi giảm hơn 60% số đặc trưng, qua đó phù hợp cho triển khai biên.

Bảng 4.3. So sánh hiệu năng tốt nhất của các phương pháp lựa chọn đặc trưng và giảm chiều

Phương pháp	Số đặc trưng	Mô hình tốt nhất	F1-Score	Huấn luyện (s)	Dung lượng (MB)
Baseline (Toàn bộ)	77	XGB	0,9971	4615,5	2,32
RF Importance	30	XGB	0,9922	1295,1	2,52
SHAP Importance	30	XGB	0,9970	1226,1	0,003 [†]
PCA (k=40)	40	XGB	0,9939	1444,8	3,82

Baseline dùng toàn bộ 77 đặc trưng số (≈ 78 trước khi loại cổng). [†]Kích thước đo bằng cách tuần tự hóa mô hình *trên cụm huấn luyện phân tán*: bước lưu mô hình phân tán ghi các phân vùng dữ liệu rừng cây trên executor đang giữ chúng, nên giá trị cỡ KB chỉ bắt được metadata cục bộ trên driver và *thấp hơn thực tế*. Vì dung lượng ensemble cây phụ thuộc số cây và số nút (không phụ thuộc số đặc trưng đầu vào), mô hình XGB SHAP Top-30 thực tế tương đương bản RF Top-30 ($\approx 2,5$ MB). Cũng vì vậy, tập đặc trưng rút gọn có thể cho mô hình *lớn hơn* baseline một chút (2,52 so với 2,32 MB): ít ứng viên tách hơn khiến cây cần nhiều nút hơn để đạt cùng độ tinh khiết. Phép đo tin cậy là bản mô hình xuất ra ở chế độ đơn nút của pipeline triển khai (Mục kết quả biên).

Ngoài ra, sự so sánh trực tiếp giữa hai phương pháp lựa chọn đặc trưng dựa trên độ quan trọng của RF và SHAP được trình bày chi tiết trong Bảng 4.4.

4.2.3 Kịch bản 3: Tối ưu hóa tham số

Sau khi chọn được bộ đặc trưng tối ưu từ SHAP, bốn mô hình Spark ML gốc (Random Forest, Decision Tree, GBT, Logistic Regression) được tinh chỉnh tham số thông qua Grid Search kết hợp Cross-validation trên tập đặc trưng SHAP Top-30; các mô hình còn lại (XGB, MLP, SVM, NB) giữ nguyên bộ tham số cố định ở Bảng 4.1 (xem lý do ở đầu mục).

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

Bảng 4.4. So sánh hiệu năng lựa chọn đặc trưng: RF Importance vs SHAP Importance (Top-30)

Thuật toán	F1 (RF Top-30)	F1 (SHAP Top-30)	Chênh lệch tương đối (%)
Random Forest	0,9879	0,9955	+0,77
XGB	0,9922	0,9970	+0,48
GBT	0,9844	0,9918	+0,75
Decision Tree	0,9835	0,9948	+1,15

Việc tinh chỉnh giúp các mô hình cây đạt F1 quanh mức 0,995 (Bảng 4.5); mức cải thiện so với cấu hình cố định là nhỏ do bài toán nhị phân trong-miền đã gần bão hòa, phù hợp với nhận định ở mục kiểm định thống kê.

Kết quả hiệu năng của phương pháp Tối ưu hóa tham số được trình bày trong Bảng 4.5.

Bảng 4.5. Kết quả hiệu năng phân lớp kịch bản Tối ưu hóa tham số

Thuật toán	Accuracy	Precision	Recall	F1-Score	AUC-ROC	AUC-PR
Decision Tree	0,9987	0,9977	0,9936	0,9957	0,9994	0,9980
Logistic Regression	0,9541	0,9787	0,7094	0,8226	0,9749	0,9371
SVM (LinearSVC)	giữ cấu hình cố định (xem Bảng 4.2)					
NB	giữ cấu hình cố định (xem Bảng 4.2)					
Random Forest	0,9986	0,9985	0,9922	0,9953	0,9999	0,9997
GBT	0,9985	0,9979	0,9924	0,9951	0,9999	0,9993
XGB	giữ cấu hình cố định (xem Bảng 4.2)					
MLP	giữ cấu hình cố định (xem Bảng 4.2)					
Hybrid Bagging	0,9986	0,9956	0,9949	0,9953	0,9999	0,9998
Soft Voting	0,9988	0,9985	0,9932	0,9958	1,0000	0,9998

4.2.4 Phân tích khả năng giải thích với SHAP

Tính giải thích (explainability) là một thành phần thiết yếu trong hệ thống IDS, nhằm cung cấp cơ sở cho việc xây dựng niềm tin và hỗ trợ vận hành thực tế. Trong nghiên cứu này, phương pháp SHAP (SHapley Additive exPlanations) được sử dụng để phân tích mức độ quan trọng của từng đặc trưng đối với quyết định phân loại của mô hình XGB.

Hình 4.2 xếp hạng 30 đặc trưng quan trọng nhất theo giá trị trung bình tuyệt đối của SHAP (mean $|\text{SHAP}|$): thanh càng dài, đặc trưng càng ảnh hưởng lớn đến quyết định phân loại xét trên toàn bộ mẫu. Để tránh rò rỉ, giá trị SHAP được tính trên một *mẫu phân tầng theo lớp lấy từ tập huấn luyện* (không phải tập kiểm tra); mẫu này được biến đổi qua đúng bộ chuẩn hóa (StandardScaler) đã fit trên tập huấn luyện trước khi tính SHAP, bảo đảm

không gian đầu vào của phép giải thích trùng khớp với không gian đầu vào mà mô hình đã học.

Bổ sung cho xếp hạng tổng thể, Hình 4.3 trình bày biểu đồ SHAP Summary dạng beeswarm: mỗi điểm đại diện cho một quan sát; trục hoành thể hiện giá trị SHAP (dương đẩy dự đoán về lớp tấn công, âm đẩy về lớp bình thường); màu sắc biểu diễn giá trị đặc trưng từ thấp đến cao. Độ phân tán theo trục hoành phản ánh mức độ ảnh hưởng của đặc trưng, độ tập trung thể hiện tính ổn định của tác động.

Bên cạnh đó, giá trị trung bình tuyệt đối của SHAP ($\text{mean } |\text{SHAP}|$) được dùng để xếp hạng mức quan trọng tổng thể. Trên tập đặc trưng đã loại trừ rò rỉ, các đặc trưng có $\text{mean } |\text{SHAP}|$ lớn nhất là: `bwd_packet_length_std`, `init_win_bytes_backward`, `init_win_bytes_forward` và `bwd_packet_length_mean`.

Các đặc trưng như `init_win_bytes_forward`, `init_win_bytes_backward` (kích thước cửa sổ TCP giai đoạn khởi tạo) và `bwd_packet_length_std`, `packet_length_std` (sự biến thiên độ dài gói tin) phản ánh đặc tính phân phối lưu lượng — là các tín hiệu mạng có ý nghĩa, không phụ thuộc vào định danh dịch vụ. Việc loại `destination_port` buộc mô hình dựa vào các đặc trưng hành vi này thay vì ghi nhớ ánh xạ cổng → nhãn.

4.2.5 Phân tích rò rỉ dữ liệu (data leakage)

Đặc trưng `destination_port` là một nguồn rò rỉ nhãn tiềm tàng trên CICIDS2017: mỗi loại tấn công được sinh nhằm vào những cổng dịch vụ cố định nên cổng đích tương quan mạnh với nhãn, khiến mô hình dễ “ghi nhớ” ánh xạ cổng–nhãn thay vì học hành vi luồng. Engelen và cộng sự [60] xếp hiện tượng này vào nhóm *shortcut learning* của CICIDS2017: các thuộc tính định danh (Flow ID, địa chỉ IP, cổng nguồn, nhãn thời gian) phải bị loại khỏi biểu diễn đặc trưng, còn riêng việc dùng cổng đích làm đặc trưng cho IDS thì các tác giả xem là *còn gây tranh cãi*. Để định lượng ảnh hưởng, luận văn thực hiện phân tích loại bỏ (ablation) giữ/loại cổng với cùng một cấu hình Random Forest cố định (Bảng 4.6). Điều đáng lưu ý là với bài toán *nhị phân trong-miền*, sau khi đã loại trùng lặp luồng ở mức đặc trưng, ảnh hưởng riêng của `destination_port` lên F1 chỉ còn *nhỏ* (chênh khoảng 0,001 điểm F1), bởi tác vụ nhị phân trên CICIDS2017 vốn gần mức bão hòa và nguồn rò rỉ *chi phối* (các luồng trùng lặp) đã được xử lý trước bằng bước loại bỏ trùng lặp. Điều này *không* có nghĩa rò rỉ vô hại: giá trị của quy trình loại-trừ-rò-rỉ thể hiện rõ hơn ở đánh giá *đa lớp* với các lớp tấn công hiếm (Mục 4.2.9) và ở *tổng quát hóa*

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

liên bộ dữ liệu (Mục 4.2.8.1), nơi các “lỗi tắt” rò rỉ không còn giúp mô hình. Pipeline loại bỏ cả `destination_port` lẫn `source_port`; phân tích ablation tập trung vào `destination_port` vì đây là cổng chi phối.

Bảng 4.6. Phân tích loại bỏ (ablation) rò rỉ: ảnh hưởng của đặc trưng `destination_port` (Random Forest cố định a priori: `numTrees=200`, `maxDepth=15`, có trọng số lớp; nhị phân)

Cấu hình	F1	Recall (Attack)	AUC-PR
Giữ <code>destination_port</code> (rò rỉ)	0,9965	0,9988	0,9999
Loại <code>destination_port</code> (đề xuất)	0,9955	0,9961	0,9997

Cả hai lượt dùng cùng một cấu hình Random Forest cố định từ trước (200 cây, độ sâu 15, trọng số lớp, khớp Bảng 4.1) để cô lập đúng ảnh hưởng của đặc trưng rò rỉ. Mức chênh lệch F1 giữa hai hàng phản ánh phần hiệu năng bị thổi phồng do đặc trưng rò rỉ; ở đây mức chênh nhỏ vì đã loại trùng lặp trước và tác vụ nhị phân gần bão hòa, nhưng chiều hướng (giữ cổng cho F1/recall cao hơn) vẫn nhất quán với hiện tượng rò rỉ. Mọi kết quả khác trong luận văn dùng cấu hình đã loại cổng.

4.2.6 Phân tích hiệu năng hệ thống

Bên cạnh độ chính xác, hiệu năng hệ thống về thời gian và tài nguyên là yếu tố then chốt cho việc triển khai thực tế. Kết quả trong Bảng 4.7 cho thấy:

Bảng 4.7. So sánh thời gian xử lý và kích thước mô hình kịch bản Baseline

Thuật toán	Huấn luyện (s)	Dự đoán (s)	Kích thước (MB)
Decision Tree	33,4	6,1	0,0004
Logistic Regression	49,2	5,4	0,009
SVM (LinearSVC)	224,3	5,4	0,002
NB	8,2	5,2	0,012
Random Forest	610,3	18,6	3,69
GBT	1346,8	8,9	0,92
XGB	4615,5	9,0	2,32
MLP	314,7	8,0	0,0004
Hybrid Bagging	11934,5	36,4	7,24
Soft Voting	–	25,6	–

- **Thời gian huấn luyện:** Các mô hình Gradient Boosting (GBT, XGB) có thời gian huấn luyện dài nhất nhưng đạt kết quả phân loại cao.

- **Thời gian dự báo:** XGB có thời gian dự báo nhanh (khoảng 9 giây cho toàn tập kiểm tra), Random Forest chậm hơn (khoảng 19 giây); cả hai phù hợp cho giám sát theo lô gần thời gian thực.
- **Kích thước mô hình:** các mô hình cây chiếm dung lượng lớn hơn hẳn mô hình tuyến tính (LR, SVM, NB chỉ vài KB); RF khoảng 3,7 MB và ensemble Hybrid Bagging lớn nhất ($\sim 7,2$ MB) do gộp nhiều bộ học cơ sở. *Lưu ý: kích thước được đo bằng cách tuần tự hóa (serialize) PipelineModel của Spark ra đĩa; giá trị này có thể biến thiên theo cách phân mảnh/partition khi lưu nên chỉ mang tính tham khảo tương đối, không dùng làm tiêu chí quyết định lựa chọn mô hình.*

4.2.7 Kiểm định ý nghĩa thống kê

Do các con số F1 giữa những phương pháp/thuật toán hàng đầu rất sát nhau, luận văn không kết luận “mô hình tốt nhất” chỉ dựa trên một lần đo. Mô hình đại diện của mỗi phương pháp, cũng như *cặp phương pháp dẫn đầu* được đưa vào kiểm định, đều được chọn theo F1 trên tập kiểm định (validation, 20% tách từ tập huấn luyện, $\text{seed}=42$); tập kiểm tra không tham gia khâu chọn lọc. Hai phương pháp này sau đó được huấn luyện lại trên N lần lấy mẫu lại tập huấn luyện/kiểm tra (mặc định $N=6$, ghép cặp trên cùng một phép chia ở mỗi vòng), tính F1 trung bình kèm hai loại khoảng tin cậy 95%: khoảng tin cậy t thông thường và khoảng tin cậy hiệu chỉnh Nadeau–Bengio [61], trong đó phương sai nhân hệ số $(1/N + n_{\text{test}}/n_{\text{train}})$ để bù tương quan giữa các lần lấy mẫu lại chồng lấn; khoảng Nadeau–Bengio được dùng làm căn cứ kết luận chính. Ý nghĩa thống kê được kiểm bằng *permutation test ghép cặp hai phía*, liệt kê đầy đủ toàn bộ $2^N = 64$ hoán vị đổi dấu (kiểm định chính xác, không xấp xỉ Monte Carlo), do đó p nhỏ nhất khả dĩ là $2/64 \approx 0,031$; độ lớn hiệu ứng được báo cáo bằng Cohen’s d ghép cặp (Bảng 4.8). Kết luận: chênh lệch F1 giữa Baseline và SHAP Top-30 có ý nghĩa thống kê ở mức sàn ($p \approx 0,031$, $d \approx 1,94$), nhưng khác biệt tuyệt đối rất nhỏ ($\approx 0,0001$ điểm F1) nên hai phương pháp *tương đương về mặt thực dụng*; khi đó tiêu chí phụ (khả năng giải thích, kích thước, chi phí biên) mới là căn cứ lựa chọn. Cần thừa nhận rằng với $N=6$, kiểm định có *lực thống kê thấp*; vì vậy một giá trị p không có ý nghĩa tự nó chỉ là bằng chứng yếu cho “tương đương”, và kết luận được căn cứ đồng thời vào khoảng tin cậy Nadeau–Bengio và Cohen’s d ; tăng N (kèm kiểm định chéo lồng) là hướng củng cố tiếp theo.

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

Bảng 4.8. Độ ổn định qua nhiều lần lấy mẫu và kiểm định ghép cặp (từ `statistical_validity_multiseed.csv`)

Phương pháp	Mô hình	F1 (mean $\pm$ CI95 t)	CI95 Nadeau–Bengio	p (ghép cặp)	Cohen’s d
Baseline (toàn bộ)	XGB	0,99734 $\pm$ 0,00007	[0,99722; 0,99746]	0,031	1,94
SHAP Top-30	XGB	0,99720 $\pm$ 0,00009	[0,99705; 0,99735]		

4.2.8 Kiểm tra độ ổn định trong-phân-phối

Mô hình đại diện của mỗi phương pháp được đánh giá lại trên một mẫu con của tập kiểm tra (không giao với tập huấn luyện; Bảng 4.9). Cần nói rõ: đây chỉ là phép kiểm tra *độ ổn định cùng phân phối* (in-distribution), không phải kiểm thử dịch chuyển phân phối; đánh giá tổng quát hóa thực sự nằm ở thí nghiệm liên bộ dữ liệu (Mục 4.2.8.1).

Bảng 4.9. Kiểm tra độ ổn định trong-phân-phối trên mẫu con tập kiểm tra (từ `robustness_holdout_summary.csv`)

Phương pháp	Mô hình	F1 (holdout)	AUC-PR
Baseline	XGB	0,9971	0,9999
RF Top-30	XGB	0,9921	0,9998
SHAP Top-30	XGB	0,9969	0,9999
PCA	XGB	0,9939	0,9998

4.2.8.1 Tổng quát hóa liên bộ dữ liệu

Đánh giá trên tập giữ lại ở trên là phép thử *cùng bộ* (in-domain, cùng phân phối). Để kiểm chứng khả năng *tổng quát hóa* thực sự, mô hình được huấn luyện trên một bộ dữ liệu và kiểm tra trên bộ *khác* (đánh giá *khác bộ*, cross-dataset; và ngược lại) trên tập đặc trưng đã loại trừ rõ ràng *chung* của hai bộ. Do hai bộ dữ liệu đặt tên cột khác nhau (CSE-CIC-IDS2018 dùng tên viết tắt kiểu CICFlowMeter v3), tên cột của CSE-CIC-IDS2018 được ánh xạ về tên chuẩn của CICIDS2017 qua một bảng bí danh (alias) tường minh trong mã nguồn; danh sách đặc trưng chung sau ánh xạ được lưu tại `common_features.txt` để bảo đảm tái lập. Mô hình dùng cho thí nghiệm là Random Forest có trọng số lớp (`weightCol`), nhất quán với cơ chế xử lý mất cân bằng ở Chương 3. Khoảng cách giữa F1 cùng bộ và F1 khác bộ (Bảng 4.10) định lượng mức suy giảm khi gặp dịch chuyển phân phối, một thước đo độ ổn định chặt chẽ hơn holdout cùng phân phối.

Do CSE-CIC-IDS2018 [54] có quy mô rất lớn (~ 16 triệu luồng, ~ 6 GB), vượt khả năng xử lý thoải mái của thiết bị biên Jetson (8 GB), thí nghiệm này dùng *mẫu phân tầng theo*

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

nhãn của CSE-CIC-IDS2018 (tỷ lệ 0,2, $\sim 3,14$ triệu luồng lấy mẫu, còn $\sim 2,39$ triệu sau khử trùng lặp mức đặc trưng, $seed=42$, công tắc `IDS_SAMPLE_FRAC`) nhằm bảo toàn tỷ lệ các lớp trong khi vừa với phần cứng; CICIDS2017 dùng $\sim 2,23$ triệu luồng. Toàn bộ quy trình tiền xử lý và căn chỉnh CSE-CIC-IDS2018 cho thí nghiệm liên bộ dữ liệu được trình bày ở Thuật toán 7; các bước loại trừ rò rỉ (giao đặc trưng chung không chứa cổng/IP, khử trùng lặp mức đặc trưng) hoàn toàn nhất quán với quy trình cho CICIDS2017 (Thuật toán 4). Các con số mẫu được báo cáo tường minh để bảo đảm tính tái lập; việc mở rộng lên toàn bộ CSE-CIC-IDS2018 là hướng phát triển tiếp theo.

Thuật toán 7. Tiền xử lý và căn chỉnh CSE-CIC-IDS2018 cho đánh giá liên bộ dữ liệu

Đầu vào: Tập tệp CSV của CSE-CIC-IDS2018; lược đồ tên cột chuẩn của CICIDS2017 $\mathcal{S}_{2017}$;

bảng bí danh tên cột $\mathcal{A}$ ($2018 \rightarrow 2017$); tỷ lệ lấy mẫu $r = 0.2$; $seed = 42$

Đầu ra: Bộ dữ liệu D_{2018} trên tập đặc trưng chung đã loại rò rỉ $\mathcal{F}_{\text{chung}}$, sẵn sàng cho thí nghiệm khác bộ

- 1: **Bước 1: Nạp và làm sạch** (giống Thuật toán 4)
 - 2: Đọc các tệp CSV; `CLEANCOLUMNAMES`; `HANDLEINFINITYVALUES`; loại null và bản ghi trùng lặp hoàn toàn
 - 3: **Bước 2: Căn chỉnh tên cột**
 - 4: $\forall c \in \text{columns}(D) : c \leftarrow \mathcal{A}(c)$ $\triangleright$ ánh xạ tên cột CICFlowMeter v3 về tên chuẩn của CICIDS2017
 - 5: **Bước 3: Tạo nhãn nhị phân (không phân biệt hoa/thường)**
 - 6: $y_i \leftarrow 0$ nếu `LOWER(labeli) = "benign"`, ngược lại $y_i \leftarrow 1$ $\triangleright$ 2018 ghi "Benign"; so khớp không phân biệt hoa/thường
 - $\rightarrow$ **Loại trừ rò rỉ dữ liệu: Bước 4 và Bước 6**
 - 7: **Bước 4 [loại rò rỉ – I]: Giao đặc trưng chung, đã loại trừ rò rỉ**
 - 8: $\mathcal{F}_{\text{chung}} \leftarrow (\mathcal{F}_{2017} \cap \mathcal{F}_{2018}) \setminus \mathcal{C}_{\text{exclude}}$ $\triangleright \mathcal{C}_{\text{exclude}}$: `destination_port`, `source_port`, `protocol`, `IP`, `timestamp`, `flow_id`
 - 9: Lưu $\mathcal{F}_{\text{chung}}$ vào `common_features.txt` $\triangleright$ cố định cho cả hai chiều huấn luyện/kiểm tra
 - 10: **Bước 5: Lấy mẫu phân tầng theo nhãn**
 - 11: $D \leftarrow \text{STRATIFIEDSAMPLE}(D, r, seed = 42)$ $\triangleright$ bảo toàn tỷ lệ lớp; còn $\sim 3,14$ triệu luồng
 - 12: **Bước 6 [loại rò rỉ – II]: Loại trùng lặp ở mức đặc trưng** (tắt định, như Bước 4b của Thuật toán 4)
 - 13: Gom nhóm theo $\mathcal{F}_{\text{chung}}$; loại toàn bộ nhóm mâu thuẫn nhãn; giữ đúng một đại diện của mỗi nhóm nhất quán $\triangleright$ còn $\sim 2,39$ triệu luồng
 - 14: **return** D_{2018} trên tập đặc trưng $\mathcal{F}_{\text{chung}}$
-

Diễn giải. Kết quả cho thấy một tương phản rõ rệt và *trung thực*: F1 cùng bộ rất cao ở cả hai chiều (0,9952 và 0,9245) nhưng F1 khác bộ giảm mạnh xuống 0,04–0,05, chủ yếu do recall gần bằng không (0,022 và 0,030) trong khi precision chiều 2017 sang 2018 vẫn đạt 0,596, tức mô hình gần như không nhận ra tấn công của bộ dữ liệu kia, chứ không tạo

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

Bảng 4.10. Tổng quát hóa liên bộ dữ liệu giữa CICIDS2017 và CSE-CIC-IDS2018 (CSE-CIC-IDS2018: mẫu phân tầng `seed=42`; từ `cross_dataset_results.csv`)

Huấn luyện	Kiểm tra	Loại	F1
CICIDS2017	CICIDS2017	cùng bộ	0,9952
CICIDS2017	CSE-CIC-IDS2018	khác bộ	0,0423
CSE-CIC-IDS2018	CSE-CIC-IDS2018	cùng bộ	0,9245
CSE-CIC-IDS2018	CICIDS2017	khác bộ	0,0535

ra cảnh báo sai tràn lan. Nguyên nhân là *dịch chuyển phân phối đặc trưng* giữa hai testbed: khác phiên bản CICFlowMeter (thang đo và cách tính một số đặc trưng thay đổi), khác hạ tầng mạng và cấu hình dịch vụ, khác tổ hợp loại tấn công (HOIC/LOIC-HTTP của 2018 không có trong 2017). Sự sụt giảm này nhất quán với các nghiên cứu cross-dataset trên IDS dựa trên đặc trưng luồng [62, 63, 64], và chính là luận cứ mạnh nhất của luận văn: *con số trong-miền gần hoàn hảo (kể cả khi đã loại rò rỉ) không bảo đảm khả năng triển khai sang môi trường khác*; đánh giá liên bộ dữ liệu phải là một phần bắt buộc của quy trình kiểm định IDS [52], và các hướng thích ứng miền (chuẩn hóa theo miền, huấn luyện liên bộ, cập nhật theo drift) là điều kiện tiên quyết cho triển khai thực tế.

4.2.9 Đánh giá đa lớp theo loại tấn công

Phân loại nhị phân (Benign/Attack) trên CICIDS2017 vốn dễ tách; để bộc lộ điểm yếu thực sự, luận văn bổ sung đánh giá *đa lớp theo từng loại tấn công gốc* (precision/recall/F1 từng lớp, macro-F1, weighted-F1 và ma trận nhầm lẫn), gồm lớp Benign và 14 loại tấn công của CICIDS2017. Kết quả cho thấy tương phản rất rõ giữa hai chỉ số tổng hợp: *weighted-F1* $\approx 0,998$ (bị chi phối bởi các lớp đa số) nhưng *macro-F1 chỉ* $\approx 0,806$, chính là bằng chứng cho thấy độ chính xác nhị phân gần hoàn hảo đã *che giấu* năng lực phát hiện kém ở các lớp hiếm. Cụ thể, các lớp đa số (Benign, DoS Hulk, DDoS, PortScan) đạt F1 rất cao ($\geq 0,97$), trong khi các lớp cực hiếm suy giảm nghiêm trọng: Web Attack – XSS recall chỉ 0,027, Web Attack – SQL Injection F1 = 0 (chỉ 6 mẫu), Bot recall 0,46. Đây là nơi khác biệt giữa các phương pháp lựa chọn đặc trưng thể hiện rõ nhất và cũng là hướng cần cải thiện (xử lý mất cân bằng, tăng cường dữ liệu lớp hiếm). Kết quả chi tiết theo từng lớp được trình bày trong Bảng 4.11, và ma trận nhầm lẫn đa lớp tương ứng được minh họa ở Hình 4.4.

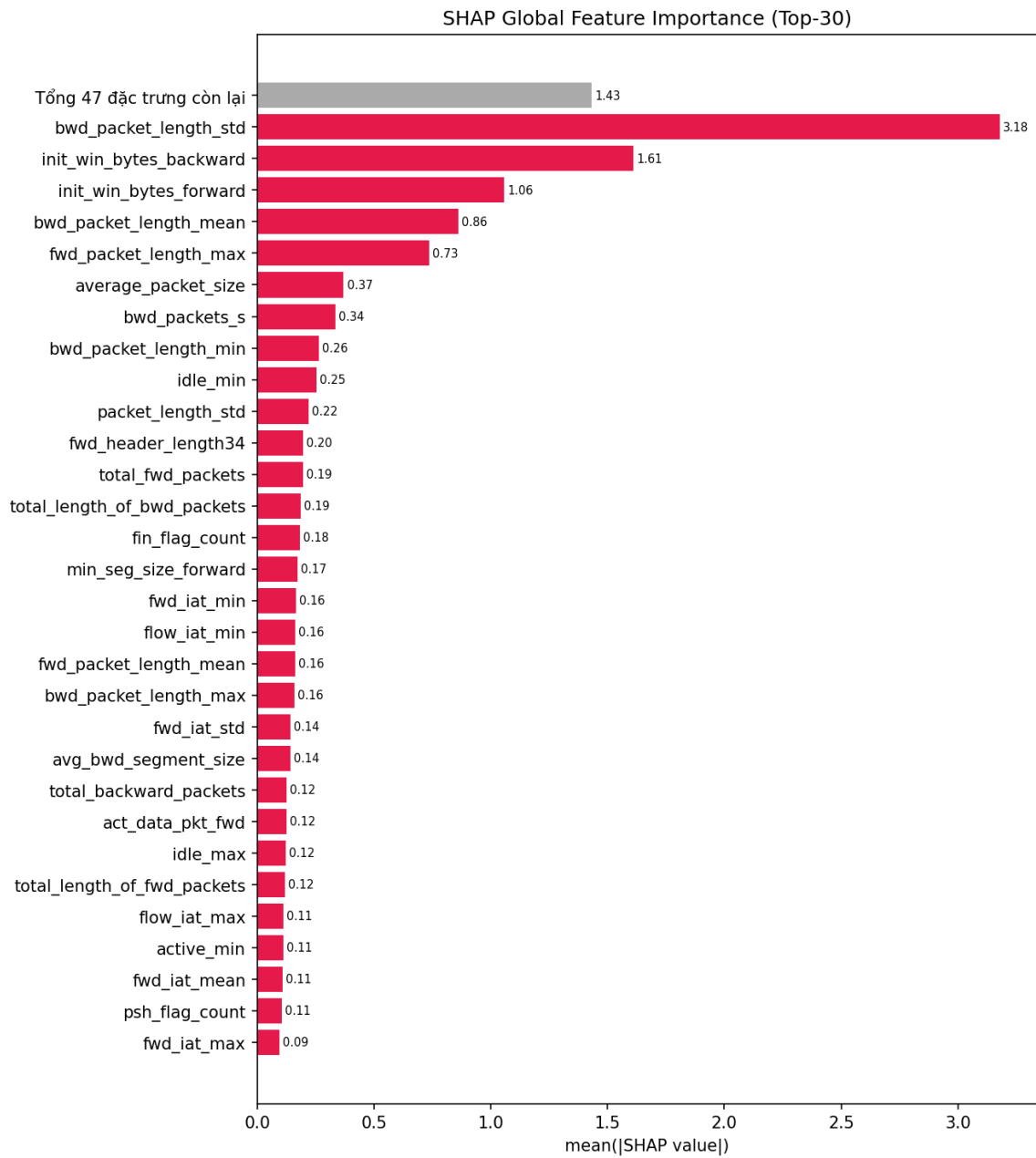
Phân tích hướng nhầm lẫn giữa các lớp hiếm. Đọc ma trận nhầm lẫn theo *đích đến* của các dự đoán sai cho thấy các lớp hiếm suy giảm theo hai kiểu khác nhau. Kiểu thứ nhất là *nhầm loại nhưng vẫn bị phát hiện*: 81/111 mẫu Web Attack – XSS bị gán thành Web

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

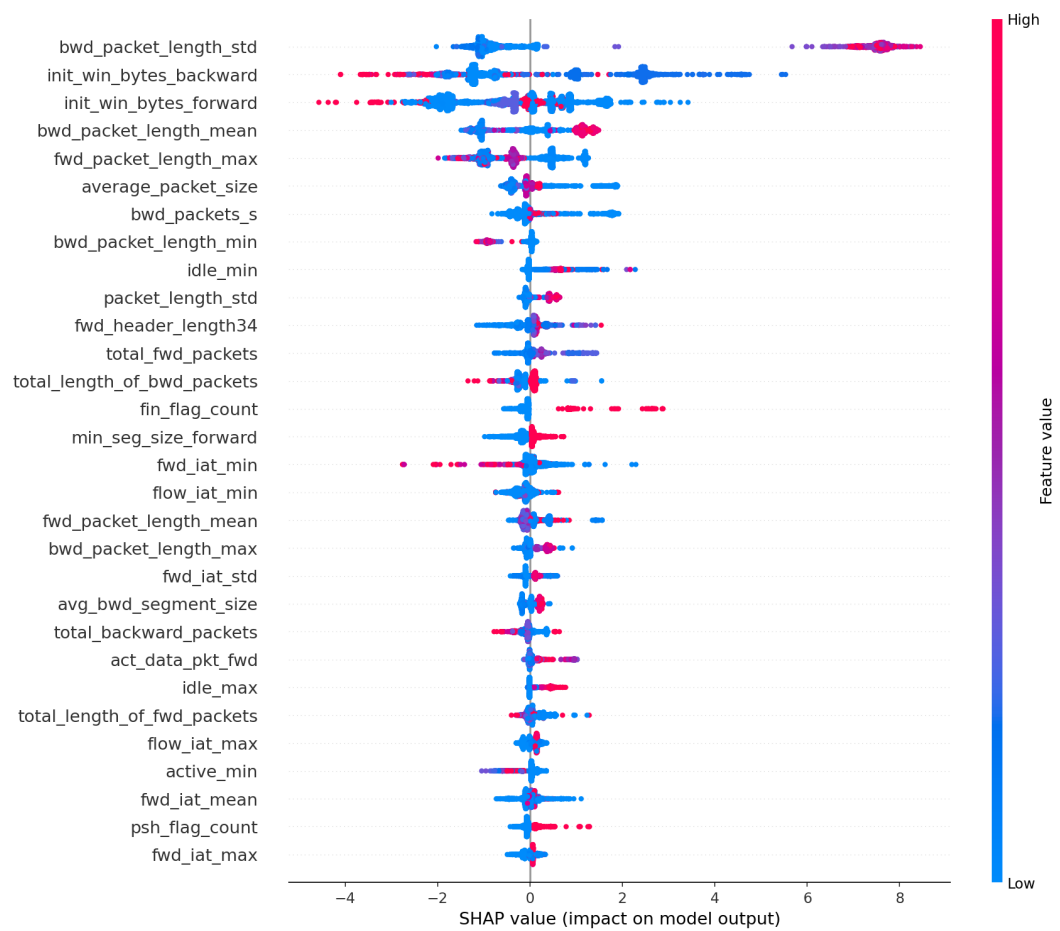
Bảng 4.11. Hiệu năng theo từng lớp tấn công (từ `per_class_metrics.csv`)

Lớp	Support	Precision	Recall	F1
Benign	380.119	0,9981	0,9997	0,9989
DoS Hulk	34.446	0,9990	0,9904	0,9947
DDoS	25.694	0,9996	0,9992	0,9994
DoS GoldenEye	2.020	1,0000	0,9757	0,9877
FTP-Patator	1.147	0,9965	0,9965	0,9965
DoS slowloris	1.068	0,9953	0,9925	0,9939
DoS Slowhttptest	1.023	0,9286	0,9912	0,9589
SSH-Patator	642	1,0000	0,9346	0,9662
PortScan	383	1,0000	0,9530	0,9759
Web Attack – Brute Force	311	0,7346	0,7299	0,7323
Bot	290	1,0000	0,4552	0,6256
Web Attack – XSS	111	1,0000	0,0270	0,0526
Infiltration	12	1,0000	0,6667	0,8000
Web Attack – SQL Injection	6	0,0000	0,0000	0,0000
Heartbleed	3	1,0000	1,0000	1,0000
macro-F1 / weighted-F1			0,806 / 0,998	

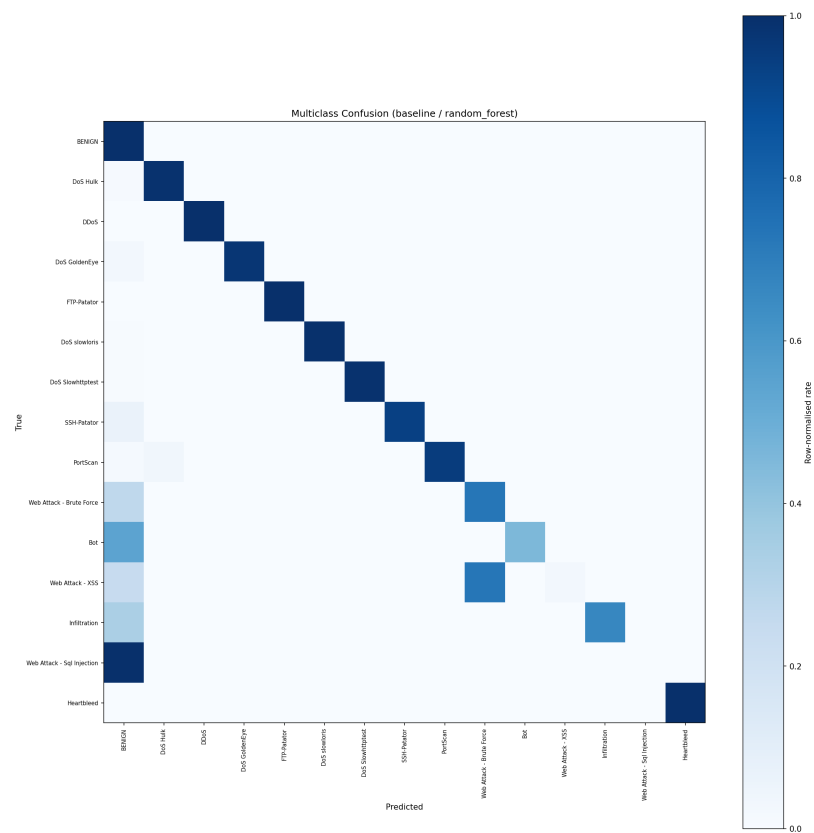
Attack – Brute Force, tức ở bài toán nhị phân triển khai trên biên, các luồng này *vẫn bị gấn cò tấn công* (76% XSS vẫn bị chặn), chỉ sai ở khâu định danh loại. Kiểu thứ hai nguy hiểm hơn: *lọt hẫng thành Benign*: toàn bộ 6/6 mẫu SQL Injection, 158/290 mẫu Bot (54%) và 84/311 mẫu Web Brute Force bị dự đoán là lưu lượng bình thường. Ba cơ chế giải thích hiện tượng này: (i) *mất cân bằng cực đoan* (6–311 mẫu so với 380.119 mẫu Benign) khiến các lớp hiểm gần như không có vùng quyết định riêng; (ii) việc *loại destination_port* ảnh hưởng nặng nhất đến nhóm tấn công web: khi mất “lõi tắt” cổng 80, các luồng HTTP tấn công (request/response ngắn) có đặc trưng hành vi rất gần lưu lượng duyệt web thường và gần lẫn nhau; giá trị recall 0,027 của XSS là con số *trung thực* thay cho con số đẹp nhờ rò rỉ; (iii) *Bot lẫn Benign do bản chất*: các luồng beaconing C&C nhỏ, đều đặn, giống lưu lượng nền khi không còn định danh cổng/dịch vụ. Nhận diện này định hướng trực tiếp cho hướng phát triển: xử lý mất cân bằng có chủ đích (tăng cường/oversampling lớp hiểm), bổ sung đặc trưng chuỗi thời gian cho hành vi beaconing, và kiểm chứng liên bộ dữ liệu (Mục 4.2.8.1).



Hình 4.2. Top 30 đặc trưng quan trọng theo giá trị trung bình tuyệt đối SHAP của mô hình XGB



Hình 4.3. Biểu đồ SHAP Summary (beeswarm): phân bố và hướng tác động của từng đặc trưng



Hình 4.4. Ma trận nhầm lẫn đa lớp theo loại tấn công (chuẩn hóa theo hàng)

4.3 Thực nghiệm và đánh giá hiệu năng mô hình trên Jetson Orin Nano Super

Trong phần này, luận văn đề xuất xây dựng một hệ thống phát hiện xâm nhập mạng IDS *phân tán* triển khai trên một cụm **hai thiết bị biên NVIDIA Jetson Orin Nano Super** [65] nhằm đánh giá khả năng hoạt động trong môi trường tài nguyên hạn chế; đây là hướng tiếp cận điện toán biên đưa phân tích về gần nguồn dữ liệu để giảm băng thông và thời gian phản hồi [66, 67], trong khi các nghiên cứu IDS biên trước đây chủ yếu đánh giá suy luận đơn nút [68]. Hệ thống được thiết kế theo kiến trúc xử lý dữ liệu thời gian thực hai tầng: một *cổng phát hiện bất thường* (anomaly gate) bằng autoencoder nhẹ lọc bớt lưu lượng bình thường trên Jetson #1, và một *bộ phân loại* PySpark PipelineModel chỉ xử lý các luồng nghi vấn trên Jetson #2. Hạ tầng trung tâm (Apache Kafka cho truyền tải thông điệp, PostgreSQL để quản lý dữ liệu, InfluxDB để lưu chỉ số hiệu năng theo thời gian, Grafana để trực quan hóa và Mailtrap để kiểm thử cảnh báo qua email) chạy trên một máy chủ điều phối.

NVIDIA Jetson Orin Nano Super là một máy tính nhúng (single-board computer) tích hợp CPU kiến trúc ARM 64-bit và GPU NVIDIA, hỗ trợ tăng tốc suy luận học máy ngay tại biên; cấu hình chi tiết đã nêu tại Bảng 3.7 (Chương 3) và mục Môi trường thực nghiệm. Do hạn chế về tài nguyên tính toán so với máy chủ truyền thống, việc triển khai trên Jetson Orin Nano Super cho phép đánh giá khả năng tối ưu hóa mô hình và hiệu năng suy luận trong điều kiện phần cứng nhúng (edge computing).

Mô hình có hiệu năng tối ưu được xác định ở phần đánh giá hiệu năng phân lớp phía trên sẽ được tích hợp vào pipeline xử lý nhằm thực hiện nhiệm vụ phát hiện và phân loại lưu lượng mạng thành hai nhóm: *Benign* và *Attack*. Các chỉ số đánh giá bao gồm độ chính xác (Accuracy), Precision, Recall, F1-Score, thời gian dự đoán trung bình cho mỗi mẫu, thông lượng xử lý (throughput) và mức tiêu thụ tài nguyên hệ thống (CPU, RAM). Kết quả thực nghiệm cho phép phân tích tính khả thi của việc triển khai hệ thống IDS dựa trên học máy trong môi trường thiết bị biên có tài nguyên giới hạn.

4.3.1 Các công cụ sử dụng trong hệ thống

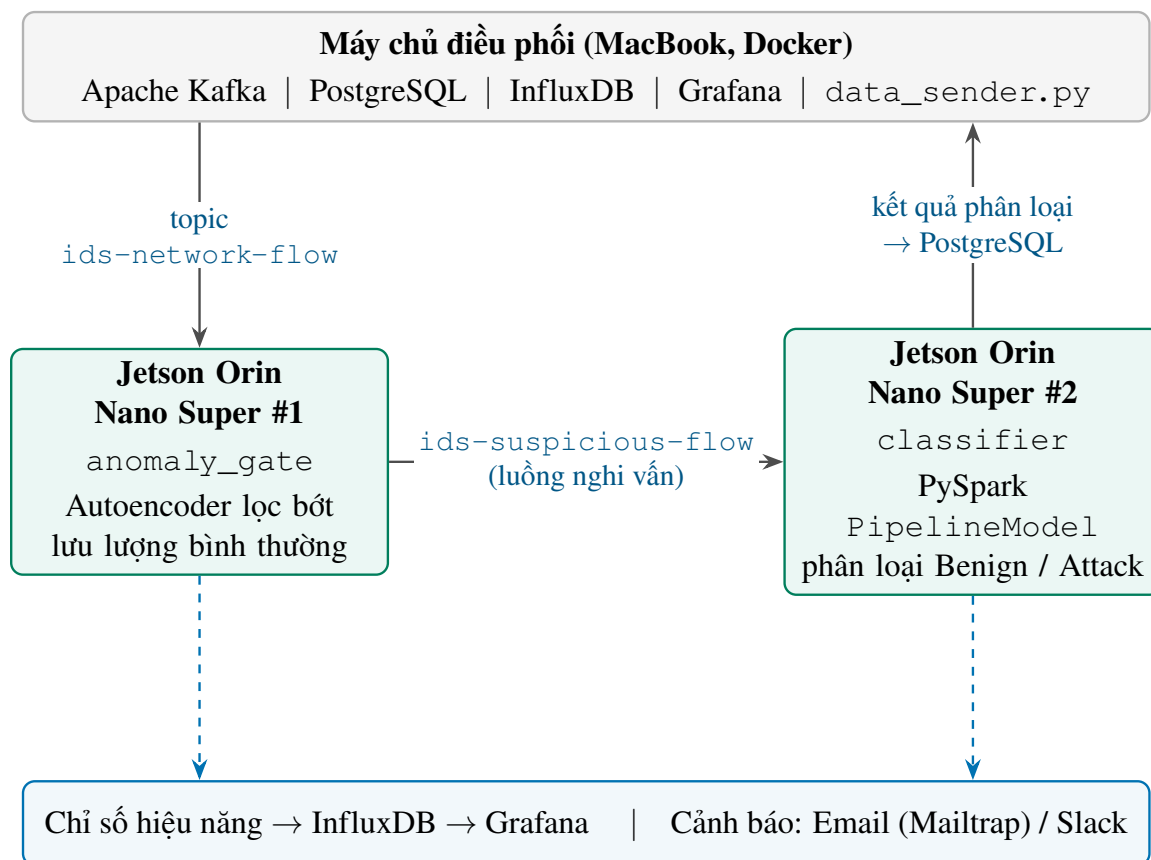
Trong hệ thống phát hiện xâm nhập mạng IDS được đề xuất, ngoài các công nghệ xử lý dữ liệu lớn đã trình bày ở Chương 2, luận văn sử dụng thêm một số công cụ và nền tảng mã nguồn mở nhằm hỗ trợ truyền tải dữ liệu, lưu trữ, giám sát hiệu năng và cảnh báo theo thời gian thực. Cụ thể:

- **Apache Kafka** [69] – Là nền tảng truyền tải thông điệp phân tán (distributed event streaming platform), được sử dụng để tiếp nhận và phân phối dữ liệu luồng theo cơ chế publish-subscribe. Trong hệ thống, Kafka đóng vai trò trung gian giữa nguồn dữ liệu (CICIDS2017 được mô phỏng bằng Python script) và thiết bị Jetson Orin Nano Super, đảm bảo khả năng truyền tải bất đồng bộ, chịu lỗi và mở rộng linh hoạt.
- **Apache Spark Streaming** – Là thành phần xử lý dữ liệu luồng thời gian thực. Spark Streaming nhận dữ liệu từ Kafka Consumer, thực hiện các bước tiền xử lý đặc trưng và chuyển dữ liệu đến mô hình học máy đã được huấn luyện để thực hiện suy luận.
- **PostgreSQL** – Là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở có độ ổn định và khả năng mở rộng cao. Trong hệ thống, PostgreSQL được sử dụng để lưu trữ kết quả dự đoán, thông tin cảnh báo và các bản ghi phục vụ báo cáo tổng hợp.
- **InfluxDB** – Là cơ sở dữ liệu chuỗi thời gian (Time-Series Database), tối ưu cho việc lưu trữ và truy vấn các chỉ số theo thời gian. Hệ thống sử dụng InfluxDB để ghi nhận các thông số hiệu năng như thời gian suy luận, mức sử dụng CPU, bộ nhớ và thông lượng xử lý.
- **Grafana** – Là nền tảng trực quan hóa dữ liệu mã nguồn mở, cho phép xây dựng các dashboard theo thời gian thực. Grafana kết nối với InfluxDB và PostgreSQL để hiển thị tình trạng hoạt động của hệ thống IDS cũng như thống kê các sự kiện tấn công.
- **Mailtrap** – Là công cụ kiểm thử email trong môi trường giả lập. Trong giai đoạn thực nghiệm, Mailtrap được sử dụng để kiểm tra cơ chế gửi cảnh báo khi hệ thống phát hiện lưu lượng mạng bất thường trước khi triển khai môi trường thực tế.

Việc tích hợp các công nghệ trên giúp hệ thống đảm bảo khả năng xử lý dữ liệu thời gian thực, lưu trữ an toàn, giám sát hiệu năng liên tục và kích hoạt cảnh báo kịp thời khi phát hiện hành vi xâm nhập.

4.3.2 Mô hình hệ thống đề xuất

Hệ thống phát hiện xâm nhập được thiết kế theo kiến trúc xử lý dữ liệu luồng như minh họa ở Hình 4.5. Luồng xử lý tổng thể bao gồm các bước sau:



Hình 4.5. Kiến trúc hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm 2 Jetson Orin Nano Super

1. **Nguồn dữ liệu:** Dữ liệu từ bộ CICIDS2017 được mô phỏng theo thời gian thực thông qua một Python script và gửi vào Kafka Producer trên topic `ids-network-flow`.
2. **Cổng phát hiện bất thường (Jetson #1):** Jetson #1 đảm nhận vai trò `anomaly_gate`: một autoencoder nhẹ (huấn luyện chỉ trên lưu lượng bình thường) tính sai số tái tạo (reconstruction error) cho từng luồng. Các luồng có sai số dưới ngưỡng được coi là bình thường và *được lọc bỏ*; chỉ các luồng *ngghi vấn* mới được chuyển tiếp sang topic `ids-suspicious-flow`. Cơ chế này giảm đáng kể khối lượng phải đưa vào suy luận Spark.
3. **Bộ phân loại (Jetson #2):** Jetson #2 đảm nhận vai trò `classifier`: tải mô hình PySpark PipelineModel (với tập đặc trưng SHAP Top-30 đã chọn) và phân loại các luồng nghi vấn thành *Benign* hoặc *Attack*. Vector đầu vào khi triển khai gồm 29 đặc trưng: trong Top-30 theo xếp hạng SHAP có `destination_port`, nhưng đặc trưng này bị loại khỏi mô hình xuất ra để triển khai vì là kênh rò rỉ nhãn trên CICIDS2017 (Mục 4.2.5), nhất quán với giao thức chống rò rỉ ở pha huấn luyện.
4. **Ghi nhận hiệu năng:** Mỗi nút ghi nhận thời gian dự đoán/mẫu, mức sử dụng CPU/RAM/nhiệt độ và thông lượng (gắn nhãn theo `EDGE_NODE_ID`); các chỉ số được lưu vào InfluxDB.
5. **Lưu trữ và cảnh báo:** Kết quả dự đoán và sự kiện tấn công được lưu vào PostgreSQL; khi phát hiện lưu lượng bất thường, hệ thống kích hoạt cảnh báo qua Email/Webhook (Mailtrap, Slack trong môi trường kiểm thử) từ một nút chính được chỉ định để tránh trùng lặp.
6. **Trực quan hóa và giám sát:** Grafana truy xuất dữ liệu từ InfluxDB và PostgreSQL để xây dựng dashboard theo thời gian thực, giám sát trạng thái hệ thống và thống kê các sự kiện tấn công.

Ba chế độ triển khai phân tán. Cụm hai Jetson hỗ trợ ba chế độ, lựa chọn theo ràng buộc tải và độ trễ:

- **Chế độ A – Pipeline split** (khuyến nghị): Jetson #1 = cổng bất thường (anomaly gate), Jetson #2 = bộ phân loại (classifier) (như mô tả trên), giúp giảm tải Spark nhờ lọc lưu lượng bình thường sớm.

- **Chế độ B – Mở rộng theo chiều ngang (Horizontal scaling):** cả hai Jetson chạy pipeline đầy đủ trong cùng một nhóm tiêu thụ (consumer group) của Kafka nhằm tăng thông lượng.
- **Chế độ C – Spark cluster:** máy chủ MacBook làm Spark master, hai Jetson làm worker, phân tán suy luận Spark trên cụm. Nhất quán với nguyên tắc ở mục Môi trường thực nghiệm, MacBook chỉ đảm nhận điều phối (master), không chạy executor; toàn bộ tính toán suy luận vẫn nằm trên hai Jetson.

Kiến trúc này đảm bảo tính mô-đun, tách biệt các thành phần xử lý, đồng thời phù hợp với mô hình triển khai phân tán trên các thiết bị biên có tài nguyên hạn chế như Jetson Orin Nano Super.

Đặc tả cổng bất thường (operating point). Cổng autoencoder là một bộ lọc *điều chỉnh được* chứ không phải một điểm cố định: nâng ngưỡng giúp giảm tải (offload) nhiều lưu lượng bình thường hơn khỏi Spark (tăng tỷ lệ bỏ qua tại cổng, gate-skip) nhưng có thể giảm recall trên lớp tấn công. Ngưỡng của cổng được chọn theo phân vị (quantile) của sai số tái tạo tính trên một *tập kiểm định benign*, gồm 10% tách từ phần lưu lượng bình thường của tập huấn luyện (`seed=42`); tập kiểm tra không tham gia việc chọn ngưỡng, tránh rò rỉ kiểu “ngưỡng oracle”. Bảng 4.12 trình bày đường đánh đổi giữa recall và mức giảm tải (recall–offload) khi quét các mức quantile trên tập kiểm định này; điểm vận hành được chọn theo ngân sách độ trễ/độ chính xác của triển khai.

Bảng 4.12. Điểm vận hành của cổng bất thường khi quét quantile trên tập kiểm định benign (10% benign train, `seed=42`; từ `gate_operating_points.csv`). FPR ở đây là tỷ lệ luồng benign bị cổng chuyển nhầm sang bộ phân loại, không phải FPR của toàn hệ thống.

Quantile (validation)	Ngưỡng	Recall (Attack)	FPR	Tỷ lệ bỏ qua tại cổng (%)
0,90	0,0023	0,7589	0,1003	80,08
0,95	0,0052	0,7552	0,0506	84,36
0,99	0,0192	0,7420	0,0101	88,00
0,995	0,0317	0,7229	0,0053	88,70

4.3.3 Lựa chọn mô hình cho thiết bị biên

Triển khai trên thiết bị biên đặt ra các ràng buộc về tài nguyên: RAM 8 GB chia sẻ giữa CPU và GPU (trong đó PySpark JVM chiếm một phần đáng kể), CPU ARM, cùng ràng buộc về nhiệt độ/bảng thông tại biên. Cũng cần nhấn mạnh rằng việc chọn Apache Spark trên Jetson 8 GB phải chịu chi phí JVM/Spark đáng kể; Spark được dùng ở đây nhằm *nhất*

quán với pha huấn luyện phân tán, chứ không nhằm tối ưu độ trễ suy luận trên một nút; chi phí đánh đổi này được định lượng qua so sánh với scikit-learn và ONNX Runtime ở mục 4.3.6, còn tăng tốc bằng TensorRT là hướng nghiên cứu tương lai. Ba mô hình đại diện cho ba thử nghiệm khác nhau được chọn để đánh giá:

1. **Decision Tree** – Mô hình cây đơn lẻ, đại diện cho cách tiếp cận đơn giản nhất.
2. **GBT (Gradient-Boosted Trees)** – Đại diện cho phương pháp Boosting, sử dụng nhiều cây yếu kết hợp tuần tự.
3. **Random Forest** – Đại diện cho phương pháp Bagging, sử dụng ensemble 200 cây song song.

Cả ba mô hình đều được huấn luyện trên tập SHAP Top-30 đặc trưng và triển khai dưới dạng PySpark PipelineModel.

Hình 4.6 minh họa giao diện terminal thực tế khi pipeline IDS đang xử lý dữ liệu trên Jetson Orin Nano Super, bao gồm thông tin về thông lượng, độ trễ và số lượng tấn công phát hiện được. Hai dashboard Grafana (Hình 4.7 và Hình 4.9) cho phép giám sát hiệu năng hệ thống và chi tiết các cuộc tấn công theo thời gian thực. Hình 4.10 ghi lại thời điểm tải đỉnh, cho thấy rõ sự phân bố tải lệch đặc trưng của chế độ tách pipeline: nút cổng chỉ sử dụng khoảng 5 % CPU trong khi nút phân loại PySpark đạt tới 91 %, xác nhận bộ phân loại là thành phần chiếm tài nguyên chủ đạo và là căn cứ cho các phân tích năng lượng–thông lượng ở phần sau. Khi phát hiện tấn công, hệ thống tự động gửi cảnh báo qua nhiều kênh: email thông qua Mailtrap SMTP (Hình 4.8) và tin nhắn Slack (Hình 4.11).

4.3.4 Kết quả huấn luyện các mô hình ứng viên trên cụm Spark

Bảng 4.13 trình bày kết quả huấn luyện và đánh giá ba mô hình ứng viên trên cụm Spark (mô tả ở mục Môi trường thực nghiệm) trước khi triển khai xuống thiết bị biên.

Bảng 4.13. Kết quả huấn luyện các mô hình ứng viên cho triển khai biên (SHAP Top-30)

Mô hình	F1-Score	Accuracy	Thời gian huấn luyện (s)	Kích thước (MB)
Decision Tree	0,9814	0,9943	24,4	0,046
GBT	0,9753	0,9925	341,8	0,865
Random Forest	0,9826	0,9947	687,3	4,427

Về tiêu chí *nhận biết ràng buộc biên* (edge-aware), ba mô hình ứng viên đánh đổi khác nhau: Decision Tree có kích thước nhỏ nhất (0,046 MB, nhỏ hơn Random Forest khoảng

CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN

```
~/Thesis_IDS/jetson -- ssh bvdung@192.168.1.205 ... ~/Desktop/Thesis_IDS/jetson -- -zsh - caffeineate ... ~/Desktop/Thesis_IDS/jetson -- -zsh ... ~/Thesis_IDS/jetson -- ssh bvdung@192.168.1.150 +
[MONITOR:jetson-nano-1] CPU: 16.9% | MEM: 12.9% (729.4MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.0 C

[MONITOR:jetson-nano-1] CPU: 17.8% | MEM: 12.9% (729.9MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.1 C

[MONITOR:jetson-nano-1] CPU: 16.8% | MEM: 12.9% (729.9MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.2 C

[MONITOR:jetson-nano-1] CPU: 16.8% | MEM: 12.9% (729.9MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.4 C

[MONITOR:jetson-nano-1] CPU: 17.8% | MEM: 12.9% (729.9MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.7 C

[MONITOR:jetson-nano-1] CPU: 17.8% | MEM: 12.9% (729.9MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.5 C

[MONITOR:jetson-nano-1] CPU: 17.3% | MEM: 12.9% (730.5MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.8 C

[MONITOR:jetson-nano-1] CPU: 16.9% | MEM: 12.9% (731.2MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.6 C

[MONITOR:jetson-nano-1] CPU: 17.8% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.0 C

[MONITOR:jetson-nano-1] CPU: 16.7% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 53.6 C

[MONITOR:jetson-nano-1] CPU: 17.8% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.0 C

[MONITOR:jetson-nano-1] CPU: 17.3% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.0 C

[MONITOR:jetson-nano-1] CPU: 16.8% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.0 C

[MONITOR:jetson-nano-1] CPU: 17.2% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.1 C

[MONITOR:jetson-nano-1] CPU: 17.8% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.2 C

[MONITOR:jetson-nano-1] CPU: 17.3% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.0 C

[MONITOR:jetson-nano-1] CPU: 16.7% | MEM: 12.9% (731.3MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.0 C

[MONITOR:jetson-nano-1] CPU: 17.8% | MEM: 12.9% (731.2MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.1 C

[MONITOR:jetson-nano-1] CPU: 17.6% | MEM: 12.9% (731.2MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.3 C

[MONITOR:jetson-nano-1] CPU: 17.3% | MEM: 12.9% (731.1MB) | Throughput: 0.0 rps | Latency: 0ms | Attacks: 0
Temp: 54.2 C
```

(a) Jetson #1 (jetson-nano-1): cổng phát hiện bất thường, giám sát tài nguyên theo node

```
thai Nguyen -- bvdung@ubuntu: ~/Thesis_IDS/jetson -- ssh bvdung@192.168.1.205 -- 208x63
~/Thesis_IDS/jetson -- ssh bvdung@192.168.1.205 ... ~/Desktop/Thesis_IDS/jetson -- -zsh - caffeineate ... ~/Desktop/Thesis_IDS/jetson -- -zsh ... ~/Thesis_IDS/jetson -- ssh bvdung@192.168.1.150 ... +
[MONITOR:jetson-nano-2] CPU: 72.4% | MEM: 58.6% (4196.0MB) | Throughput: 6.88 rps | Latency: 3.833ms | Attacks: 32
Temp: 56.7 C
[Stage 984:]> (0 + 0) / 63
[MONITOR:jetson-nano-2] CPU: 17.6% | MEM: 49.6% (3512.9MB) | Throughput: 6.87 rps | Latency: 3.968ms | Attacks: 32
Temp: 56.2 C
[MONITOR:jetson-nano-2] CPU: 17.3% | MEM: 57.5% (4111.0MB) | Throughput: 6.84 rps | Latency: 4.472ms | Attacks: 32
Temp: 56.1 C
[Stage 1089:]> (0 + 63) / 63
[MONITOR:jetson-nano-2] CPU: 97.7% | MEM: 54.7% (3980.9MB) | Throughput: 3.83 rps | Latency: 4.514ms | Attacks: 16
Temp: 57.3 C
[MONITOR:jetson-nano-2] CPU: 21.4% | MEM: 46.7% (3298.0MB) | Throughput: 6.88 rps | Latency: 4.334ms | Attacks: 32
Temp: 56.2 C
[MONITOR:jetson-nano-2] CPU: 67.8% | MEM: 55.8% (3927.0MB) | Throughput: 6.86 rps | Latency: 3.638ms | Attacks: 32
Temp: 56.7 C
[Stage 1847:]> (0 + 63) / 63
[MONITOR:jetson-nano-2] CPU: 68.2% | MEM: 58.9% (3611.9MB) | Throughput: 6.88 rps | Latency: 4.156ms | Attacks: 32
Temp: 57.0 C
[MONITOR:jetson-nano-2] CPU: 16.9% | MEM: 59.6% (4274.3MB) | Throughput: 6.88 rps | Latency: 4.483ms | Attacks: 32
Temp: 55.9 C
[jetson-nano-2 2.408] Batch: 187ms | Attacks: 2373/2480 | Avg: 9.7ms
[MONITOR:jetson-nano-2] CPU: 85.5% | MEM: 49.1% (3474.0MB) | Throughput: 6.88 rps | Latency: 3.256ms | Attacks: 32
Temp: 56.5 C
[Stage 1688:]> (0 + 80) / 63
[MONITOR:jetson-nano-2] CPU: 66.6% | MEM: 51.7% (3673.5MB) | Throughput: 3.84 rps | Latency: 3.737ms | Attacks: 16
Temp: 56.9 C
[MONITOR:jetson-nano-2] CPU: 28.7% | MEM: 56.8% (4082.1MB) | Throughput: 6.88 rps | Latency: 4.399ms | Attacks: 32
Temp: 56.1 C
[MONITOR:jetson-nano-2] CPU: 74.4% | MEM: 54.8% (3968.1MB) | Throughput: 6.88 rps | Latency: 3.923ms | Attacks: 32
Temp: 56.9 C
[MONITOR:jetson-nano-2] CPU: 34.2% | MEM: 56.8% (4063.0MB) | Throughput: 5.88 rps | Latency: 4.574ms | Attacks: 31
Temp: 56.8 C
[MONITOR:jetson-nano-2] CPU: 16.7% | MEM: 55.8% (3924.2MB) | Throughput: 6.88 rps | Latency: 4.42ms | Attacks: 32
Temp: 56.1 C
[Stage 1149:]> (1 + 53) / 63
[MONITOR:jetson-nano-2] CPU: 98.8% | MEM: 59.1% (4234.9MB) | Throughput: 3.84 rps | Latency: 3.157ms | Attacks: 16
Temp: 57.2 C
[MONITOR:jetson-nano-2] CPU: 16.8% | MEM: 52.5% (3733.5MB) | Throughput: 6.88 rps | Latency: 3.994ms | Attacks: 32
Temp: 56.9 C
[MONITOR:jetson-nano-2] CPU: 18.8% | MEM: 57.8% (4077.7MB) | Throughput: 5.98 rps | Latency: 4.577ms | Attacks: 31
Temp: 56.1 C
[Stage 1187:]> (0 + 63) / 63
[MONITOR:jetson-nano-2] CPU: 45.8% | MEM: 60.6% (4351.7MB) | Throughput: 5.98 rps | Latency: 4.121ms | Attacks: 31
Temp: 56.7 C
[MONITOR:jetson-nano-2] CPU: 16.7% | MEM: 52.7% (3758.0MB) | Throughput: 5.99 rps | Latency: 3.338ms | Attacks: 31
Temp: 55.9 C
[Stage 1212:]> (3 + 3) / 63
```

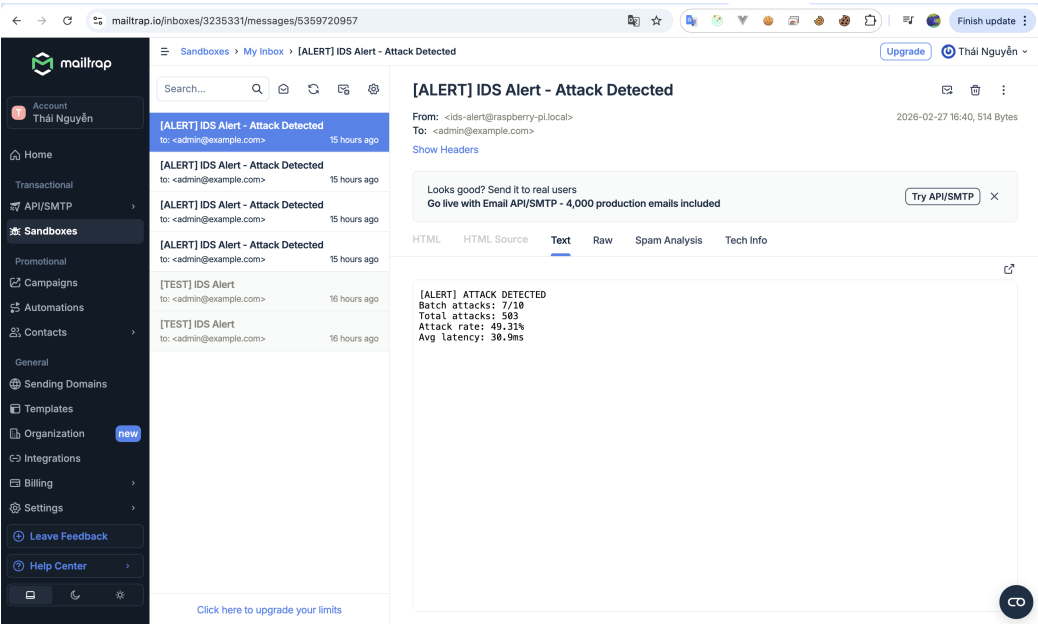
(b) Jetson #2 (jetson-nano-2): bộ phân loại PySpark, thông lượng ~6 luồng/giây, độ trễ suy luận ~4 ms, đếm tấn công theo lô

Hình 4.6. Giao diện terminal khi pipeline IDS hoạt động trên hai node Jetson Orin Nano Super: (a) node cổng lọc bất thường và (b) node phân loại PySpark

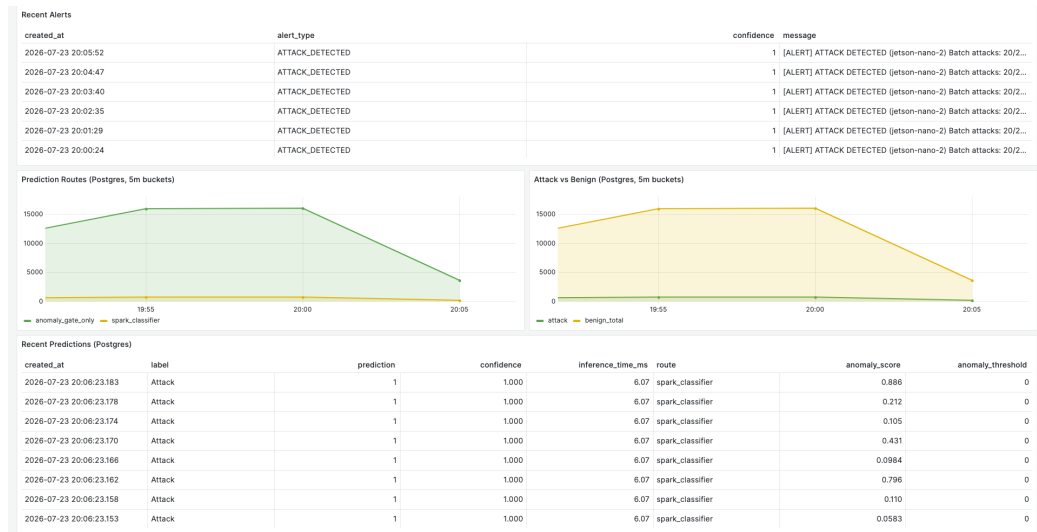
CHƯƠNG 4. THỰC NGHIỆM, ĐÁNH GIÁ VÀ THẢO LUẬN



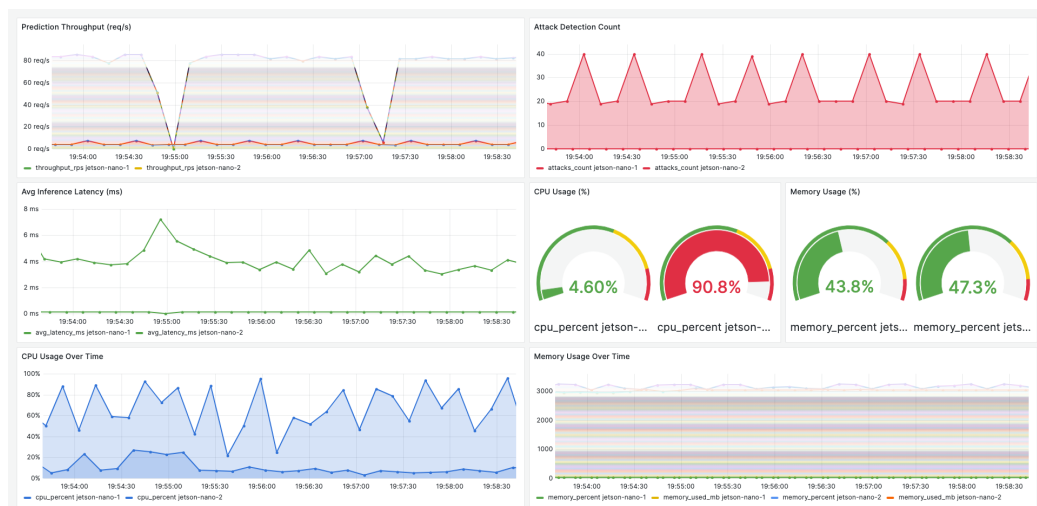
Hình 4.7. Dashboard Grafana giám sát hiệu năng IDS theo thời gian thực



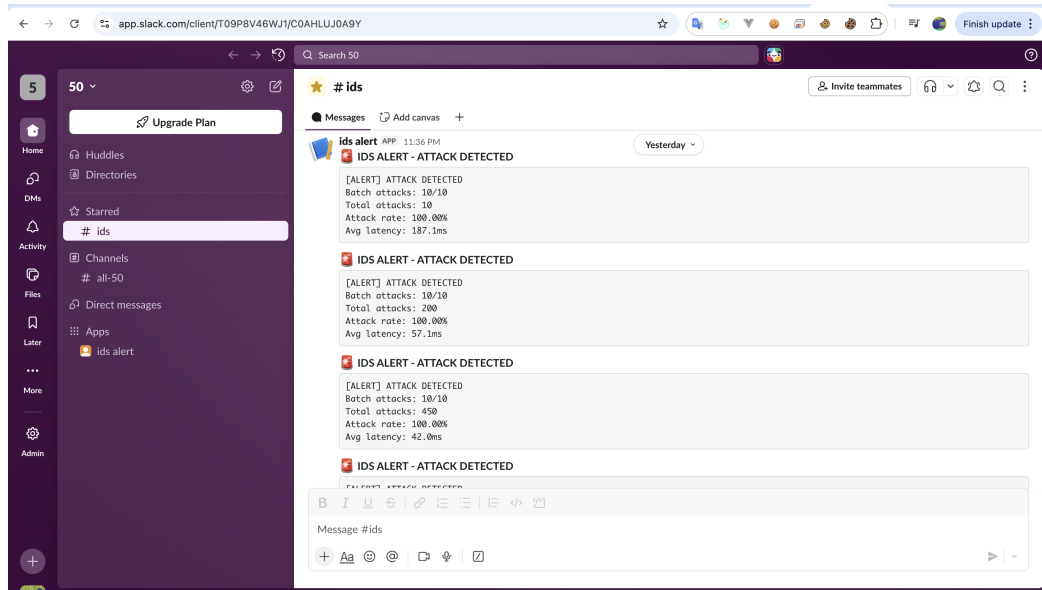
Hình 4.8. Email cảnh báo tấn công được gửi qua Mailtrap trong môi trường kiểm thử (ảnh chụp từ giai đoạn thử nghiệm sớm của hệ thống; mô-đun cảnh báo và định dạng thông điệp giữ nguyên khi triển khai trên cụm Jetson)



Hình 4.9. Dashboard Grafana hiển thị chi tiết các cuộc tấn công được phát hiện và phân loại theo thời gian



Hình 4.10. Dashboard Grafana tại thời điểm tải đỉnh của chế độ tách pipeline: nút cổng (jetson-nano-1) chỉ sử dụng khoảng 5 % CPU trong khi nút phân loại PySpark (jetson-nano-2) đạt tới 91 %



Hình 4.11. Thông báo cảnh báo tấn công được gửi qua Slack Incoming Webhook (ảnh chụp từ giai đoạn thử nghiệm sớm; định dạng thông điệp giữ nguyên khi triển khai trên cụm Jetson)

96 lần) và huấn luyện nhanh hơn khoảng 28 lần, trong khi Random Forest đạt F1 cao nhất. Riêng F1/Accuracy chưa đủ để kết luận: quyết định cuối cùng dựa trên số đo vận hành thực tế trên thiết bị (thông lượng, độ trễ, CPU, RAM) ở mức tiếp theo. (Các mô hình ứng viên biên được huấn luyện trên mẫu con phân tầng khi xuất để vừa bộ nhớ một nút, nên F1 thấp hơn chút ít so với kết quả huấn luyện phân tán đầy đủ ở Bảng 4.3; độ chính xác chính thức lấy từ pha huấn luyện đầy đủ.)

4.3.5 Kết quả đo hiệu năng (benchmark) trên Jetson Orin Nano Super

Bảng 4.14 trình bày kết quả đo hiệu năng (benchmark) thực tế trên Jetson Orin Nano Super. Mỗi cấu hình được đo lặp lại 5 lần; trước mỗi cửa sổ đo có giai đoạn khởi động (warm-up) bằng chính lưu lượng thật để làm nóng JVM/JIT, và dữ liệu warm-up bị loại khỏi cửa sổ đo; kết quả báo cáo là trung bình $\pm$ độ lệch chuẩn qua các lần chạy. Hai loại độ trễ được ghi nhận: (i) *độ trễ suy luận* (tiền xử lý + dự đoán) trên từng nút, và (ii) *độ trễ đầu cuối* (từ thời điểm producer gửi đến khi có phán quyết) tính từ nhãn thời gian của producer; giá trị tuyệt đối của loại (ii) chỉ có ý nghĩa khi các máy đồng bộ đồng hồ NTP. Các phân vị độ trễ (p50/p95/p99) được tính ở mức từng lần chạy từ *log thô theo từng luồng* trong đúng cửa sổ tải; với cụm nhiều nút, p95 báo cáo là p95 của nút xấu nhất (không lấy phân vị của các giá trị trung bình theo nút).

Bảng 4.14. So sánh hiệu năng triển khai trên Jetson Orin Nano Super

Mô hình	Thông lượng (mẫu/giây)	Độ trễ suy luận (ms) p50 / p95 / p99	CPU (%)	RAM (%)	Kích thước (MB)
Decision Tree	22,0	441 / 537 / –	46,4	21,9	0,046
GBT	24,3	414 / 439 / –	40,9	24,0	0,865
Random Forest	26,8	378 / 397 / –	35,2	27,5	4,427

4.3.6 So sánh engine suy luận: chi phí của việc đồng nhất stack

Luận văn triển khai mô hình dưới dạng `PipelineModel` của Spark để đồng nhất với pha huấn luyện phân tán: cùng một stack vừa huấn luyện trên cụm vừa suy luận trên thiết bị. Để định lượng chi phí đánh đổi của sự đồng nhất này, luận văn đo lại cùng một mô hình Random Forest (cùng siêu tham số) trên cùng tập đặc trưng SHAP đã chọn để triển khai nhưng qua các engine nhẹ hơn ở chế độ một nút: scikit-learn (CPU, không JVM) và ONNX Runtime (CPU execution provider; Bảng 4.15, sinh từ `benchmark_engines.py`). Vì cùng thuật toán và đặc trưng nên độ chính xác tương đương; phép so sánh tập trung vào chi phí suy luận. (Cả ba dòng của Bảng 4.15 được đo lại trong cùng một phiên trên cùng trạng thái bo mạch để so sánh công bằng, nên dòng PySpark chênh lệch nhỏ so với Bảng 4.14 đo ở phiên trước: 22,6 so với 26,8 luồng/giây, do khác phiên đo và xung CPU.) Kết quả đo (Bảng 4.15) cho thấy stack JVM/Spark chịu chi phí suy luận cao hơn đáng kể so với scikit-learn trên bo mạch ARM 8 GB: thông lượng thấp hơn khoảng 9 lần, độ trễ p95 cao hơn khoảng 11 lần, tỷ lệ RAM gần gấp đôi và năng lượng mỗi suy luận cao hơn khoảng 25 lần. Biên dịch cùng mô hình rừng ngẫu nhiên sang ONNX Runtime còn nới rộng khoảng cách này đáng kể: ≈ 7900 luồng/giây với p95 1,1 ms, tức gấp ≈ 348 lần thông lượng của Spark đã triển khai, ở mức năng lượng chỉ $\approx 2\%$ so với scikit-learn; việc nêu rõ điều này là cơ sở cho hướng phát triển xuất mô hình sang ONNX/TensorRT để suy luận trong khi vẫn giữ Spark cho huấn luyện (việc tích hợp phiên ONNX vào pipeline streaming và khai thác GPU qua TensorRT thuộc hướng phát triển).

Bảng 4.15. Chi phí suy luận một nút trên cùng mô hình/đặc trưng (Spark vs sklearn vs ONNX), đo trên nút phân loại trong cùng điều kiện bo mạch (batch 10; năng lượng là phần hoạt động, đã trừ idle)

Engine	Thông lượng (luồng/giây)	Độ trễ p95 (ms)	RAM (%)	Năng lượng /suy luận (mJ)
PySpark (triển khai)	22,6	532	24,3	71,6
scikit-learn	204,4	49,8	13,3	2,91
ONNX Runtime	7870,8	1,1	15,7	0,06

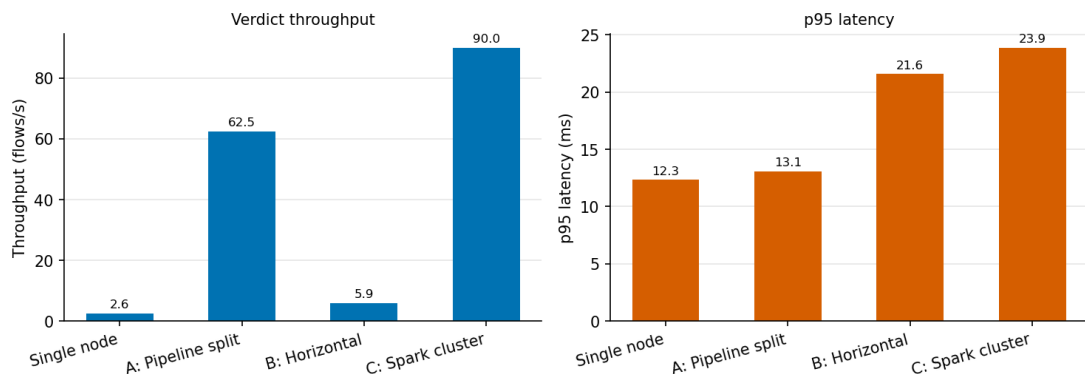
4.3.7 Kết quả ba chế độ triển khai phân tán

Ngoài việc so sánh các mô hình trên một nút, luận văn đánh giá ba chế độ triển khai phân tán trên cụm hai Jetson Orin Nano Super (Bảng 4.16). Chỉ số chính gồm thông lượng, độ trễ p95 (ms), tỷ lệ bỏ qua tại cổng (gate-skip: % lưu lượng bình thường được cổng lọc khỏi suy luận Spark) và F1 trên lớp tấn công. *Thông lượng* được định nghĩa là số *phán quyết cuối cùng* ghi vào PostgreSQL trên giây (một luồng nhận phán quyết hoặc tại cổng (nếu bị bỏ qua là benign) hoặc tại bộ phân loại), không cộng dồn đầu ra của hai tầng để tránh đếm trùng luồng được chuyển tiếp ở chế độ pipeline split.

Bảng 4.16. So sánh ba chế độ triển khai phân tán trên cụm 2 Jetson Orin Nano Super

Chế độ	Thông lượng (luồng/giây)	Độ trễ p95 (ms)	Bỏ qua tại cổng (%)	Năng lượng /suy luận (mJ)	F1 (Attack)
Một nút (single)	2,60 ± 0,41	12,3 ± 3,0	(nội tuyến)	2490	†
A: Pipeline split	62,5 ± 0,6	13,1 ± 2,6	95,8	178	†
B: Horizontal scaling	5,9 ± 1,4	21,6 ± 19,2	(nội tuyến)	2250	†
C: Spark cluster	90,0 ± 25,6	23,9 ± 17,8	97,9	119	†

Phát lại CICIDS2017 ở 100 luồng/giây; trung bình ± độ lệch chuẩn trên ≥3 lần lặp; p95 tính từ log thô từng luồng (nút xấu nhất). “(nội tuyến)”: ở vai trò `full`, cổng chạy trong cùng tiến trình và các luồng bị bỏ qua vẫn được ghi phán quyết nên không có tỷ lệ bỏ qua riêng. †: chất lượng phát hiện không phụ thuộc chế độ (các chế độ dùng cùng một mô hình xuất ra); F1 lớp tấn công ngoại tuyến xem Bảng 4.6.



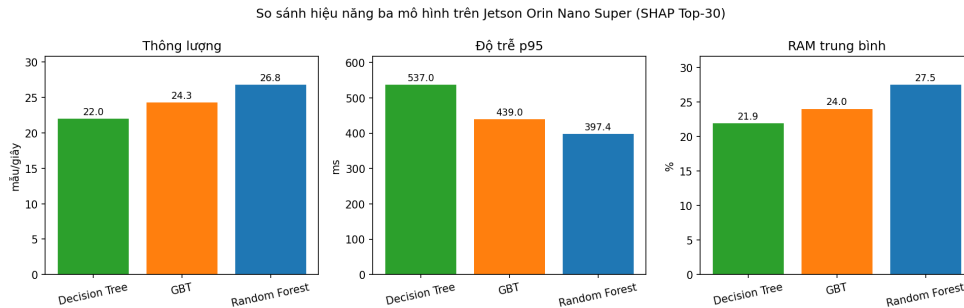
Hình 4.12. Thông lượng và độ trễ p95 của bốn cấu hình (một nút cơ sở + ba chế độ phân tán) trên cụm 2 Jetson

Kết quả thực đo (Hình 4.12) cho thấy: chế độ *pipeline split* tách cổng khỏi bộ phân loại Spark giúp thông lượng phán quyết tăng từ 2,6 lên 62,5 luồng/giây (gấp ≈24 lần) với p95 gần như không đổi, nhờ cổng lọc 95,8% lưu lượng bình thường và không còn bị Spark chặn vòng xử lý. *Mở rộng theo chiều ngang* chỉ tăng xấp xỉ gấp đôi một nút (5,9 so với 2,6) vì mỗi nút vẫn chạy trọn bộ phân loại Spark trong tiến trình. *Spark cluster* (cổng + bộ phân loại nổi cụm Spark standalone) đạt thông lượng trung bình cao nhất (90,0) nhưng

dao động lớn giữa các lần chạy (± 26) và phụ thuộc thêm nút chủ (master) trên máy điều phối, nên pipeline split vẫn là khuyến nghị mặc định. *Năng lượng mỗi suy luận* đo bằng `tegrastats` (nền idle 30 giây trước mỗi cửa sổ tải; chế độ hai nút cộng công suất cả hai bo mạch): ở mức tải 100 luồng/giây, phần công suất *hoạt động* (đã trừ idle) nằm dưới ngưỡng nhiễu lấy mẫu ($< 0,15$ W mọi nút), do đó bảng báo cáo năng lượng *toàn bo mạch* trên mỗi phán quyết, tức thước đo mức dàn trải công suất bo mạch theo thông lượng ở cấp triển khai (không phải chi phí riêng của mô hình): các chế độ dùng cổng (A, C) dàn trải công suất hai bo mạch trên số phán quyết lớn gấp 24–35 lần, giảm năng lượng mỗi phán quyết từ $\approx 2,5$ J xuống 0,12–0,18 J.

4.3.8 Phân tích và lựa chọn mô hình

Hình 4.13 minh họa trực quan sự khác biệt giữa ba mô hình về thông lượng, độ trễ và mức sử dụng RAM trên Jetson Orin Nano Super.



Hình 4.13. So sánh hiệu năng ba mô hình trên Jetson Orin Nano Super

Kết quả benchmark cho thấy sự đánh đổi rõ ràng giữa các chỉ số:

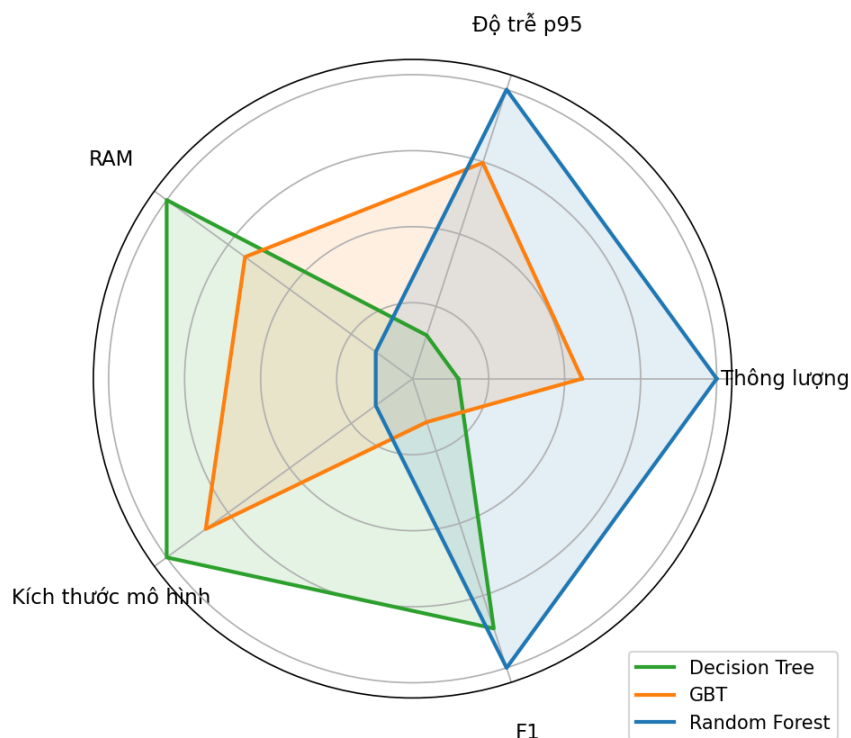
- **Random Forest** thắng ở cả ba chỉ số vận hành quan trọng nhất với phát hiện xâm nhập thời gian thực: thông lượng cao nhất (26,8 mẫu/giây), độ trễ p95 thấp nhất (397 ms) và CPU thấp nhất (35,2%), nhờ Spark song song hóa tốt trên nhiều cây. Cái giá phải trả là RAM cao nhất (27,5%) và mô hình lớn nhất ($\approx 4,4$ MB).
- **GBT** có hiệu năng trung bình ở tất cả các chỉ số, đóng vai trò cầu nối giữa Decision Tree và Random Forest.
- **Decision Tree** nhẹ nhất về bộ nhớ (RAM 21,9%, mô hình $\approx 0,05$ MB) nhưng chậm nhất: thông lượng 22,0 mẫu/giây và độ trễ p95 537 ms, cao hơn Random Forest khoảng 35%.

Hình 4.14 thể hiện sự đánh đổi đa chiều giữa các tiêu chí lựa chọn mô hình: Decision Tree chiếm ưu thế về bộ nhớ, còn Random Forest chiếm ưu thế về tốc độ xử lý và tải CPU.

Random Forest được chọn làm mô hình triển khai chính trên thiết bị biên dựa trên ba tiêu chí:

1. **Năng lực xử lý thời gian thực:** thông lượng cao nhất (26,8 mẫu/giây) và độ trễ p95 thấp nhất (397 ms). Với IDS, độ trễ phán quyết là ràng buộc trực tiếp — một mô hình nhỏ nhưng chậm hơn 35% về p95 sẽ kéo dài cửa sổ mà tấn công chưa bị chặn.
2. **Chất lượng phát hiện:** cao nhất trong ba mô hình ứng viên ở *cả hai* phép đo — F1 0,9955 trên đánh giá ngoại tuyến đầy đủ (Bảng 4.3) và 0,9826 ở bản xuất huấn luyện trên mẫu con để vừa bộ nhớ một nút (Bảng 4.13).

Đánh đổi giữa các mô hình trên Jetson Orin Nano Super
(chuẩn hoá min-max, càng ra ngoài càng tốt)



Hình 4.14. Biểu đồ radar thể hiện sự đánh đổi (trade-off) giữa các mô hình trên thiết bị biên

- Tương thích nền tảng:** Random Forest là estimator gốc của Spark ML nên `PipelineModel` xuất ra nạp được trực tiếp trên board ARM64, không cần các thư viện native mà tích hợp XGB của Spark đòi hỏi.

Đánh đổi được chấp nhận là mức tiêu thụ bộ nhớ: Random Forest dùng 27,5% RAM so với 21,9% của Decision Tree và mô hình nặng $\approx 4,4$ MB so với 0,05 MB. Trên board 8 GB, khoảng chênh này còn dư địa lớn, nên bộ nhớ không phải là ràng buộc quyết định; Decision Tree vẫn là phương án dự phòng hợp lý nếu triển khai xuống các thiết bị IoT eo hẹp bộ nhớ hơn.

4.4 Thảo luận và So sánh tổng hợp

Kết quả thực nghiệm cho thấy lựa chọn đặc trưng dựa trên SHAP là hướng tiếp cận phù hợp cho hệ thống IDS trên bộ dữ liệu CICIDS2017: dưới cấu hình đánh giá loại trừ rò rỉ dữ liệu, SHAP Top-30 giữ được hiệu năng gần như nguyên vẹn trong khi cắt khoảng 60% số đặc trưng. Ngược lại, học tổ hợp *không* mang lại lợi thế trên chính cấu hình này: Hybrid Bagging đạt F1 0,9966 và Soft Voting đạt 0,9967, đều *thấp hơn* XGB đơn lẻ (0,9970). Học

tổ hợp chỉ vượt mô hình đơn ở những cấu hình mà mô hình đơn yếu hơn (RF Top-30: 0,9938 so với 0,9922; PCA $k=40$: 0,9940 so với 0,9939), nghĩa là nó bù cho bộ đặc trưng kém chứ không tạo thêm năng lực phân loại. Chi phí đi kèm lại lớn: trên SHAP Top-30, Hybrid Bagging cần 4909 giây huấn luyện (so với 1226 giây của XGB), 46,6 giây dự đoán (so với 7,1 giây) và mô hình nặng 7,15 MB — lớn nhất trong tất cả các cấu hình đã thử. Vì vậy cấu hình được chọn để triển khai lên thiết bị biên là một *mô hình đơn* trên tập đặc trưng SHAP Top-30, không phải mô hình học tổ hợp.

Đặc biệt, khi triển khai trên thiết bị biên Jetson Orin Nano Super, Random Forest với SHAP Top-30 đặc trưng đạt F1-Score 0,9955 ngoại tuyến (đã loại trừ rò rỉ), thông lượng 26,8 mẫu/giây, độ trễ p95 397 ms và sử dụng 27,5% RAM. Các số liệu này cho phép đánh giá tính khả thi của việc triển khai hệ thống IDS dựa trên học máy trên các thiết bị biên có tài nguyên giới hạn, mở ra hướng ứng dụng thực tế cho giám sát an ninh mạng phân tán.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Trong chương này, luận văn sẽ trình bày những kết quả mà luận văn đã đạt được so với mục tiêu được đặt ra từ ban đầu khi bắt đầu thực hiện. Bên cạnh đó, luận văn cũng trình bày những hạn chế và đưa ra hướng phát triển trong tương lai của nghiên cứu.

5.1 Những đóng góp chính của luận văn

Sau khi thực hiện luận văn với đề tài “*Phát hiện xâm nhập (IDS) dựa trên Apache Spark cho bảo mật mạng IoT*”, luận văn đã đạt được một số kết quả chính tương ứng với các mục tiêu đề ra ban đầu:

- **Xây dựng nền tảng xử lý dữ liệu lớn:** Đề xuất và triển khai thành công hệ thống IDS trên nền tảng Apache Spark. Giải pháp này tận dụng khả năng tính toán song song để xử lý các luồng dữ liệu mạng từ môi trường IoT, hướng tới khả năng mở rộng trong việc giám sát bảo mật; khả năng xử lý phân tán được kiểm chứng ở mức minh chứng khái niệm (proof-of-concept) trên cụm thiết bị biên.
- **Đánh giá học tổ hợp bằng bằng chứng thay vì kỳ vọng:** Xây dựng mô hình Hybrid Bagging Ensemble kết hợp Top-3 thuật toán có F1 cao nhất trên tập kiểm định (xác định ở chương thực nghiệm, không cố định trước), rồi so sánh trực tiếp với các mô hình đơn lẻ kèm kiểm định ý nghĩa thống kê. Dưới quy trình đánh giá loại trừ rò rỉ dữ liệu, học tổ hợp *không* mang lại lợi thế trên cấu hình tốt nhất: Hybrid Bagging đạt F1 0,9966 so với 0,9970 của XGB đơn lẻ, trong khi chi phí huấn luyện và dung lượng mô hình cao hơn nhiều lần. Đây là một kết quả âm tính nhưng có giá trị: trên bài toán nhị phân đã gần bão hòa, việc ghép mô hình không bù lại được chi phí, và mô hình đơn mới là lựa chọn phù hợp cho thiết bị biên.
- **Đánh giá trung thực, loại trừ rò rỉ dữ liệu:** Nhận diện và xử lý các nguồn rò rỉ thường gặp trên CICIDS2017 (đặc trưng `destination_port` rò rỉ nhân và trùng lặp luồng ở mức đặc trưng), đồng thời định lượng mức thổi phồng hiệu năng do rò rỉ — giúp kết quả phản ánh đúng năng lực thực của mô hình.
- **Tối ưu hóa và Giải thích mô hình:** Ứng dụng kỹ thuật SHAP để lựa chọn các đặc trưng quan trọng nhất, giúp giảm số lượng đặc trưng (khoảng 60%) mà vẫn giữ nguyên hiệu năng phân loại: dưới cấu hình loại trừ rò rỉ dữ liệu, Random Forest với

SHAP Top-30 đạt F1 0,9955, tương đương cấu hình dùng toàn bộ đặc trưng (0,9949). Đồng thời, SHAP cung cấp khả năng giải thích chi tiết về quyết định của mô hình, tăng cường độ tin cậy và khả năng ứng dụng thực tế.

5.2 Hạn chế của đề tài

Mặc dù đã đạt được các kết quả nêu trên, luận văn vẫn còn một số hạn chế cần được khắc phục trong các giai đoạn tiếp theo:

- **Phạm vi phân loại:** Hệ thống triển khai chính dựa trên bài toán phân loại nhị phân (Tấn công so với Bình thường); luận văn đã bổ sung đánh giá đa lớp theo từng loại tấn công (Mục 4.2.9) để phân tích điểm yếu ở các lớp hiếm, nhưng việc xây dựng một bộ phân loại đa lớp hoàn chỉnh cho triển khai thời gian thực vẫn là hướng phát triển tiếp theo.
- **Phạm vi bộ dữ liệu:** Luận văn đã thiết lập quy trình đánh giá tổng quát hóa liên bộ dữ liệu (huấn luyện trên một bộ, kiểm tra trên bộ khác, Mục 4.2.8.1) trên cặp CICIDS2017 và CSE-CIC-IDS2018; việc mở rộng sang nhiều bộ mới hơn (CIC-IoT, RoEduNet) để củng cố kết luận về tổng quát hóa vẫn là hướng tiếp theo.
- **Quy mô dữ liệu và phần cứng biên:** Khả năng xử lý phân tán mới được kiểm chứng ở mức minh chứng khái niệm (proof-of-concept) trên cụm hai thiết bị Jetson (8 GB) với khối lượng dữ liệu cỡ vài triệu luồng của CICIDS2017, chứ chưa phải ở quy mô dữ liệu lớn thực thụ (terabyte trở lên) hay trên cụm tính toán nhiều nút mạnh; việc đánh giá khả năng mở rộng ở quy mô lớn hơn vẫn là hướng tiếp theo.
- **Chưa đánh giá trước lưu lượng đối kháng:** Hệ thống chưa được kiểm thử trước các tấn công đối kháng/né tránh (adversarial/evasion) và giả định phân bố tấn công xấp xỉ CICIDS2017; do đó độ bền vững trước kẻ tấn công cố ý đánh lừa mô hình vẫn là câu hỏi mở.

5.3 Hướng phát triển trong tương lai

Dựa trên những kết quả đã đạt được và nhận diện các hạn chế hiện có, luận văn đề xuất một số hướng phát triển trong tương lai, mỗi hướng tương ứng với một hạn chế đã nêu:

- **Cải thiện độ nhạy cho các lớp tấn công hiếm:** Phân tích hướng nhằm lẫn giữa các lớp (Mục 4.2.9) cho thấy các lớp hiếm suy giảm theo hai kiểu: nhầm loại trong cùng

họ (XSS bị nhận thành Brute Force) và lọt hẳn thành Benign (SQL Injection, 54% Bot). Bốn hướng khắc phục tương ứng: (i) *xử lý mất cân bằng có chủ đích* ở pha huấn luyện đa lớp: undersampling lớp Benign, oversampling/SMOTE [70] cho lớp hiếm (áp dụng *sau* khi chia tập để tránh rò rỉ), hoặc gộp các biến thể cùng họ (gộp ba biến thể Web Attack thành một lớp) để đổi độ mịn định danh lấy recall; (ii) *điều chỉnh thuật toán*: trọng số lớp cho bài đa lớp, focal loss [71] với XGBoost, và hạ ngưỡng quyết định theo lớp (chấp nhận thêm false positive ở lớp hiếm vì với IDS, báo nhầm rẻ hơn bỏ sót); (iii) *kiến trúc phân tầng*: tận dụng chính thiết kế cổng hai tầng của đề tài: tầng nhị phân trên biên giữ recall tổng, còn bộ định danh loại huấn luyện *chỉ trên các luồng tấn công* (không phải cạnh tranh với 380 nghìn mẫu Benign) và có thể huấn luyện lại ngoại tuyến với cân bằng lớp mà không đung hệ thống đang chạy; (iv) *đặc trưng chuỗi thời gian theo host* (chu kỳ kết nối lặp, entropy kích thước gói qua cửa sổ trượt), nhằm trị đúng bệnh của hành vi beaconing C&C khiến 54% Bot lọt lưới, vốn không thể hiện hình qua thống kê từng luồng độc lập.

- **Kiểm chứng trên bộ dữ liệu IoT chuyên biệt:** Đánh giá quy trình trên các bộ dữ liệu chứa lưu lượng IoT thực thụ (Bot-IoT, TON_IoT, CIC-IoT-2023) và mở rộng thí nghiệm liên bộ dữ liệu lên toàn bộ CSE-CIC-IDS2018, nhằm thu hẹp khoảng cách giữa phạm vi “IoT” của đề tài và dữ liệu CICIDS2017 hiện dùng.
- **Tối ưu suy luận biên bằng ONNX/TensorRT:** Tích hợp phiên suy luận ONNX vào pipeline streaming và tăng tốc bằng TensorRT trên GPU Jetson để giảm chi phí JVM/Spark ở pha suy luận (trong khi vẫn giữ Spark cho huấn luyện phân tán): kết quả đo ngoại tuyến ở Bảng 4.15 cho thấy cùng mô hình rừng ngẫu nhiên chạy qua ONNX Runtime đạt thông lượng gấp ≈ 348 lần bản PySpark đã triển khai, xác nhận dư địa rất lớn của hướng này; đồng thời đo độ trễ đầu cuối đầy đủ (từ Kafka đến phán quyết rồi ghi cơ sở dữ liệu).
- **Độ bền vững trước tấn công đối kháng:** Đánh giá và gia cố hệ thống trước lưu lượng né tránh (adversarial/evasion), đặc biệt là hai bề mặt tấn công mới do cổng bắt thường tạo ra: giả dạng lưu lượng bình thường để vượt cổng và cố ý tạo sai số tái tạo cao để dồn tải lên nút phân loại.

Phụ lục A. CÔNG TRÌNH KHOA HỌC ĐÃ CÔNG BỐ

Phần này liệt kê các bài báo khoa học liên quan đến đề tài luận văn (viết chung với Nguyễn Vũ Thái, Đỗ Trí Nhựt và TS. Nguyễn Văn Dũ), cả hai đều đã được chấp nhận đăng:

1. Thai Nguyen Vu, Dung Van Bui, Nhut Tri Do, Van Du Nguyen, “A Comparison of Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML with a Leakage-Aware Evaluation Pipeline,” National Conference on Fundamental and Applied Information Technology Research (FAIR), 2026.
2. Thai Nguyen Vu, Dung Van Bui, Nhut Tri Do, Van Du Nguyen, “A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark,” Symposium on Information and Communication Technology (SOICT), 2026.

TÀI LIỆU THAM KHẢO

- [1] Ansam Khraisat, Iqbal Gondal, Peter Vamplew, và Joarder Kamruzzaman. “Survey of intrusion detection systems: techniques, datasets and challenges”. In: *Cybersecurity 2.1* (2019), pp. 1–22.
- [2] Mohamed Amine Ferrag, Leandros Maglaras, Helge Janicke, Jianhua Jiang, và Lei Shu. “Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study”. In: *Journal of Information Security and Applications* 50 (2020), p. 102419.
- [3] R Vinayakumar, Mamoun Alazab, K P Soman, Prabaharan Poornachandran, Ameer Al-Nemrat, và Sitalakshmi Venkatraman. “Deep learning approach for intelligent intrusion detection system”. In: *IEEE Access* 7 (2019), pp. 41525–41550.
- [4] Amardeep Singh và Julian Jang-Jaccard. *Autoencoder-based Unsupervised Intrusion Detection using Multi-Scale Convolutional Recurrent Networks*. 2022. arXiv: [2204.03779](https://arxiv.org/abs/2204.03779) [cs.CR].
- [5] Pengfei Sun, Pengju Liu, Qi Li, Chenxi Liu, Xiangling Lu, Ruochen Hao, và Jinpeng Chen. “DL-IDS: Extracting features using CNN-LSTM hybrid network for intrusion detection system”. In: *Security and Communication Networks* 2020 (2020), p. 8890306. DOI: [10.1155/2020/8890306](https://doi.org/10.1155/2020/8890306).
- [6] N. Dash, S. Chakravarty, A. K. Rath, và các cộng sự. “An optimized LSTM-based deep learning model for anomaly network intrusion detection”. In: *Scientific Reports* 15 (2025), p. 1554. DOI: [10.1038/s41598-025-85248-z](https://doi.org/10.1038/s41598-025-85248-z).
- [7] Sukhvinder Singh Bamber, Aditya Vardhan Reddy Katkuri, Shubham Sharma, và Mohit Angurala. “A hybrid CNN-LSTM approach for intelligent cyber intrusion detection system”. In: *Computers & Security* 148 (2025), p. 104146. DOI: [10.1016/j.cose.2024.104146](https://doi.org/10.1016/j.cose.2024.104146).
- [8] Tasnimul Hasan, Abrar Hossain, Mufakir Qamar Ansari, và Talha Hussain Syed. *Enhanced Intrusion Detection in IIoT Networks: A Lightweight Approach with Autoencoder-Based Feature Learning*. arXiv:2501.15266. 2025.
- [9] P. R. Chithra Rani và K. Baalaji. “Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms”. In: *Discover Applied Sciences* 7 (2025), p. 928. DOI: [10.1007/s42452-025-06871-z](https://doi.org/10.1007/s42452-025-06871-z).

- [10] Kushboo Nasir, Sahar K. Badri, Daniyal M. Alghazzawi, Mohammed Yahya Alghamdi, Mona Alkhozae, Abeer Almakky, Rania M. Alhazmi, và Muhammad Zubair Asghar. “HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning”. In: *Scientific Reports* 16 (2026), p. 2313. DOI: [10.1038/s41598-025-32183-8](https://doi.org/10.1038/s41598-025-32183-8).
- [11] Jackson Diaz-Gorriñ, Candido Caballero-Gil, và Ljiljana Brankovic. “Enhancing Network Security with Generative AI on Jetson Orin Nano”. In: *Applied Sciences* 16.3 (2026), p. 1442. DOI: [10.3390/app16031442](https://doi.org/10.3390/app16031442).
- [12] Zhenxiong Zhang, Lei Zhang, Jialong Xu, Zhengze Chen, và Peng Wang. “An Edge-Deployable Lightweight Intrusion Detection System for Industrial Control”. In: *Electronics* 15.3 (2026), p. 644. DOI: [10.3390/electronics15030644](https://doi.org/10.3390/electronics15030644).
- [13] Laila Nassef, Mohammed Ibrahim Alghamdi, Slim Ben Chaabane, Qaisar Abbas, Wedad M. Alawad, Omar H. Albalawi, Osamah I. Alqaisi, và Bahjat Fakieh. “Lightweight and Energy-Aware Intrusion Detection for Industrial IoT Using TinyML and Edge AI”. In: *Scientific Reports* 16.1 (2026), p. 18524. DOI: [10.1038/s41598-026-50690-0](https://doi.org/10.1038/s41598-026-50690-0).
- [14] Kotyada Mohan Kiran Kumar, Mokshith Sreekar, Karthik Ullas, và M. Supriya. “Distributed Intrusion Detection System using Kafka and Spark Streaming”. In: *2025 International Conference on Visual Analytics and Data Visualization (ICVADV)*. IEEE, 2025, pp. 302–309. DOI: [10.1109/ICVADV63329.2025.10961691](https://doi.org/10.1109/ICVADV63329.2025.10961691).
- [15] İlhan Firat Kilincer, Fatih Ertam, và Abdulkadir Sengur. “Machine learning methods for cyber security intrusion detection: Datasets and comparative study”. In: *Computer Networks* 188 (2021), p. 107840. DOI: [10.1016/j.comnet.2021.107840](https://doi.org/10.1016/j.comnet.2021.107840).
- [16] M. Al Lail, A. Garcia, và S. Olivo. “Machine learning for network intrusion detection—A comparative study”. In: *Future Internet* 15.7 (2023), p. 243. DOI: [10.3390/fi15070243](https://doi.org/10.3390/fi15070243).
- [17] Yung-Chung Wang, Yi-Chun Houn, Han-Xuan Chen, và Shu-Ming Tseng. “Network Anomaly Intrusion Detection Based on Deep Learning Approach”. In: *Sensors* 23.4 (2023), p. 2171. DOI: [10.3390/s23042171](https://doi.org/10.3390/s23042171).

- [18] L. Yang, A. Moubayed, I. Hamieh, và A. Shami. “Tree-Based Intelligent Intrusion Detection System in Internet of Vehicles”. In: *2019 IEEE Global Communications Conference (GLOBECOM)*. 2019, pp. 1–6. DOI: [10.1109/GLOBECOM38437.2019.9013892](https://doi.org/10.1109/GLOBECOM38437.2019.9013892).
- [19] L. Yang, A. Moubayed, và A. Shami. “MTH-IDS: A Multi-Tiered Hybrid Intrusion Detection System for Internet of Vehicles”. In: *IEEE Internet of Things Journal* 9.1 (2022), pp. 616–632. DOI: [10.1109/JIOT.2021.3084796](https://doi.org/10.1109/JIOT.2021.3084796).
- [20] L. Yang, A. Shami, G. Stevens, và S. DeRusett. “LCCDE: A Decision-Based Ensemble Framework for Intrusion Detection in The Internet of Vehicles”. In: *2022 IEEE Global Communications Conference (GLOBECOM)*. 2022, pp. 1–6. DOI: [10.1109/GLOBECOM48099.2022.10001280](https://doi.org/10.1109/GLOBECOM48099.2022.10001280).
- [21] Jose Luis Hernandez-Ramos, Georgios Karopoulos, Efstratios Chatzoglou, Vasileios Kouliaridis, Enrique Marmol, Aurora Gonzalez-Vidal, và Georgios Kambourakis. “Intrusion Detection Based on Federated Learning: A Systematic Review”. In: *ACM Computing Surveys* 57.12 (2025), pp. 1–65. DOI: [10.1145/3731596](https://doi.org/10.1145/3731596).
- [22] R. Baidar, S. Maric, và R. Abbas. *Hybrid deep learning–federated learning powered intrusion detection system for IoT/5G advanced edge computing network*. <https://arxiv.org/abs/2509.15555>. arXiv:2509.15555. 2025.
- [23] B. Williams và L. Qian. “Semi-supervised learning for intrusion detection in large computer networks”. In: *Applied Sciences* 15.11 (2025), p. 5930. DOI: [10.3390/app15115930](https://doi.org/10.3390/app15115930).
- [24] N. Albanbay, Y. Tursynbek, K. Graffi, R. Uskenbayeva, Z. Kalpeyeva, Z. Abilkaiyr, và Y. Ayapov. “Federated learning-based intrusion detection in IoT networks: Performance evaluation and data scaling study”. In: *Journal of Sensor and Actuator Networks* 14.4 (2025), p. 78. DOI: [10.3390/jsan14040078](https://doi.org/10.3390/jsan14040078).
- [25] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan, và Ahmad-Reza Sadeghi. “D²IoT: A Federated Self-learning Anomaly Detection System for IoT”. In: *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. 2019, pp. 756–767. DOI: [10.1109/ICDCS.2019.00080](https://doi.org/10.1109/ICDCS.2019.00080).

- [26] Lynda Boukela, Gongxuan Zhang, Méziane Yacoub, và các cộng sự. “A Near-Autonomous and Incremental Intrusion Detection System Through Active Learning of Known and Unknown Attacks”. In: *2021 International Conference on Security, Pattern Analysis, and Cybernetics (SPAC)*. 2021, pp. 374–379. DOI: [10.1109/SPAC53836.2021.9539947](https://doi.org/10.1109/SPAC53836.2021.9539947).
- [27] L. Atzori, A. Iera, và G. Morabito. “The Internet of Things: A survey”. In: *Computer Networks* 54.15 (2010), pp. 2787–2805. DOI: [10.1016/j.comnet.2010.05.010](https://doi.org/10.1016/j.comnet.2010.05.010).
- [28] Cisco. *Cisco Annual Internet Report (2018–2023) White Paper*. <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>. 2020.
- [29] R. Khan, S. U. Khan, R. Zaheer, và S. Khan. “Future Internet: The Internet of Things architecture, possible applications and key challenges”. In: *Proceedings of the 10th International Conference on Frontiers of Information Technology (FIT)*. 2012, pp. 257–260. DOI: [10.1109/FIT.2012.53](https://doi.org/10.1109/FIT.2012.53).
- [30] H. HaddadPajouh, A. Dehghantanha, R. M. Parizi, M. Aledhari, và H. Karimipour. “A survey on Internet of Things security: Requirements, challenges, and solutions”. In: *Internet of Things* 14 (2021), p. 100129. DOI: [10.1016/j.iot.2019.100129](https://doi.org/10.1016/j.iot.2019.100129).
- [31] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, và K.-Y. Tung. “Intrusion detection system: A comprehensive review”. In: *Journal of Network and Computer Applications* 36.1 (2013), pp. 16–24. DOI: [10.1016/j.jnca.2012.09.004](https://doi.org/10.1016/j.jnca.2012.09.004).
- [32] Varun Chandola, Arindam Banerjee, và Vipin Kumar. “Anomaly detection: A survey”. In: *ACM Computing Surveys* 41.3 (2009), pp. 1–58. DOI: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [33] Lukas Ruff, Jacob R. Kauffmann, Robert A. Vandermeulen, Grégoire Montavon, Wojciech Samek, Marius Kloft, Thomas G. Dietterich, và Klaus-Robert Müller. “A unifying review of deep and shallow anomaly detection”. In: *Proceedings of the IEEE* 109.5 (2021), pp. 756–795.

- [34] Manuel Lopez-Martin, Belen Carro, Antonio Sanchez-Esguevillas, và Jaime Lloret. “Conditional Variational Autoencoder for Prediction and Feature Recovery Applied to Intrusion Detection in IoT”. In: *Sensors* 17.9 (2017), p. 1967. DOI: [10.3390/s17091967](https://doi.org/10.3390/s17091967).
- [35] Khaled Alrawashdeh và Carla Purdy. “Toward an online anomaly intrusion detection system based on deep learning”. In: *Proc. 15th IEEE Int. Conf. on Machine Learning and Applications (ICMLA)*. 2016, pp. 195–200. DOI: [10.1109/ICMLA.2016.0040](https://doi.org/10.1109/ICMLA.2016.0040).
- [36] L. Husamaldin và N. Saeed. “Big Data Analytics Correlation Taxonomy”. In: *Information* 11.1 (2020), p. 17. DOI: [10.3390/info11010017](https://doi.org/10.3390/info11010017).
- [37] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, và Ion Stoica. “Apache Spark: A Unified Engine for Big Data Processing”. In: *Communications of the ACM* 59.11 (2016), pp. 56–65. DOI: [10.1145/2934664](https://doi.org/10.1145/2934664).
- [38] H. Karau, A. Konwinski, P. Wendell, và M. Zaharia. *Learning Spark: Lightning-fast big data analysis*. O’Reilly Media, 2015.
- [39] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, DB Tsai, Manish Amde, Sean Owen, Doris Xin, Reynold Xin, Michael J. Franklin, Reza Zadeh, Matei Zaharia, và Ameet Talwalkar. “MLlib: Machine Learning in Apache Spark”. In: *Journal of Machine Learning Research* 17.34 (2016), pp. 1–7.
- [40] A. L. Buczak và E. Guven. “A survey of data mining and machine learning methods for cyber security intrusion detection”. In: *IEEE Communications Surveys & Tutorials* 18.2 (2016), pp. 1153–1176. DOI: [10.1109/COMST.2015.2494502](https://doi.org/10.1109/COMST.2015.2494502).
- [41] N. Koroniotis, N. Moustafa, E. Sitnikova, và B. Turnbull. “Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset”. In: *Future Generation Computer Systems* 100 (2019), pp. 779–796. DOI: [10.1016/j.future.2019.05.041](https://doi.org/10.1016/j.future.2019.05.041).
- [42] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.
- [43] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32.

- [44] D. R. Cox. “The regression analysis of binary sequences”. In: *Journal of the Royal Statistical Society. Series B (Methodological)* 20.2 (1958), pp. 215–232.
- [45] Jerome H. Friedman. “Greedy function approximation: A gradient boosting machine”. In: *Annals of Statistics* 29.5 (2001), pp. 1189–1232.
- [46] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, và Tie-Yan Liu. “LightGBM: A highly efficient gradient boosting decision tree”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 3149–3157.
- [47] Andrew McCallum và Kamal Nigam. “A comparison of event models for naive Bayes text classification”. In: *AAAI Workshop on Learning for Text Categorization*. 1998.
- [48] Corinna Cortes và Vladimir Vapnik. “Support-vector networks”. In: *Machine learning* 20.3 (1995), pp. 273–297.
- [49] David E Rumelhart, Geoffrey E Hinton, và Ronald J Williams. “Learning representations by back-propagating errors”. In: *Nature* 323 (1986), pp. 533–536.
- [50] Tianqi Chen và Carlos Guestrin. “XGBoost: A scalable tree boosting system”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794.
- [51] Zhi-Hua Zhou. *Ensemble Methods: Foundations and Algorithms*. Chapman và Hall/CRC, 2012.
- [52] Patrik Goldschmidt và Daniela Chudá. “Network intrusion datasets: A survey, limitations, and recommendations”. In: *Computers & Security* 156 (2025), p. 104510. DOI: [10.1016/j.cose.2025.104510](https://doi.org/10.1016/j.cose.2025.104510).
- [53] Iman Sharafaldin, Arash Habibi Lashkari, và Ali A Ghorbani. “Toward generating a new intrusion detection dataset and intrusion traffic characterization”. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. 2018, pp. 108–116.
- [54] Canadian Institute for Cybersecurity and Communications Security Establishment. *CSE-CIC-IDS2018 dataset: A collaborative project between the Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC)*. <https://www.unb.ca/cic/datasets/ids-2018.html>. 2018.

- [55] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, và Konrad Rieck. “Dos and Don’ts of Machine Learning in Computer Security”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 3971–3988.
- [56] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, và Éric Totel. “Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes”. In: *Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers*. Springer, 2023, pp. 18–33. DOI: [10.1007/978-3-031-31108-6_2](https://doi.org/10.1007/978-3-031-31108-6_2).
- [57] Isabelle Guyon và André Elisseeff. “An introduction to variable and feature selection”. In: *Journal of Machine Learning Research* 3 (2003), pp. 1157–1182.
- [58] Ian T. Jolliffe. *Principal Component Analysis*. 2nd ed. Springer Series in Statistics. Springer, 2002. DOI: [10.1007/b98835](https://doi.org/10.1007/b98835).
- [59] Scott M. Lundberg và Su-In Lee. “A Unified Approach to Interpreting Model Predictions”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 4765–4774.
- [60] Gints Engelen, Vera Rimmer, và Wouter Joosen. “Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study”. In: *2021 IEEE Security and Privacy Workshops (SPW)*. 2021, pp. 7–12. DOI: [10.1109/SPW53761.2021.00009](https://doi.org/10.1109/SPW53761.2021.00009).
- [61] Claude Nadeau và Yoshua Bengio. “Inference for the Generalization Error”. In: *Machine Learning* 52.3 (2003), pp. 239–281.
- [62] Laurens D’hooge, Tim Wauters, Bruno Volckaert, và Filip De Turck. “Inter-dataset generalization strength of supervised machine learning methods for intrusion detection”. In: *Journal of Information Security and Applications* 54 (2020), p. 102564.
- [63] Marco Cantone, Claudio Marrocco, và Alessandro Bria. “Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study”. In: *IEEE Access* 12 (2024), pp. 144489–144508. DOI: [10.1109/ACCESS.2024.3472907](https://doi.org/10.1109/ACCESS.2024.3472907).
- [64] Mohanad Sarhan, Siamak Layeghy, và Marius Portmann. “Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection”. In: *Big Data Research* 30 (2022), p. 100359. DOI: [10.1016/j.bdr.2022.100359](https://doi.org/10.1016/j.bdr.2022.100359).

- [65] NVIDIA Corporation. *Jetson Orin Nano Super Developer Kit*. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/>. 2024.
- [66] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, và Lanyu Xu. “Edge computing: Vision and challenges”. In: *IEEE Internet of Things Journal* 3.5 (2016), pp. 637–646. DOI: [10.1109/JIOT.2016.2579198](https://doi.org/10.1109/JIOT.2016.2579198).
- [67] Salam Hamdan, Moussa Ayyash, và Sufyan Almajali. “Edge-Computing Architectures for Internet of Things Applications: A Survey”. In: *Sensors* 20.22 (2020), p. 6441. DOI: [10.3390/s20226441](https://doi.org/10.3390/s20226441).
- [68] Alessandro Sforzin, Félix Gómez Mármol, Mauro Conti, và Jens-Matthias Bohli. “RPiDS: Raspberry Pi IDS — A Fruitful Intrusion Detection System for IoT”. In: *Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom)*. 2016, pp. 440–448. DOI: [10.1109/UIC-ATC-ScalCom-CBDCoM-IoP-SmartWorld.2016.0080](https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCoM-IoP-SmartWorld.2016.0080).
- [69] Jay Kreps, Neha Narkhede, và Jun Rao. “Kafka: a Distributed Messaging System for Log Processing”. In: *Proceedings of the NetDB Workshop*. 2011.
- [70] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, và W. Philip Kegelmeyer. “SMOTE: Synthetic Minority Over-sampling Technique”. In: *Journal of Artificial Intelligence Research* 16 (2002), pp. 321–357.
- [71] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, và Piotr Dollár. “Focal Loss for Dense Object Detection”. In: *2017 IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2999–3007. DOI: [10.1109/ICCV.2017.324](https://doi.org/10.1109/ICCV.2017.324).